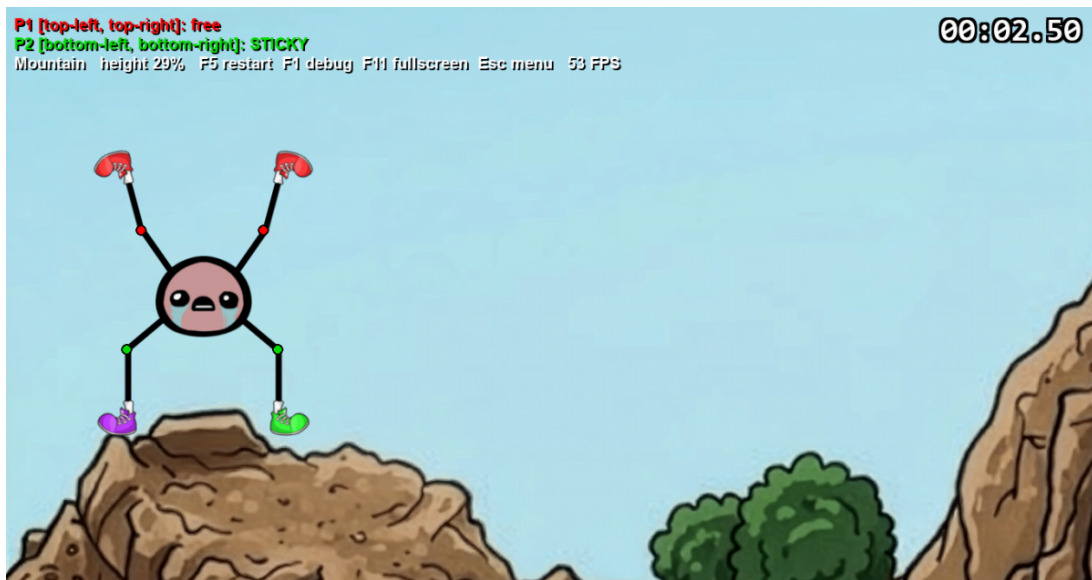


All For One

Code Guide

How every part of the game works - structure, data flow, physics, vision, and every file explained block by block with its full source



From The Group MeEeEe - ME461

06 October 2026

Contents

Part I - The big picture	4
1. What the game is	4
2. Running it	4
3. The files	5
4. How the parts fit together	6
5. One frame, start to finish	7
6. Coordinates, units and conventions	8
Part II - The core systems	10
7. Terrain and collision: the signed distance field	10
8. Physics of the character	11
9. From the camera to the legs	13
10. Drawing	14
11. The screens	15
12. Options and settings	18
13. The Python environment (Windows and Linux)	19
Part III - Every file, block by block	20
main.py (51 lines)	20
app.py (60 lines)	21
config.py (174 lines)	23
settings.py (148 lines)	26
display.py (141 lines)	29
minimap.py (50 lines)	32
menu.py (166 lines)	33
ui.py (136 lines)	36
game.py (305 lines)	39
scoreboard.py (70 lines)	44
test_scene.py (77 lines)	46
options_menu.py (169 lines)	48
maps.py (84 lines)	51
windmill.py (156 lines)	53
terrain.py (116 lines)	56
flag.py (47 lines)	58
viewport.py (44 lines)	59
character.py (329 lines)	60
assets.py (110 lines)	66
inputs.py (67 lines)	68

vision.py (384 lines)	70
player_state.py (111 lines)	76
preview.py (85 lines)	78
export_engine.py (17 lines)	80
download_models.py (45 lines)	80
setup.ps1 (99 lines)	81
setup.sh (119 lines)	83
requirements.txt (13 lines)	85
Part IV - Reference	86
14. Every setting in config.py	86
15. How do I...?	89
16. Glossary	89
Part V - How the AI models work	91
17. Pose detection: YOLO	91
18. Face landmarks: MediaPipe Face Landmarker	91
19. Limits to keep in mind	92

Part I - The big picture

1. What the game is

All For One is a co-op climbing game. Up to four people stand in front of one webcam and together control one cartoon character with four legs. Every player's arm (shoulder and elbow) drives one of the character's legs (hip and knee). There are no buttons: you push the character up the mountain by moving your arms, and you make its shoes sticky by opening your mouth. The goal is to touch the flag with the character's head as fast as possible.



The game window: the game view on the left; on the right the camera picture and the whole map with the character (yellow dot). P2's mouth is open, so the bottom shoes are sticky - purple = glued to the rock, green = sticky but still in the air.

Behind that simple idea there are four separate problems, and the code is organised around them:

- **Seeing the players** - find every person's body joints and mouth in the webcam picture (`vision.py`, using two AI models).
- **Turning bodies into commands** - convert arm positions into leg angles and mouth openings into sticky feet (`player_state.py`, `game.py`).
- **Physics** - make a character with rigid legs fall, stand, walk, jump and climb on an arbitrary drawn landscape (`terrain.py`, `character.py`).
- **Showing it all** - menus, the game view that follows the character, the camera preview, fullscreen (`display.py`, `menu.py`, `game.py`, `preview.py`).

2. Running it

Once per computer, run the setup (details in chapter 13). It creates a private Python environment with the right package versions and downloads the AI models:

System	Set up once	Then start the game
Windows	double-click <code>setup.bat</code>	double-click <code>run.bat</code>
Linux	<code>./setup.sh</code>	<code>./run.sh</code>

Options can be added after `run.bat` / `./run.sh` (or after `python main.py` inside the environment):

Option	What it does
(none)	Normal game with webcam 0.
--source 1	Use another webcam (or a video file path).
--keyboard	No camera: legs move with Q/A W/S E/D R/F U/J I/K O/L P/; keys, hold 1-4 to open a player's mouth. Good for testing physics.
--debug	Start with collision circles shown (same as pressing F1).

In the game: **F11** fullscreen, **F5** restart, **F1** collision circles, **Esc** back to the menu. On the title screen: **OPTIONS** (or O) to tune the game, **1P TEST** (or T) for the test screen.

3. The files

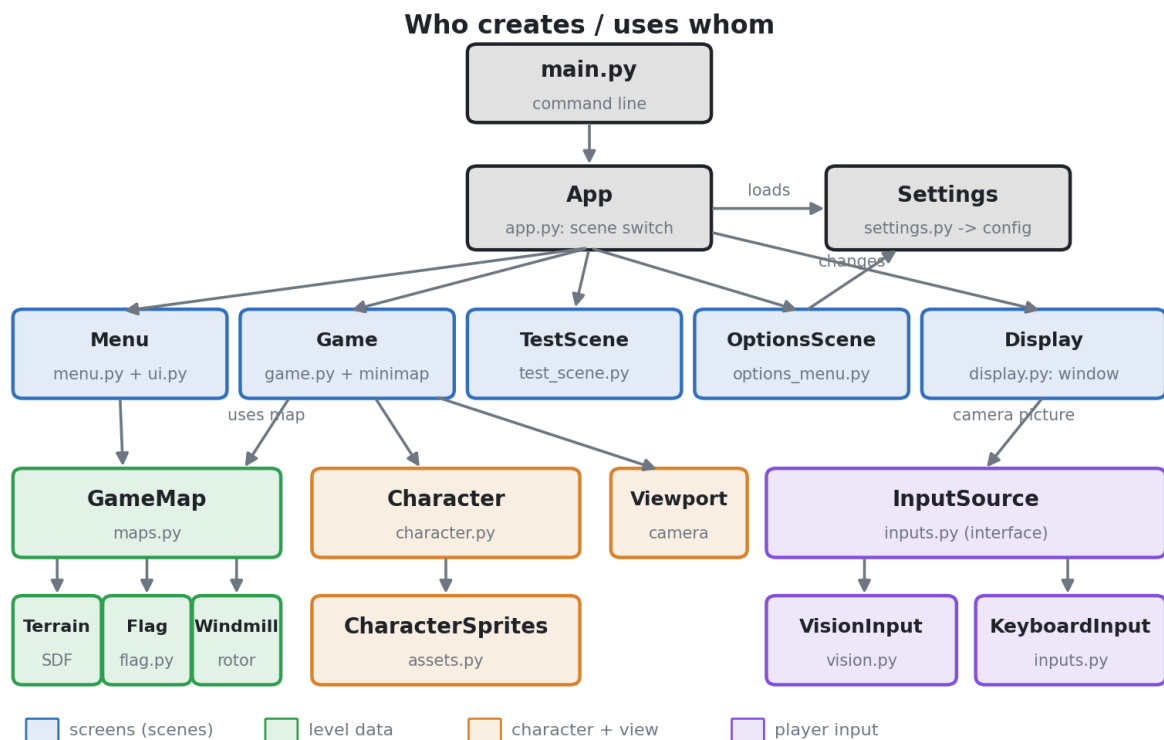
Every file has one job (the assignment asked for separate files per task and object-oriented code). The right column says which class or function to look for.

File	Job	Main contents
main.py	Entry point: read the command line, choose camera or keyboard	<code>main()</code>
app.py	Create the window and switch between the screens	<code>App</code>
config.py	Every tunable number in one place	constants
settings.py	Which settings players may change, their safe ranges, settings.json	<code>Setting</code> , <code>Settings</code>
options_menu.py	The Options screen (sliders, switches, live readout)	<code>OptionsScene</code>
display.py	The one window: game panel + camera and map panels, scaling, fullscreen	<code>Display</code>
minimap.py	The whole-map overview under the camera	<code>MiniMap</code>
menu.py	Title page, player-count page, map page	<code>Menu</code>
ui.py	Buttons, sliders, switches and outlined text	<code>Button</code> , <code>MapButton</code> , <code>Slider</code> , <code>Toggle</code> , <code>draw_text</code>
game.py	The level: loop, physics steps, HUD, timer, finish, name entry	<code>Game</code> , <code>apply_players</code>
scoreboard.py	Finish times with team names, saved in scores.json	<code>ScoreBoard</code>
test_scene.py	1-player test screen (no physics)	<code>TestScene</code>
maps.py	Load a map: scale it, find start point and flag	<code>GameMap</code> , <code>load_maps</code>
terrain.py	Map picture + collision (signed distance field)	<code>Terrain</code>
windmill.py	Windmill map: tower picture + spinning, solid rotor	<code>Windmill</code>
flag.py	The goal flag: placement, touch test, drawing	<code>Flag</code>
viewport.py	The game camera that follows the character	<code>Viewport</code> , <code>view_size</code>
character.py	Legs, kinematics, physics, drawing of the character	<code>Leg</code> , <code>Character</code>
assets.py	Character pictures: crop, scale, stretch, recolour, rotate	<code>PivotSprite</code> , <code>CharacterSprites</code>
inputs.py	Input interface + keyboard input	<code>InputSource</code> , <code>KeyboardInput</code>
vision.py	Webcam thread: pose, faces, players, mouth, head tilt	<code>VisionInput</code> and helpers
player_state.py	Player data + arm-angle maths	<code>PlayerState</code> , <code>limb_poses</code>
preview.py	Draws skeletons and lips on the camera picture	<code>PreviewRenderer</code>
export_engine.py	Builds the TensorRT engine	<code>script</code>

File	Job	Main contents
download_models.py	Downloads both AI models, reports GPU/CPU (run by the setup)	script
setup.ps1 / setup.bat	Windows setup: environment, packages, models	script
setup.sh	Linux setup: environment, packages, models, system checks	script
run.bat / run.sh	Start the game with the environment	script
requirements*.txt	Pinned package versions: game / optional GPU / optional docs tools	lists
tools/docs/	Rebuilds this PDF from the code (make_docs.py)	scripts
pose_and_face_test.py	Your original pose + face experiment (not used by the game)	script

The `assets/` folder holds the pictures: `background*.png` (sky and far scenery), `foreground*.png` (the rock you collide with - its transparent pixels are air), the character parts `head.png`, `leg-part.png`, `leg-part-with-shoe.png`, `character-demo.png` (menu picture) and `flag.png`. The windmill map adds `map3-windmillbase.png` (the tower, only a picture) and `map3-windmillblade.png` (the rotor, solid); `map3-fullmapdemo.png` shows how the map looks assembled. The two AI model files `yolo26n-pose.pt` and `face_landmarker.task` download themselves on first run.

4. How the parts fit together



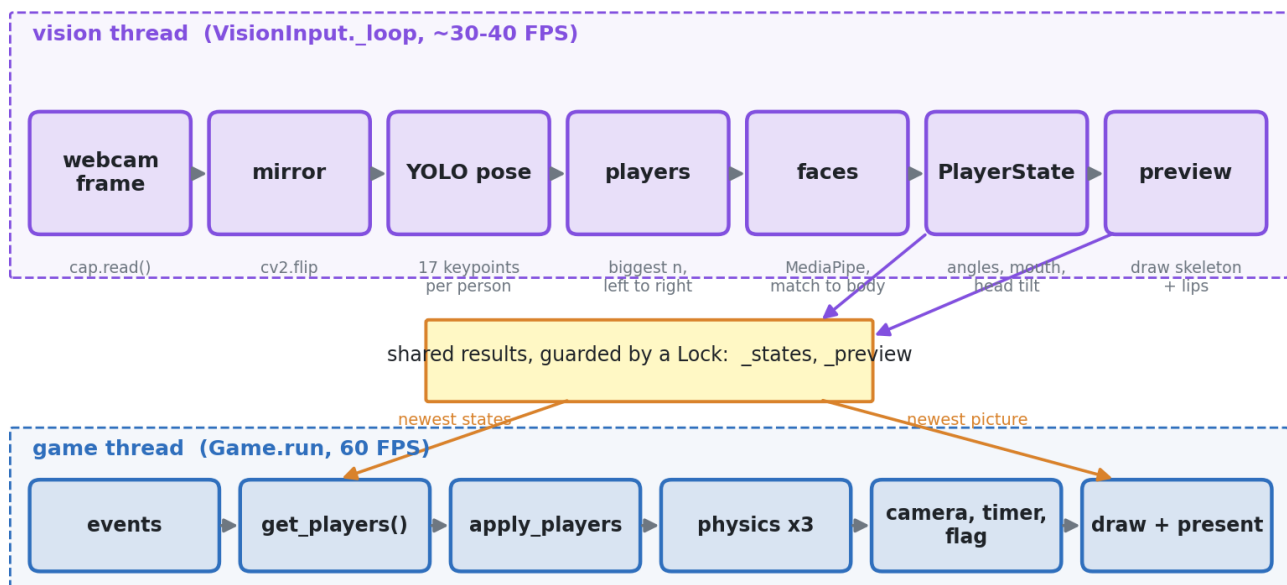
Who creates and uses whom. Arrows point from the user to the thing it uses.

Reading the diagram top-down: `main.py` creates an **input source** (camera or keyboard) and gives it to `App`. `App` creates the window (`Display`), loads the maps and creates the three **scenes** (Menu, Game, TestScene). A scene is a screen with its own loop. `App.run()` asks the menu what to do, runs that scene until it returns, and goes back to the menu.

The game scene uses a `GameMap` (which owns a `Terrain` for collisions, a `Flag` and, on the windmill map, a `Windmill`), a `MiniMap` (the overview under the camera) and a `ScoreBoard` (finish times in `scores.json`), builds a `Character` (which uses `CharacterSprites` for its pictures) and a `Viewport` (the game camera). It never talks to the webcam directly - it only calls `input.get_players()`. That is the key design idea: **the game does not know whether a camera or a keyboard is behind the input**. Both classes inherit from `InputSource` and offer the same methods (`get_players`, `get_preview`, `set_num_players`, `close`). This is called an *interface*, and it is why keyboard mode was trivial to add.

5. One frame, start to finish

Two things run at the same time (two *threads*). The **vision thread** reads the webcam and runs the AI models as fast as it can (about 30-40 times per second). The **game thread** runs the game at a steady 60 frames per second. They meet in one place: a small shared box with the newest results, protected by a *lock* so the game never reads a result while the vision thread is half-way through writing it.



The two loops and the shared results between them.

Why two threads? Running the AI models takes 20-110 ms per picture. If the game waited for them every frame, it would stutter at 9-30 FPS. With a separate thread the game always runs smoothly and simply uses the newest available result.

Start-up, in order

1. `main()` parses the command line and creates `VisionInput` - this loads the pose model (picking TensorRT, GPU or CPU), the face model, opens the webcam and starts the vision thread.
2. `App.__init__` starts pygame, applies the player's saved settings from `settings.json` (chapter 12) and creates the `Display` (the window).
3. `load_maps()` loads every map in `config.MAPS`: scales the pictures, computes the collision field, places the flag and finds the start point (about 2 seconds in total).
4. The three scenes are created. `App.run()` shows the menu.
5. The player picks PLAY, a player count and a map; `Game.run(players, map)` starts the level.

Every game frame (Game.run)

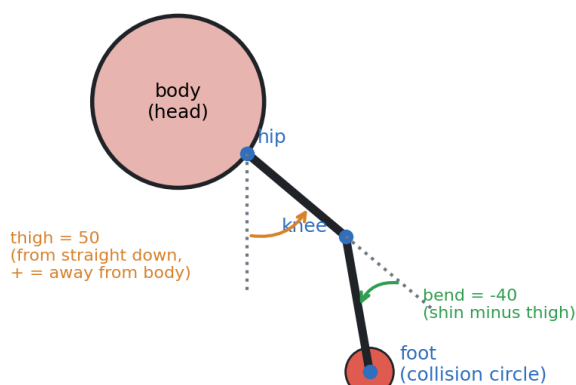
1. `clock.tick(60)` waits so frames are 1/60 s apart and returns the real time since the last frame (`dt`).
2. Keyboard/window events: Esc, F5, F1, F11, closing the window.
3. `input.get_players()` returns one `PlayerState` per player (limb angles, mouth, head tilt).
4. `apply_players` copies each controlling limb's angles into its leg as a *target*, and sets each leg's *sticky* flag from the mouth.
5. Physics: `character.update()` three times with `dt/3` each (substeps).
6. The viewport glides towards the character; the timer runs; the flag touch is checked.
7. Drawing: map part, flag, character, knee dots, texts, timer, and `display.present()` shows it.

6. Coordinates, units and conventions

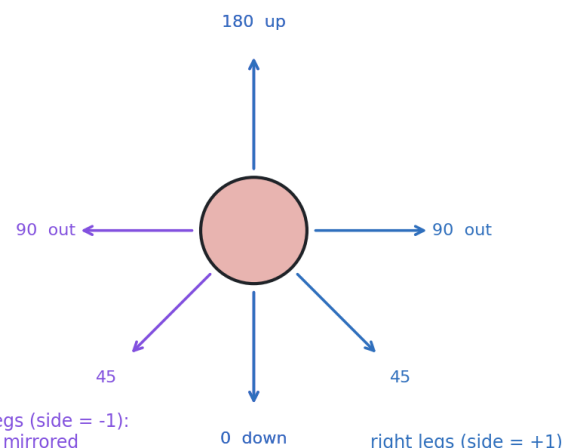
Most confusion in game code comes from coordinates. These are the conventions used everywhere here:

- **Map pixels** are the unit of the world. All maps are scaled so the mountain map is 2164 px tall (`MAP_HEIGHT`); positions, speeds (px/s) and gravity (px/s²) are in map pixels.
- **y points down** (screen convention). Gravity is *+y*; "up" means a smaller y. The HUD height % is computed as `map height - y`.
- **Three sizes of picture**: the *map* (e.g. 4096 x 2164), the *canvas* (1365 x 721, the size the scenes draw at = what the viewport shows) and the *window* (whatever fits your screen). `display.present()` scales canvas to window; `display.to_canvas()` converts mouse clicks back.
- **Screen position = map position - viewport position**. That is why every `draw()` gets an *offset*.
- **Angles are degrees**. Leg and arm angles use the *outward* convention below.
- **Colours**: pygame uses (R, G, B), OpenCV uses (B, G, R). `config.PLAYER_COLORS_BGR` is the master list and the RGB list is made by reversing each tuple.

right leg: angles in the 'outward' convention



the same thigh angle points outward on both sides



Left: a leg is two angles. Right: left legs are mirrored so 'away from the body' is positive on both sides.

A leg is fully described by two numbers: **thigh** = the hip angle measured from straight down (0 = hanging, 90 = sideways, 180 = up) and **bend** = how much the shin turns relative to the thigh. The *same* two numbers are measured on the player's arm (shoulder angle and elbow angle), which is why an arm can drive a leg directly: arm hanging down -> leg hanging down, arm raised -> leg raised.

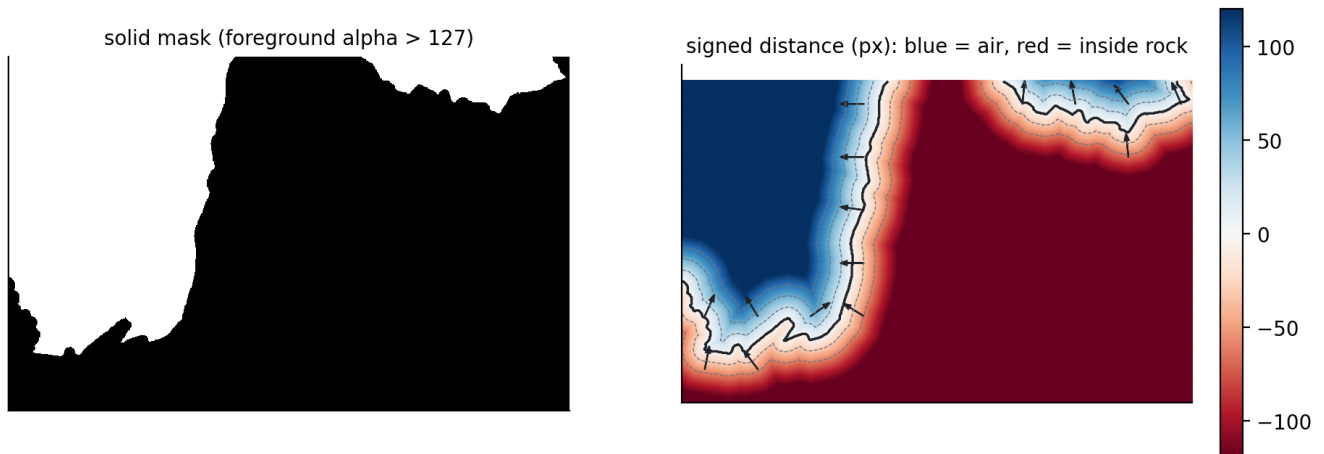
To draw or compute a leg on screen, the outward angle is multiplied by the leg's `side` (-1 left, +1 right) to get a normal screen rotation (`Leg.screen_angles`).

Part II - The core systems

7. Terrain and collision: the signed distance field

The landscape is a drawing, not a set of boxes or polygons, so collision must work on pixels. The foreground picture's transparency decides what is solid: a pixel more than half opaque is rock.

Terrain turns that yes/no mask into a **signed distance field (SDF)**: a table with one number per pixel telling how far that pixel is from the nearest rock surface - positive in the air, negative inside rock, zero on the surface.



A part of the mountain. Right: the distance field; the black line is distance 0 (the surface), dashed lines are every 30 px, arrows are the normals (the way out).

OpenCV computes this table once per map with `cv2.distanceTransform` (run on the air and on the rock, then subtracted). After that, three questions are cheap lookups:

- **Is this point inside rock, and how deep?** `terrain.distance(x, y)`. A foot circle of radius r is in the ground when `distance < r`, and it is $r - \text{distance}$ deep.
- **Which way is out?** `terrain.normal(x, y)` - the direction in which the distance grows fastest (its gradient), found by sampling the distance 2 px left/right and up/down.
- **Where is the ground under the sky in this column?** `terrain.ground_y(x)`, used to place the flag.

```
distance(x, y) = bilinear blend of the 4 table cells around (x, y)
normal = normalize( d(x+2,y) - d(x-2,y) , d(x,y+2) - d(x,y-2) )
```

Bilinear blending means the distance changes smoothly between pixels instead of in 1 px steps, which keeps the physics from jittering. Points outside the map are handled specially: beyond the left and right edges counts as ever-deeper rock (invisible walls), and above the top edge the top row continues upward - sky stays sky, but a solid top border (like the cave's) becomes rock that gets deeper going up, so nothing can escape through it.

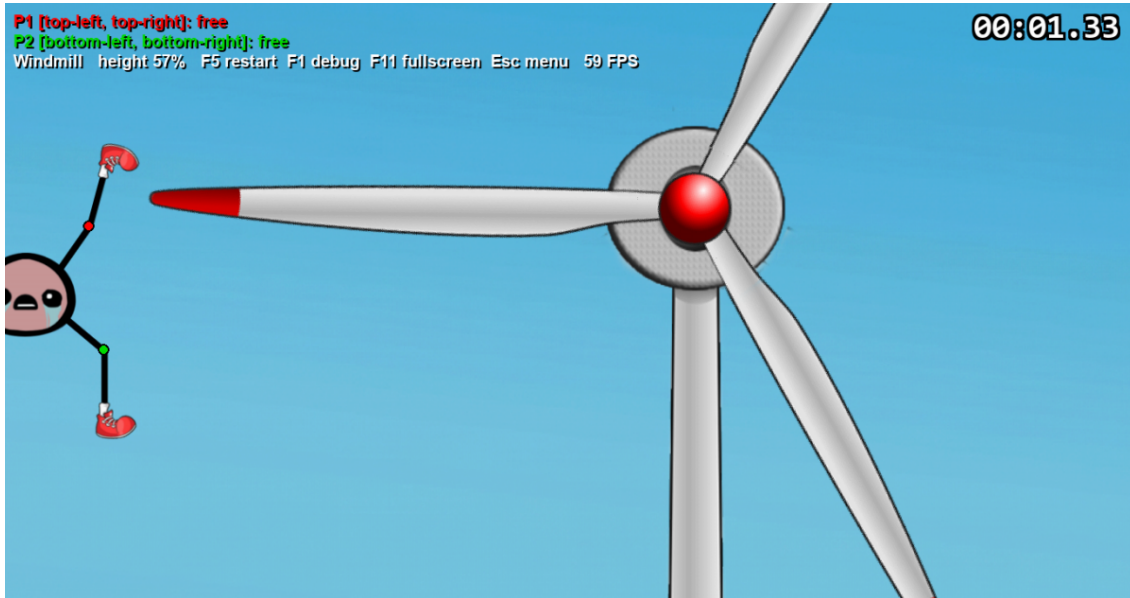
Moving parts: the windmill rotor

The windmill map has a solid part that moves. Its rotor gets its own distance table, made once for the rotor standing still. To ask how far a point is from the rotor *now*, the point is turned back around the hub by the rotor's current angle and looked up in that still table - turning the question is much cheaper than rebuilding the table 180 times a second. `Terrain.distance` returns the nearest of the rock and every moving part, so the physics needs no special code to collide with the rotor.

`Terrain.mover_at` tells which moving part a point touches.

```
local point = hub + rotate(point - hub, -angle)
rotor distance(point) = still table at the local point
```

The rotor turns clockwise at `rps = 0.25` turns per second (set per map in `MAPS`). The tower is just part of the map picture and has no collision. Drawing the rotor means rotating a big picture every frame, so it is cut into four pieces (three blades and the hub); only pieces inside the view are rotated. The tower and the rotor are drawn after the character, so the character passes behind them.



The windmill map: the rotor is solid and turning; the map panel shows it too.

8. Physics of the character

The character is deliberately simple: **only the body (the head) has a position and a velocity**. It cannot rotate. The legs have no mass - each leg is just two angles that turn towards the angles the player asks for, at most 420 degrees per second. Where a foot is follows from the body position plus the two angles (*kinematics*, `Leg.joints`):

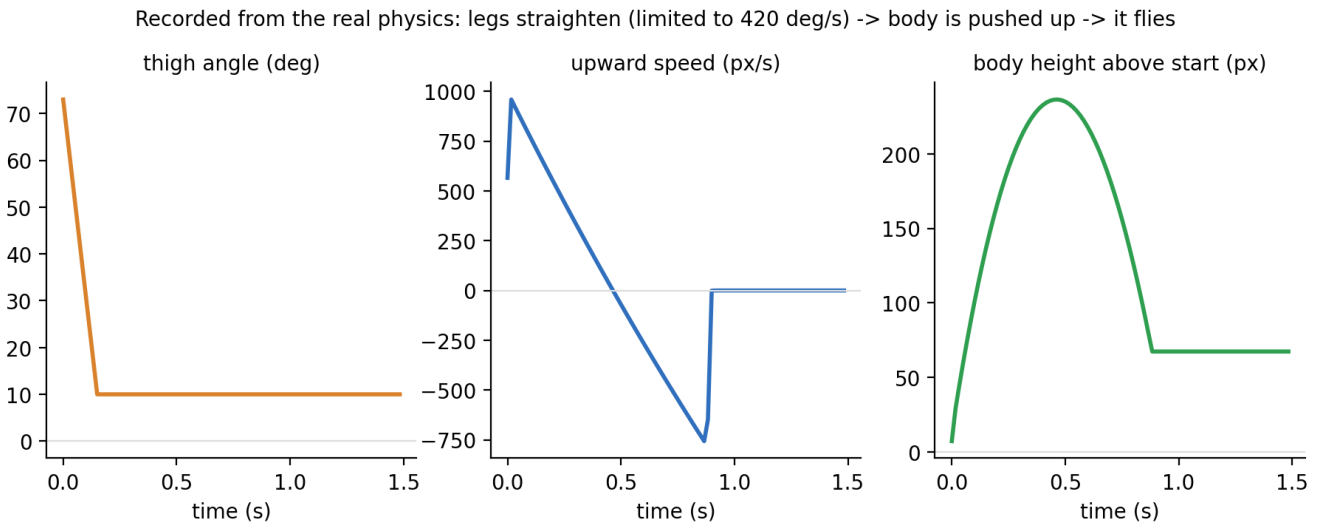
```
hip = body + hip_offset
knee = hip + rotate((0, thigh_length), thigh angle)
foot = knee + rotate(foot_offset, shin angle) # shin angle = thigh + bend
```

The method is **position-based dynamics** (PBD): instead of computing forces, each step first moves the body freely, then *corrects its position* until all contacts are satisfied, and finally derives the velocity from how far the body actually moved. One `_step` does:

1. **Sticky feet** - a sticky foot that is within 8 px of the rock gets *glued*: its position is remembered as an anchor. A foot that stops being sticky lets go.
2. **Predict** - add gravity to the velocity, apply a little air drag, move the body by velocity x dt.
3. **Solve** (4 passes) - for every foot: if glued, move the body so the foot goes back towards its anchor; otherwise, if the foot is inside rock, push the body out along the normal and apply friction. Then push the head out of the rock. Each correction can upset another, so the passes repeat (the Gauss-Seidel method).
4. **Rock has the last word** - two extra passes make sure no glued foot is more than 6 px deep.
5. **Velocity** - `velocity = (new position - old position) / dt`, then limits (head friction, max speed, stay inside the map). The 1500 px/s limit applies to the whole speed whenever something pushed or pulled the body; in free flight only the falling speed is limited, so a sideways fling keeps its sideways speed instead of drifting down slowly.

Why this gives jumping and walking for free

Jumping: if a planted leg straightens quickly, its foot would go into the ground. The solver pushes the body up instead, by exactly the amount the foot would have sunk. Because velocity is *measured* from the movement, the body now has upward speed and keeps flying after the foot leaves the ground. The recording below comes from the real physics:



A recorded jump on the mountain: the thigh angle drops (leg straightens), the body gets about 950 px/s upward, rises ~240 px and lands.

Walking: when a free foot first touches walkable ground (flatter than 55 degrees) it remembers that point (**grip**). While it stays down, the solver does not let the foot slide along the surface (*static friction*). So if the player sweeps the leg backwards, the foot cannot move - the body moves forward instead. On steeper slopes there is no grip and feet slide, which is what makes walls need sticky feet.

Climbing: a glued foot works in both directions - it can push *and pull* the body. Hanging from a ledge, a player can bend the glued leg to pull the body up. If the glue is stretched more than 40 px it tears off.

Riding the rotor: a foot glued to the rotor remembers that (**anchor_on**). At the start of every physics step its anchor is turned with the rotor (**Windmill.carry**), so the body is pulled along. Because velocity is measured from movement, letting go flings the character with the speed it had. Free (non-sticky) feet only grip the rotor where it is flatter than **ROTOR_WALKABLE_SLOPE** (10 degrees instead of 55), so without sticky feet the character slips off the blades quickly. A blade that overlapped a free foot or the head deeply would push it out in one step - a huge kick - so leaving a blade may be at most **ROTOR_PUSH_SPEED** (400 px/s) faster than the blade moves there (**Windmill.velocity_at**).

Keeping feet out of the rock while glued

With a foot glued, the body is pinned and cannot always move out of the way, so a leg could be forced into the rock. **Character.update** therefore checks every step: if a moving leg pushed its own foot deeper than allowed (3 px for free feet, 6 px for sticky feet), the step is undone and redone with that leg held still - the leg stops at the rock. A glued foot pushed in by *another* leg is held at 6 px by the extra solver passes.

Substeps

Physics runs three times per frame with a third of the time each. Smaller steps mean feet move less per step, so they cannot jump through thin rock, and the solver stays stable.

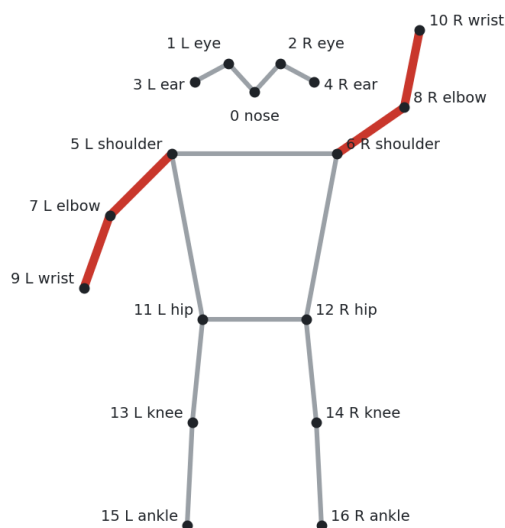
Setting	Value	Effect when you raise it
GRAVITY	2000 px/s ²	Heavier, falls faster, lower jumps
LEG_MAX_SPEED	420 deg/s	Legs react faster, kicks and jumps get stronger
MAX_SPEED	1500 px/s	Allows faster flights
WALKABLE_SLOPE	55 deg	Feet grip on steeper slopes without sticking
SOLVER_ITERATIONS	4	Stiffer, more exact contacts (slower)
STICKY_BREAK_DIST	40 px	Glue holds longer before tearing off
PHYSICS_SUBSTEPS	3	More stable, slower

9. From the camera to the legs

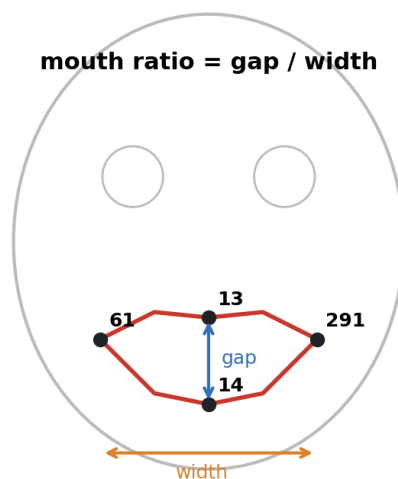
`VisionInput._loop` runs in the background thread. For each webcam frame it does:

1. **Mirror** the picture so it behaves like a mirror (your right hand appears on the right).
2. **Pose**: YOLO finds every person and 17 body keypoints each, with a confidence per keypoint.
3. **Number the players** (`assign_players`): take the n biggest people (closest to the camera) and number them left to right. With fewer people than players, the picture is split into n vertical strips and you get the number of the strip you stand in.
4. **Faces**: MediaPipe finds face landmarks on the whole frame. `match_faces` gives each face to the person whose head keypoints are closest in pixels (closest pairs first). A player without a face gets a second try on a crop around their head.
5. **Per player**: limb angles (`limb_poses`) smoothed by `AngleSmoother`, the mouth state (`MouthTracker`) and, with one player, the head tilt (`HeadTilt`).
6. **Preview**: draw the skeletons and lips on a copy of the frame (`PreviewRenderer`).
7. Store the results under the lock.

COCO 17 keypoints (YOLO pose output);
controlling arms coloured, rest grey



4 of the 478 face-mesh points used for the mouth



Left: the 17 keypoints of one body. Right: the four face points used for the mouth.

Arm angles

For an arm, the shoulder angle is the direction of shoulder -> elbow and the elbow angle is the direction of elbow -> wrist, both measured from 'down' and corrected by the torso direction (shoulders -> hips) so leaning does not swing the legs. Left limbs are mirrored (multiplied by -1) to

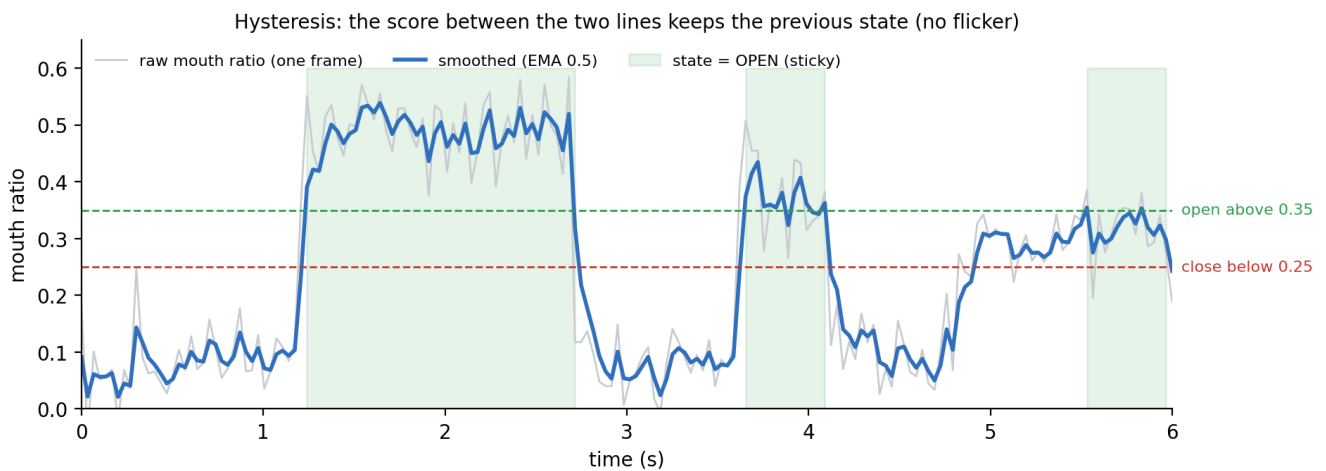
get the outward convention. Sides come from where the joints are on screen, not from the model's left/right labels, because on a mirrored picture those can be swapped.

```
heading(v) = atan2(v.x, v.y) # angle from straight down
thigh = side * wrap(heading(elbow - shoulder) - torso)
bend = wrap(side * wrap(heading(wrist - elbow) - torso) - thigh)
```

Keypoints jitter by a few pixels every frame. **AngleSmoother** applies an *exponential moving average* (EMA): each new value moves the smoothed value 60% of the way. It is *wrap-safe*: going from 179 to -179 degrees is treated as a 2 degree step, not a 358 degree swing.

Mouth: open or closed

The mouth ratio is the gap between the lips divided by the mouth width. Dividing by the width makes it the same whether the face is near (big) or far (small). The ratio is smoothed, then a **hysteresis** decides the state: it opens only above 0.35 and closes only below 0.25. Between the two lines the state stays as it was, so a ratio hovering around one value cannot make the feet flicker.



Hysteresis on a made-up recording: the 0.30 bump at $t = 3.8$ s is not enough to open; noise around the thresholds does not toggle the state.

If the face is lost for a moment, the last state is kept for 0.5 s before it falls back to closed.

Head tilt (1 player)

With one player, both legs belong to the same mouth, so the sideways tilt of the head picks which foot the open mouth makes sticky. The roll angle is the slope of the line between the ears (or the eyes if an ear is hidden). Tilting right (right ear lower) beyond 10 degrees selects the right foot, tilting left the left foot; in between, the last choice is kept.

10. Drawing

Sprites that rotate around a joint

All character pictures share one 500 x 500 canvas, and the joint positions were measured on it (`config.LEG_PIVOT` etc.). `PivotSprite.draw` rotates an image around its joint: pygame rotates around the image centre, so the code rotates the vector from the joint to the centre by the same angle and places the rotated image's centre there - the joint ends up exactly where it should.

Which way a toe points: the shoe picture points right while the shin hangs down, so left legs use a mirrored copy to point outward. The two upper legs are usually raised, which turns the shoe upside down and its toe inward - so the legs listed in `config.MIRRORED_SHOES` (the upper two) use the

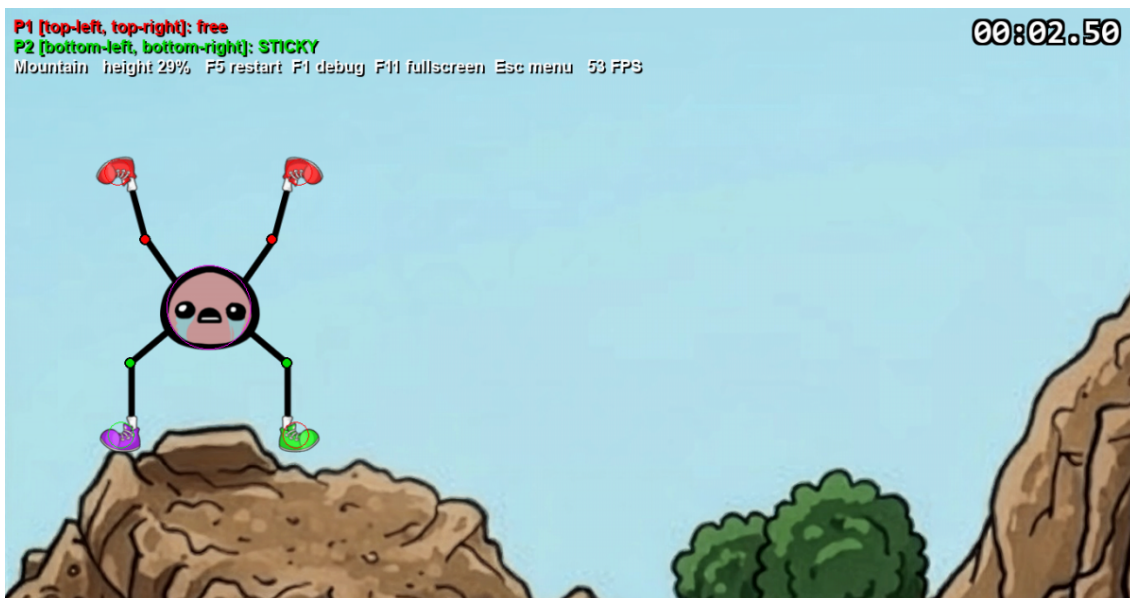
opposite copy, and their toes point outward too, like in `character-demo.png`. The foot's collision circle follows the same choice (`Leg.shoe_side`).

Pictures are prepared once: cropped to their content (so rotating is cheap), scaled, mirrored for left legs (toes point outward) and recoloured. The recolour filter takes clearly red pixels and moves their 'redness' into other channels, which keeps the shading: green = sticky, purple = glued. For the 1-player long legs, `stretch_leg` stretches only the black stick part and moves the shoe down unchanged.

One window, three panels

Every scene draws onto the same canvas (1365 x 721). `Display.present()` fills the window, scales the canvas into the left panel, draws the newest camera picture at the top of the right column and the map overview under it, and calls `pygame.display.flip()`, which shows the finished frame all at once (no flicker). F11 recreates the window in fullscreen and recomputes the layout.

The map panel (`MiniMap`) shows the whole map with its flag, a white frame around the part the game view shows and a yellow dot for the character. The shrunk map picture is made once per panel size; only the frame, the dot and the windmill rotor are drawn each frame. On the other screens the panel shows a short note.



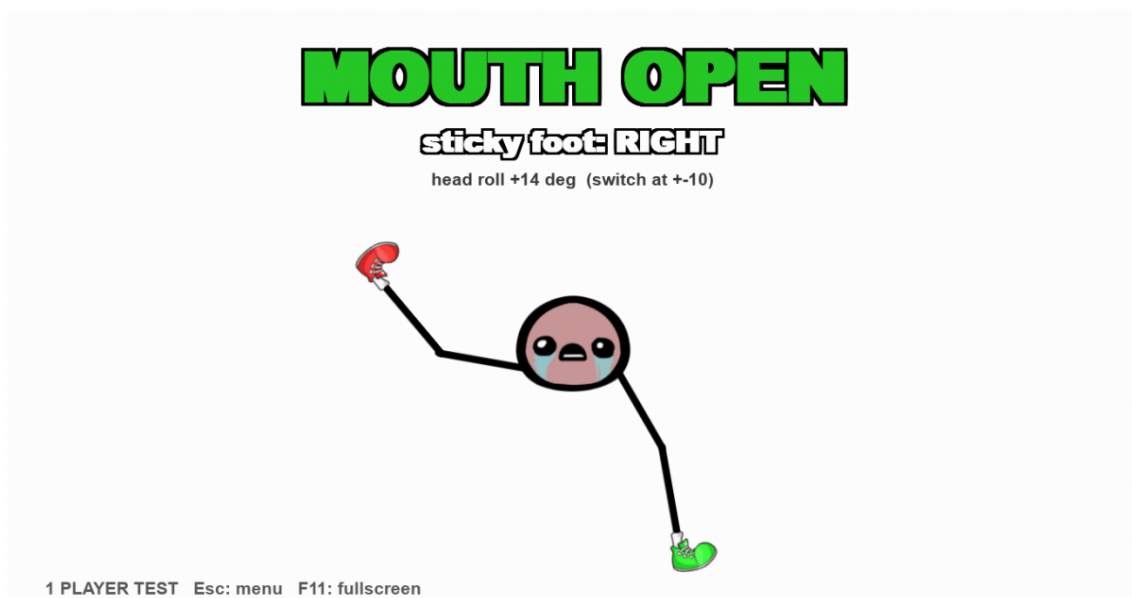
F1 debug view: magenta = head circle, feet circles coloured green (glued), yellow (touching) or red (in the air).

11. The screens

The menu has three pages in one loop (`page` = title, players, maps). PLAY leads to the player count, then to the map. 1P TEST opens the test screen. Every button position is a fraction of the canvas size.



The map page. Each picture is the whole map shrunk, with its flag; the cards are laid out side by side with equal gaps.



The 1-player test screen: no physics, the body stays still and the legs follow the arms; mouth state in big letters.



Touching the flag with the head stops the timer and asks for the team name.

The score board

After the flag, the team types a name (up to 16 characters, any keyboard layout - the letters arrive as `TEXTINPUT` events). **Enter** saves, **Esc** skips saving. The board then shows the 8 fastest times on this map with this many players; the new run is gold, and if it is not in the top 8 it takes the last line with its real place. Enter goes to the menu, F5 plays again.

`ScoreBoard` keeps every saved run in `scores.json` (name, map, player count, time, date) and never deletes any. It writes a temporary file first and then swaps it in, so a crash cannot leave a half-written file. A broken file is renamed to `scores.json.broken` and the board starts empty. Like `settings.json`, the file is not in git.



The score board after saving (demo names).

12. Options and settings

The **OPTIONS** button on the title screen lets players tune the game without touching code: camera, mouth, head tilt, controls, physics and fullscreen. Changes apply at once and are remembered in `settings.json` next to the game.



The Mouth tab. The green line shows each player's live mouth score, so the thresholds can be set while watching it. A gold * marks a changed value.

How a setting reaches the game

There is no separate settings system inside the game: every part already reads its numbers from `config` (for example `config.GRAVITY` in the physics step). A `Setting` in `settings.py` names one of those values, and `Settings.set()` simply writes the new value into `config` - so the next physics step, the next mouth check or the next arm reading already uses it.

1. Start-up: `App` creates `Settings`, which first remembers the values written in `config.py` as the defaults, then `load()` reads `settings.json` and applies it.
2. In the Options screen a slider or switch calls `settings.set(key, value)`.
3. Leaving the screen (BACK, Esc or closing the window) calls `save()`, which writes only the values that differ from the defaults. **RESET TO DEFAULTS** puts the `config.py` values back.

For this to work a value must be *read when used*, not copied once at start-up. Two places did copy: `limb_poses` (joint confidence) and `AngleSmoother` (arm smoothing); they now read `config` each time.

Why it cannot break the game

- Only safe values are offered. Things like the camera number, model files, sprite geometry or solver internals are not in the list, because a wrong value there stops the game.
- Every number has a minimum, maximum and step (`Setting.clamp`); anything outside is pulled back in.
- The two mouth thresholds are kept in order: 'close' always stays at least 0.05 below 'open'.
- A missing, broken or hand-edited `settings.json` never crashes the game: wrong types and unknown names are ignored, numbers are clamped, unreadable files fall back to the defaults.
- `settings.json` is not in git: every computer keeps its own tuning.

Tab	Settings
Camera	mirror on/off, joint confidence needed

Tab	Settings
Mouth	open threshold, close threshold, smoothing, keep state when the face is lost
Head tilt	switch angle, swap sides (1 player)
Controls	leg speed, arm smoothing
Physics	gravity, grip on slopes, sticky-foot tear-off distance
Display	fullscreen (applied at once, remembered)

13. The Python environment (Windows and Linux)

The game depends on a few big libraries (PyTorch, Ultralytics, MediaPipe, OpenCV, pygame) that change often. To make the game install the same way on every computer, it runs in its own **virtual environment**: a private folder `.venv` with exactly the tested package versions, separate from any other Python projects on the computer.

What the setup script does

1. Finds **Python 3.12** - exactly that version, because newer ones (3.13, 3.14) do not have mediapipe/PyTorch packages yet. If it is missing, Windows offers to install it with `winget`; Linux offers to get it with `uv`, which downloads a private Python 3.12 for your user (no `sudo`, the system Python is not changed).
2. Creates `.venv` with Python's built-in `venv` module (or updates it if it exists). A `.venv` made with another Python version is deleted and rebuilt with 3.12.
3. Installs **PyTorch first**: the CUDA build when an NVIDIA GPU is found (`nvidia-smi`), otherwise the small CPU build. It must come first, because `ultralytics` would otherwise pull a default build.
4. Installs the pinned game packages from `requirements.txt`.
5. Optionally (NVIDIA only, asks y/N) installs TensorRT and builds the engine (`export_engine.py`).
6. Runs `download_models.py`: downloads both AI models and reports whether the GPU will be used.

File	Contents
<code>requirements.txt</code>	pygame, numpy, OpenCV, mediapipe, ultralytics (exact versions). Both OpenCV packages are pinned to the same version: they install the same <code>cv2</code> module and would overwrite each other otherwise.
<code>requirements-gpu.txt</code>	TensorRT (optional, NVIDIA)
<code>requirements-dev.txt</code>	tools to rebuild this PDF (not needed to play)

Linux specifics

- `setup.sh` checks things Windows does not need: that the `venv` module is installed (`sudo apt install python3.12-venv`), that OpenCV finds its system library (`sudo apt install libgl1 libglu1-mesa-dev`), and that your user may open the webcam (`sudo usermod -aG video $USER`, then log in again).
- Fonts: the game asks for a list of fonts (`config.FONT_*`) and uses the first one installed - Arial and Consolas on Windows, DejaVu or Liberation on Linux. If no emoji font exists, the sheep in the menu is simply left out instead of showing an empty box.
- NVIDIA on Linux: the CUDA 13 build of PyTorch needs driver 580 or newer; with an older driver `download_models.py` says so and the game runs on the CPU. The TensorRT engine must be built on each computer anyway (`export_engine.py`), because it only works on the GPU it was made for.
- `.gitattributes` keeps `.sh` files with Linux line endings even when committed from Windows.

Part III - Every file, block by block

Each file below starts with what it is for, then a table that walks through the file from top to bottom (the line numbers match the real file), then the complete source with its comments. Read the table next to the listing: the table says *why*, the comments in the code say *what*.

main.py (51 lines)

The entry point. It only reads the command line, creates the right input source and starts the app.

Lines	What happens there
1-12	Module docstring: how to start the program and what to install.
13-18	Imports. <code>KeyboardInput</code> is cheap to import; the camera input is imported later.
21-26	Warn if this is not Python 3.12 (the version the packages are pinned for).
28-34	Declare the four command-line options; <code>parse_args</code> reads what the user typed.
36-44	Pick the input source. The vision module is imported only here because loading <code>torch/ultralytics/mediapipe</code> takes seconds.
45	Create the app with that source and run it until the player quits.
48-51	Standard Python idiom: run <code>main()</code> only when the file is started directly.

Full source

```

1  """All For One - entry point.
2
3      python main.py                # webcam 0
4      python main.py --source 1     # another webcam
5      python main.py --keyboard     # no camera, keyboard debug controls (see inputs.py)
6      python main.py --debug        # draw collision circles
7
8  Install: pip install pygame ultralytics mediapipe opencv-python
9
10 This file only reads the command line, picks an input source (camera or
11 keyboard) and hands it to App, which runs everything else.
12 """
13 import argparse
14 import sys
15
16 import config
17 from app import App
18 from inputs import KeyboardInput
19
20
21 def main():
22     # The packages are pinned for Python 3.12 (newer Pythons lack mediapipe / PyTorch
23     # builds). run.bat / run.sh use the .venv made by the setup, which is 3.12.
24     if sys.version_info[:2] != (3, 12):
25         print(f"Warning: this is Python {sys.version.split()[0]}, the game is made for 3.12.\n"
26               "Start it with run.bat / ./run.sh (after setup.bat / ./setup.sh).")
27
28     # Describe the command-line options; argparse also builds '--help' from this.
29     ap = argparse.ArgumentParser()
30     ap.add_argument("--source", default=config.CAMERA_SOURCE, help="webcam index or video path")
31     ap.add_argument("--model", default=config.POSE_MODEL)
32     ap.add_argument("--keyboard", action="store_true", help="play without a camera")
33     ap.add_argument("--debug", action="store_true")
34     args = ap.parse_args()
35
36     # Both input sources have the same methods (see inputs.InputSource), so the
37     # rest of the game never needs to know which one it got.
38     if args.keyboard:
39         source = KeyboardInput()
40     else:
41         # Imported here, not at the top: loading torch/ultralytics/mediapipe takes
42         # seconds and is not needed in keyboard mode.
43         from vision import VisionInput # heavy imports only in camera mode
44         source = VisionInput(args.source, args.model)
45     App(source, debug=args.debug).run()
46
47
48 # Run main() only when this file is started directly (python main.py),
49 # not when another file imports it.
50 if __name__ == "__main__":
51     main()

```

app.py (60 lines)

Owens the window and the three scenes, and switches between them. Each scene's `run()` returns where to go next.

Lines	What happens there
1-6	Docstring explaining scenes.
7-17	Imports of every scene and helper.
20-24	Constructor: start pygame, title the window.
25-28	Load <code>settings.json</code> into <code>config</code> before anything is built (chapter 12).
30-38	Create the display first (images can only be converted for fast drawing once a window exists), then load maps, then the scenes.
40-60	The switchboard loop: menu -> chosen scene (game, test or options) -> back to menu, until quit. The <code>finally</code> block always stops the camera thread and closes pygame, even after a crash.

Full source

```

1  """App: owns the single window and switches between menu, game and test scenes.
2
3  A "scene" is a screen with its own loop (menu, game, 1P test). Each scene's
4  run() keeps drawing frames until the player leaves it, then returns what
5  should happen next; App.run() is the switchboard between them.
6  """
7  import pygame
8
9  import config
10 from display import Display
11 from game import Game
12 from menu import Menu
13 from maps import load_maps
14 from options_menu import OptionsScene
15 from settings import Settings
16 from test_scene import TestScene
17 from viewport import view_size
18
19
20 class App:
21     def __init__(self, input_source, debug=False):
22         pygame.init() # start pygame: video, fonts, events
23         pygame.display.set_caption(config.GAME_TITLE) # window title bar text
24         self.input = input_source
25         # Saved player settings go into config first, so everything built below
26         # (e.g. the window in fullscreen or not) already uses them.
27         self.settings = Settings()
28         self.settings.load()
29
30         # The window must exist before images can be converted for fast drawing,
31         # so Display comes first, then the maps (which load and convert images).
32         self.display = Display(view_size(), input_source)
33         self.maps = load_maps()
34         # Every scene shares the same display (window) and input source.
35         self.menu = Menu(self.display, self.maps, input_source)
36         self.game = Game(self.display, input_source, debug)
37         self.test = TestScene(self.display, input_source)
38         self.options = OptionsScene(self.display, self.maps, input_source, self.settings)
39
40     def run(self):
41         # try/finally: even if the game crashes, the camera thread is stopped and
42         # the window is closed properly.
43         try:
44             while True:
45                 choice = self.menu.run() # blocks until the player picks something
46                 if choice is None: # Esc / window closed on the title page
47                     break
48                 if choice[0] == "test":
49                     scene = self.test.run()
50                 elif choice[0] == "options":
51                     scene = self.options.run()
52                 else:
53                     _, players, map_index = choice
54                     scene = self.game.run(players, self.maps[map_index])
55                 if scene == "quit": # window closed during a scene
56                     break
57                 # otherwise the scene returned "menu": loop back to the menu
58         finally:
59             self.input.close()
60             pygame.quit()

```

config.py (174 lines)

Every number that changes how the game looks or feels. Nothing here runs logic; the other files read these values. The Part IV reference table lists every setting.

Lines	What happens there
1-10	Docstring: units and colour conventions.
11-12	Paths: the project folder and the assets folder, built from this file's location.
15-17	Helper: full path of an asset picture.
20-38	Maps: reference height, the list of maps (name, pictures, spawn, scale) and how high the character starts.
40-53	Game view size, window layout, frame rate, physics substeps, camera smoothing, sky colour.
55-71	Character size and the geometry measured on the 500 px sprite canvas (head, joints, feet).
72-80	The four legs: hip position, side, rest pose.
81-93	Control schemes for 1-4 players: (player, limb, leg) rows.
94-97	Player limit, leg stretch for 1 player, where the shoe starts in the shin picture.
99-104	Flag size and placement settings.
106-121	Physics constants.
123-161	Camera, model files, keypoint threshold, face matching and crop settings, mouth and head-tilt thresholds, smoothing, colours.
163-169	Font lists with Windows and Linux names; pygame uses the first one installed.
171-174	Title and credit text.

Full source

```

1  """All tunable numbers live here so gameplay can be tweaked without touching logic.
2
3  Every other file does `import config` and reads values like `config.GRAVITY`.
4  Units: "px" means map pixels (the mountain map's pixels), angles are degrees,
5  times are seconds, colours are (R, G, B) for pygame or (B, G, R) for OpenCV.
6  """
7  import os
8
9  # Folder this file is in = the project folder. Building paths from it means the
10 # game finds its files no matter which folder you start Python from.
11 ROOT = os.path.dirname(os.path.abspath(__file__))
12 ASSET_DIR = os.path.join(ROOT, "assets")
13
14
15 def asset(name):
16     """Full path of an image in the assets folder, e.g. asset("head.png")."""
17     return os.path.join(ASSET_DIR, name)
18
19
20 # ----- maps -----
21 # Every map is scaled to MAP_HEIGHT px tall (times its "scale"), so character size,
22 # view and physics feel the same on all maps; scale 1.5 = map 1.5x as big compared to
23 # the character. spawn = (x, y) as fractions of the map: the character drops from
24 # there to the floor. The flag goes on the top-right high ground.
25 MAP_HEIGHT = 2164
26 MAPS = [
27     {"name": "Mountain", "background": "background.png", "foreground": "foreground.png",
28      "spawn": (0.06, 0.0)},
29     {"name": "Cave", "background": "background-cave.png", "foreground": "foreground-cave.png",
30      "spawn": (0.04, 0.84), "scale": 1.5},
31     # windmill: tower picture (no collision) + rotor that spins and is solid.
32     # hub = rotation centre in the images' pixels, rps = turns per second (clockwise)
33     {"name": "Windmill", "background": "map3-background.png", "foreground": "map3-foreground.png",
34      "spawn": (0.04, 0.0),
35      "windmill": {"base": "map3-windmillbase.png", "blade": "map3-windmillblade.png",
36                  "hub": (1555, 700), "rps": 0.25}},
37 ]
38 SPAWN_LIFT = 1 / 8          # start this far (MAP_HEIGHT fraction) above the floor, or mid-gap if lower
39
40 # ----- map / view -----
41 VIEW_FRACTION = 1 / 3      # game view height = 1/3 of the map height
42 VIEW_ASPECT = 4096 / 2164  # view width / height (the mountain map's shape)
43 WINDOW_SCALE = 1.0        # max scale of the window (it also shrinks to fit the screen)
44 # one window: game screen left, camera right (see display.py), sizes relative to the game view width
45 LAYOUT_MARGIN = 0.025     # gap between window edge / game / camera
46 LAYOUT_CAMERA_WIDTH = 0.40 # camera panel width
47 LAYOUT_SCREEN_FILL = 0.92  # window may use at most this fraction of the desktop
48 LAYOUT_BG = (40, 42, 48)   # dark grey around the two panels
49 START_FULLSCREEN = False   # Fll toggles fullscreen at any time
50 FPS = 60                  # frames drawn per second (the loop waits to keep this rate)
51 PHYSICS_SUBSTEPS = 3      # physics steps per rendered frame (stability)
52 CAMERA_LERP = 6.0         # how fast the game view follows the character (1/s)
53 SKY_COLOR = (135, 206, 235) # fills transparent gaps behind the map images
54
55 # ----- character size -----
56 # All character pngs share a 500x500 canvas with the same scale.
57 # character-demo.png canvas height = CHARACTER_FRACTION * map height.
58 CHARACTER_FRACTION = 1 / 8
59 SPRITE_CANVAS = 500       # size of the character png canvases
60
61 # Geometry measured on the 500x500 canvas (px)
62 HEAD_CENTER = (250, 250)
63 HEAD_RADIUS = 95          # collision circle of the head
64 LEG_PIVOT = (249.5, 190.5) # joint at the top of leg-part.png / leg-part-with-shoe.png
65 SEGMENT_LENGTH = 117      # pivot -> pivot of leg-part.png
66 FOOT_OFFSET = (20, 160)   # shoe centre from LEG_PIVOT while the sprite hangs down (toe = +x)
67 FOOT_RADIUS = 30          # collision circle of the shoe
68
69 # name: (hip offset from head centre, side (-1 left / +1 right), rest thigh, rest bend)
70 # Angles in degrees, "outward" convention: 0 = pointing down, 90 = pointing

```

```

71 # sideways away from the body, 180 = pointing up. bend = shin angle - thigh angle.
72 LEGS = {
73     "upper_left": ((-75, -55), -1, 145, 20),
74     "upper_right": ((75, -55), +1, 145, 20),
75     "lower_left": ((-85, 50), -1, 50, -50),
76     "lower_right": ((85, 50), +1, 50, -50),
77 }
78 # Legs whose shoe is drawn mirrored. Upper legs usually point up, which turns a normal
79 # shoe's toe inward; mirroring makes their toes point outward, like character-demo.png.
80 MIRRORED_SHOES = {"upper_left", "upper_right"}
81 # Who drives which leg, per number of players: (player index, body limb, character leg).
82 # Limbs are named by screen side of the mirrored camera image (= the player's own side).
83 # Players are numbered left -> right as they stand in front of the camera.
84 # The character only gets the legs listed in its scheme.
85 CONTROL_SCHEMES = {
86     1: [(0, "left_arm", "lower_left"), (0, "right_arm", "lower_right")],
87     2: [(0, "left_arm", "upper_left"), (0, "right_arm", "upper_right"),
88         (1, "left_arm", "lower_left"), (1, "right_arm", "lower_right")],
89     3: [(0, "left_arm", "upper_left"), (0, "right_arm", "upper_right"),
90         (1, "left_arm", "lower_left"), (2, "right_arm", "lower_right")],
91     4: [(0, "left_arm", "upper_left"), (1, "right_arm", "upper_right"),
92         (2, "left_arm", "lower_left"), (3, "right_arm", "lower_right")],
93 }
94 MAX_PLAYERS = 4 # highest player count offered in the menu
95 # leg length multiplier per player count (only the black sticks stretch, shoes keep their size)
96 LEG_STRETCH = {1: 1.5}
97 SHOE_TOP = 300 # canvas y where the shoe starts on leg-part-with-shoe.png
98
99 # ----- flag --
100 # flag.png stands on the highest ground in the right-most FLAG_SEARCH part of the map
101 # that has room above it for the whole flag.
102 FLAG_SEARCH = 0.1 # fraction of the map width searched for the peak
103 FLAG_FRACTION = 1 / 9 # flag.png canvas height relative to MAP_HEIGHT
104 FLAG_BASE = (160, 492) # canvas px of flag.png that sits on the ground (pole foot)
105
106 # ----- physics --
107 GRAVITY = 2000.0 # px/s^2 (map pixels)
108 MAX_SPEED = 1500.0 # px/s, speed limit while touching; falling speed limit in the air
109 AIR_DRAG = 0.3 # 1/s
110 LEG_MAX_SPEED = 420.0 # deg/s, how fast a leg joint can follow the arm
111 SOLVER_ITERATIONS = 4 # passes over all contacts per physics step (more = stiffer, slower)
112 WALKABLE_SLOPE = 55.0 # deg; feet only get friction on surfaces flatter than this
113 ROTOR_WALKABLE_SLOPE = 10.0 # deg; same for the windmill rotor: slippery (sticky feet still glue)
114 ROTOR_PUSH_SPEED = 400.0 # px/s; a blade can push a free foot/head away at most this much faster than it moves
115 CONTACT_EPS = 2.0 # px; a foot this close to the ground still counts as touching
116 STICKY_GRAB_DIST = 8.0 # px; a sticky foot grabs ground within this distance
117 STICKY_BREAK_DIST = 40.0 # px; a sticky foot tears off if pulled this far from its anchor
118 STICKY_STIFFNESS = 0.6 # share of a glued foot's error fixed per solver pass (1 = all)
119 LEG_BLOCK_TOLERANCE = 3.0 # px a free foot may sink into rock while another foot is glued
120 STICKY_SINK = 6.0 # px a sticky/glued foot may sink into rock while another foot is glued
121 HEAD_FRICTION = 6.0 # 1/s, damping while the head scrapes the ground
122
123 # ----- vision --
124 CAMERA_SOURCE = "0" # webcam index or video path
125 POSE_MODEL = "yolo26n-pose.pt" # YOLO "nano" pose model, downloaded automatically
126 # Use yolo26n-pose.engine (TensorRT, made for THIS GPU) when it exists and a GPU is present.
127 # Rebuild it after changing GPU/driver/TensorRT: python export_engine.py
128 USE_TENSORRT = True
129 FACE_MODEL = os.path.join(ROOT, "face_landmarker.task") # MediaPipe face model file
130 FACE_URL = ("https://storage.googleapis.com/mediapipe-models/face_landmarker/"
131             "face_landmarker/float16/1/face_landmarker.task")
132 MIRROR_CAMERA = True # flip the webcam so it behaves like a mirror
133 KEYPOINT_THR = 0.5 # keypoints with lower confidence (0..1) count as "not seen"
134
135 # face <-> pose matching (pixel closeness)
136 FACE_MATCH_FACTOR = 1.0 # max face-to-head distance = factor * shoulder width
137 FACE_MATCH_MIN_PX = 40
138 FACE_CROP_FALLBACK = True # if the full-frame face search misses a player, retry on a head crop
139 CROP_SCALE = 2.4 # head crop size = spread of nose/eyes/ears * this
140 CROP_DOWN = 0.25 # move the crop down (in spreads) so chin and mouth are inside

```

```

141 MIN_CROP = 14      # px; smaller heads heads are skipped (too few pixels for a face mesh)
142 UPSCALE_TO = 256    # small crops are enlarged to this before the face model
143
144 # mouth -> sticky feet
145 MOUTH_OPEN_T, MOUTH_CLOSE_T = 0.35, 0.25 # hysteresis on lip gap / lip width
146 MOUTH_EMA = 0.5      # smoothing of the mouth score (1 = no smoothing)
147 MOUTH_TIMEOUT = 0.5  # s; forget the mouth state if the face is lost for this long
148
149 # 1 player: sideways head tilt (roll) picks which foot the open mouth sticks.
150 # head tilted to the right (right ear lower) -> right-arm leg, tilted left -> left-arm leg.
151 HEAD_TILT_PLAYERS = {1} # player counts that use this
152 HEAD_TILT_ANGLE = 10.0  # deg the ear line must tilt to switch side (in between: keep last)
153 HEAD_TILT_INVERT = False # True: swap (tilt right -> left-arm leg)
154
155 # arm angles -> legs
156 ANGLE_EMA = 0.6         # smoothing of arm angles (1 = no smoothing)
157
158 # player colours (BGR for OpenCV): red, green, blue, yellow
159 PLAYER_COLORS_BGR = [(0, 0, 255), (0, 220, 0), (255, 120, 0), (0, 220, 255)]
160 PLAYER_COLORS_RGB = [tuple(reversed(c)) for c in PLAYER_COLORS_BGR]
161 BODY_COLOR_BGR = (150, 150, 150) # grey for non-controlling body parts in the preview
162
163 # ----- fonts -----
164 # Comma-separated lists: pygame uses the first one installed. Windows has Arial /
165 # Consolas / Segoe UI Emoji; Linux usually has DejaVu, Liberation or FreeFont.
166 FONT_HEAVY = "arialblack,arial,dejavusans,liberation,free,sans" # titles, buttons
167 FONT_TEXT = "arial,dejavusans,liberation,free,sans" # normal text
168 FONT_MONO = "consolas,couriernew,dejavusansmono,liberationmono,free,mono" # numbers, timer
169 FONT_EMOJI = "segoeuiemoji,notocoloreemoji,notoemoji,symbola" # the sheep in the menu
170
171 # ----- menu -----
172 GAME_TITLE = "All For One"
173 GAME_CREDIT = "From The Group MeEeEe"
174 GAME_CREDIT_EMOJI = "[sheep emoji]" # sheep

```

settings.py (148 lines)

Everything about player settings: which config values the Options screen may change, the safe range of each, and loading/saving `settings.json` (chapter 12).

Lines	What happens there
1-13	Docstring: how settings flow into config , and why they cannot break the game.
14-21	Imports, the file location and the minimum gap between the mouth thresholds.
24-34	Setting : one tunable value - config name, label, tab, type, range, step, unit, help.
36-44	Convert a value to the right type and force it into the safe range (None if impossible).
47-84	The tabs, and the list of all tunable settings with their ranges and help texts.
85	Look-up table: config name -> Setting.
88-92	Remember the config.py values as the defaults.
94-95	Current value (read from config).
97-109	Clamp, write into config, keep 'close' below 'open'.
111-114	Put the defaults back.
116-120	Only the values that differ from the defaults.
121-136	Read settings.json safely; anything wrong is skipped.
138-148	Write the changed values (or delete the file when nothing changed).

Full source

```

1  """Player-tunable settings: what can be changed in the Options screen, the safe range
2  of each value, and saving/loading them to settings.json.
3
4  How it works: the game reads all its numbers from `config`. A Setting names one of
5  those config values; Settings.set() writes the new value straight into `config`
6  (e.g. config.GRAVITY = 2400), so every part of the game sees it immediately.
7  settings.json only stores the values the player changed; at start-up they are
8  put back into `config` before anything else is built.
9
10 Only values that cannot break the game are offered, each limited to a safe
11 range. A missing, broken or hand-edited settings.json never crashes the game:
12 unknown names and wrong types are ignored and numbers are clamped into range.
13 """
14 import json
15 import os
16 from dataclasses import dataclass
17
18 import config
19
20 SETTINGS_FILE = os.path.join(config.ROOT, "settings.json")
21 MOUTH_GAP = 0.05      # "close" threshold is kept at least this far below "open"
22
23
24 @dataclass
25 class Setting:
26     key: str          # name of the value in config.py
27     label: str        # text shown in the Options screen
28     group: str        # tab it appears in
29     kind: str         # "float", "int" or "bool"
30     lo: float = 0.0   # smallest allowed value (numbers only)
31     hi: float = 1.0   # largest allowed value
32     step: float = 0.01 # slider step
33     unit: str = ""    # shown after the value, e.g. "deg"
34     help: str = ""    # one-line explanation shown at the bottom of the screen
35
36     def clamp(self, value):
37         """Convert to the right type and force into the safe range (None if impossible)."""
38         if self.kind == "bool":
39             return value if isinstance(value, bool) else None
40         if isinstance(value, bool) or not isinstance(value, (int, float)):
41             return None
42         value = min(max(float(value), self.lo), self.hi)
43         value = round(round(value / self.step) * self.step, 6) # snap to the slider step
44         return int(round(value)) if self.kind == "int" else float(value)
45
46
47 GROUPS = ["Camera", "Mouth", "Head tilt", "Controls", "Physics", "Display"]
48
49 SETTINGS = [
50     Setting("MIRROR_CAMERA", "Mirror camera", "Camera", "bool",
51             help="Flip the camera picture like a mirror (your right hand shows on the right)."),
52     Setting("KEYPOINT_THR", "Joint confidence needed", "Camera", "float", 0.3, 0.8, 0.05,
53             help="Lower: arms are tracked in worse light but jitter more. "
54                  "Higher: steadier, lost more often."),
55
56     Setting("MOUTH_OPEN_T", "Open above", "Mouth", "float", 0.15, 0.6, 0.01,
57             help="Mouth ratio (lip gap / mouth width) needed to turn sticky on. Lower = easier."),
58     Setting("MOUTH_CLOSE_T", "Close below", "Mouth", "float", 0.05, 0.55, 0.01,
59             help="Ratio below which sticky turns off again. Always kept a bit below 'Open above'."),
60     Setting("MOUTH_EMA", "Mouth smoothing", "Mouth", "float", 0.2, 1.0, 0.05,
61             help="1 = react instantly, lower = smoother but slower to open/close."),
62     Setting("MOUTH_TIMEOUT", "Keep state when face lost", "Mouth", "float", 0.0, 2.0, 0.1, "s",
63             help="How long the mouth state is kept when the face is not found for a moment."),
64
65     Setting("HEAD_TILT_ANGLE", "Switch angle", "Head tilt", "float", 5, 25, 1, "deg",
66             help="1 player: how far you tilt your head sideways to choose the sticky foot."),
67     Setting("HEAD_TILT_INVERT", "Swap sides", "Head tilt", "bool",
68             help="1 player: tilt right picks the LEFT foot instead of the right one."),
69
70     Setting("LEG_MAX_SPEED", "Leg speed", "Controls", "float", 240, 720, 10, "deg/s",

```

```

71         help="How fast the legs follow your arms. Higher = snappier, stronger kicks and jumps."),
72     Setting("ANGLE_EMA", "Arm smoothing", "Controls", "float", 0.2, 1.0, 0.05,
73         help="1 = legs copy every arm movement, lower = smoother legs but a little delay."),
74
75     Setting("GRAVITY", "Gravity", "Physics", "float", 1200, 3000, 50, "px/s2",
76         help="Higher = heavier character: falls faster, jumps lower."),
77     Setting("WALKABLE_SLOPE", "Grip on slopes up to", "Physics", "float", 30, 70, 1, "deg",
78         help="Feet grip (don't slide) on ground flatter than this, without being sticky."),
79     Setting("STICKY_BREAK_DIST", "Sticky foot tears off at", "Physics", "float", 20, 80, 2, "px",
80         help="How far a glued foot can be pulled from its spot before it lets go."),
81
82     Setting("START_FULLSCREEN", "Fullscreen", "Display", "bool",
83         help="Play in fullscreen (also F11 at any time). Remembered for the next start."),
84 ]
85 BY_KEY = {s.key: s for s in SETTINGS}
86
87
88 class Settings:
89     def __init__(self, path=SETTINGS_FILE):
90         self.path = path
91         # the values written in config.py = the defaults "Reset to defaults" goes back to
92         self.defaults = {s.key: getattr(config, s.key) for s in SETTINGS}
93
94     def get(self, key):
95         return getattr(config, key)
96
97     def set(self, key, value):
98         """Change one setting (clamped to its safe range) and apply it immediately."""
99         setting = BY_KEY.get(key)
100         value = setting.clamp(value) if setting else None
101         if value is None:
102             return
103         setattr(config, key, value)
104         # keep the mouth thresholds in a valid order (close < open)
105         if key == "MOUTH_OPEN_T" and config.MOUTH_CLOSE_T > value - MOUTH_GAP:
106             config.MOUTH_CLOSE_T = BY_KEY["MOUTH_CLOSE_T"].clamp(value - MOUTH_GAP)
107         if key == "MOUTH_CLOSE_T" and config.MOUTH_OPEN_T < value + MOUTH_GAP:
108             config.MOUTH_OPEN_T = BY_KEY["MOUTH_OPEN_T"].clamp(value + MOUTH_GAP)
109             config.MOUTH_CLOSE_T = min(config.MOUTH_CLOSE_T, config.MOUTH_OPEN_T - MOUTH_GAP)
110
111     def reset(self):
112         """Back to the values written in config.py."""
113         for key, value in self.defaults.items():
114             setattr(config, key, value)
115
116     def changed(self):
117         """Only the settings that differ from the defaults (what gets saved)."""
118         return {k: self.get(k) for k in self.defaults if self.get(k) != self.defaults[k]}
119
120 # ----- file -----
121     def load(self):
122         """Apply settings.json if it exists. Anything wrong in it is skipped."""
123         try:
124             with open(self.path, encoding="utf-8") as f:
125                 data = json.load(f)
126         except FileNotFoundError:
127             return
128         except (OSError, ValueError) as e:
129             print(f"settings.json ignored ({e}); using defaults")
130             return
131         if not isinstance(data, dict):
132             return
133         # open threshold first, so the close threshold is checked against it
134         for key in sorted(data, key=lambda k: k != "MOUTH_OPEN_T"):
135             if key in BY_KEY:
136                 self.set(key, data[key])
137
138     def save(self):
139         """Write the changed settings; with nothing changed the file is removed."""
140         changed = self.changed()
141
142         try:
143             if changed:
144                 with open(self.path, "w", encoding="utf-8") as f:
145                     json.dump(changed, f, indent=2)
146             elif os.path.exists(self.path):
147                 os.remove(self.path)
148         except OSError as e:
149             print(f"could not save settings ({e})")

```

display.py (141 lines)

The single window. Scenes draw on a fixed-size canvas; this class scales it into the window next to the camera picture and handles fullscreen.

Lines	What happens there
1-19	Docstring with the layout sketch.
20-23	Imports.
26-41	Constructor: layout size in canvas units (game + camera + margins), create the window and the canvas.
42-53	Create the window: fullscreen at screen size, or windowed at the biggest scale that fits 92% of the desktop.
55-73	Compute where the game panel, the camera panel (top of the right column, 4:3) and the map panel (the rest of the column) go for a given window size.
75-76	Switch mode.
78-83	Keys every scene shares (F11). Returns True if it used the event.
85-90	Convert a mouse position from window pixels to canvas pixels.
91-106	Draw the frame: background, scaled canvas, frame, camera, map; then <code>flip()</code> shows it.
108-128	Turn the newest camera frame (numpy) into a pygame image only when it changed, fit it into the panel, or show 'no camera'.
130-141	Let the game draw the map overview (it passes a function), or show a note.

Full source

```

1  """The single app window: game screen on the left, camera preview on the right.
2
3
4  +-----+
5  | +-----+ |
6  | | game screen | | camera |
7  | |           | | +-----+ |
8  | |           | | +-----+ |
9  | |           | | | map |
10 | +-----+ | +-----+
11 +-----+
12
13 Scenes draw onto `canvas` (map-view sized); present() scales it into the
14 window. Mouse positions must go through to_canvas() before hit-testing.
15 F11 (handled in handle_event) switches between window and fullscreen.
16
17 Why a canvas: scenes always draw at the same size (1365 x 721) no matter how
18 big the window is; only present() deals with the real window size.
19 """
20 import pygame
21
22 import config
23 from ui import draw_text
24
25
26 class Display:
27     def __init__(self, canvas_size, input_source):
28         self.input = input_source
29         self.canvas_size = canvas_size
30         cw, ch = canvas_size
31         # whole layout at canvas scale (1 unit = 1 canvas px)
32         self.margin = config.LAYOUT_MARGIN * cw
33         self.full_w = cw + config.LAYOUT_CAMERA_WIDTH * cw + 3 * self.margin # game + camera + 3 gaps
34         self.full_h = ch + 4 * self.margin
35         self.fullscreen = False
36         self.cam_surface = None # last camera picture, already scaled for the panel
37         self._cam_source = None # the frame cam_surface was made from
38         self._set_mode(config.START_FULLSCREEN)
39         self.canvas = pygame.Surface(canvas_size).convert()
40
41     # ----- window -----
42     def _set_mode(self, fullscreen):
43         """Create (or re-create) the window, then work out where things go in it."""
44         self.fullscreen = fullscreen
45         if fullscreen:
46             self.window = pygame.display.set_mode((0, 0), pygame.FULLSCREEN) # (0, 0) = screen size
47         else:
48             # biggest scale that fits LAYOUT_SCREEN_FILL of the desktop (and WINDOW_SCALE)
49             desk_w, desk_h = pygame.display.get_desktop_sizes()[0]
50             s = min(config.WINDOW_SCALE, desk_w * config.LAYOUT_SCREEN_FILL / self.full_w,
51                    desk_h * config.LAYOUT_SCREEN_FILL / self.full_h)
52             self.window = pygame.display.set_mode((round(self.full_w * s), round(self.full_h * s)))
53             self._layout(*self.window.get_size())
54
55     def _layout(self, w, h):
56         """Fit the layout into a w x h window, centred (letterboxed in fullscreen)."""
57         cw, ch = self.canvas_size
58         s = min(w / self.full_w, h / self.full_h) # scale that fits both width and height
59         self.scale = s
60         left = (w - self.full_w * s) / 2 # empty space on each side, if any
61         m = self.margin * s
62         self.game_rect = pygame.Rect(round(left + m), 0, round(cw * s), round(ch * s))
63         self.game_rect.centery = h // 2
64         cam_w = round(config.LAYOUT_CAMERA_WIDTH * cw * s)
65         self.label_font = pygame.font.SysFont(config.FONT_TEXT, max(14, cam_w // 22), bold=True)
66         label = self.label_font.get_linesize() + 6 # room for a "CAMERA" / "MAP" title
67         # right column, top-aligned with the game: camera (4:3 like a webcam picture),
68         # then the map overview using the rest of the height
69         x = self.game_rect.right + round(m)
70         self.cam_rect = pygame.Rect(x, self.game_rect.top + label, cam_w, round(cam_w * 3 / 4))

```

```

71     map_top = self.cam_rect.bottom + round(m) + label
72     self.map_rect = pygame.Rect(x, map_top, cam_w, max(20, self.game_rect.bottom - map_top))
73     self._cam_source = None # rescale the camera image for the new size
74
75     def toggle_fullscreen(self):
76         self._set_mode(not self.fullscreen)
77
78     def handle_event(self, event):
79         """Window-level keys shared by every scene. True if the event was used."""
80         if event.type == pygame.KEYDOWN and event.key == pygame.K_F11:
81             self.toggle_fullscreen()
82             return True
83         return False
84
85     def to_canvas(self, pos):
86         """Window pixel -> canvas pixel (for mouse clicks)."""
87         # undo what present() did: subtract the game panel's corner, divide by the scale
88         return ((pos[0] - self.game_rect.x) / self.scale, (pos[1] - self.game_rect.y) / self.scale)
89
90     # ----- draw -----
91     def present(self, minimap=None):
92         """Put the finished canvas, the camera picture and the map overview in the
93         window and show it. minimap(surface, rect) draws the map (the game passes
94         one); without it the map panel shows a short note."""
95         self.window.fill(config.LAYOUT_BG)
96         if self.game_rect.size == self.canvas.get_size():
97             self.window.blit(self.canvas, self.game_rect) # same size: plain copy (fast)
98         else:
99             self.window.blit(pygame.transform.smoothscale(self.canvas, self.game_rect.size),
100                             self.game_rect)
101         pygame.draw.rect(self.window, (0, 0, 0), self.game_rect.inflate(6, 6), 3) # black frame
102         self._draw_camera()
103         self._draw_map(minimap)
104         # Everything above was drawn into a hidden buffer; flip() shows it all at
105         # once, so the player never sees a half-drawn frame.
106         pygame.display.flip()
107
108     def _draw_camera(self):
109         frame, fresh = self.input.get_preview()
110         # convert numpy -> pygame image only when a NEW camera frame arrived
111         # (the camera runs slower than the game; re-converting each frame is waste)
112         if frame is not None and (fresh or self._cam_source is not frame):
113             self._cam_source = frame
114             h, w = frame.shape[:2]
115             img = pygame.image.frombuffer(frame.tobytes(), (w, h), "RGB")
116             fit = min(self.cam_rect.w / w, self.cam_rect.h / h) # fit inside the panel, keep shape
117             self.cam_surface = pygame.transform.smoothscale(img, (round(w * fit), round(h * fit)))
118         if self.cam_surface is not None and self._cam_source is not None:
119             rect = self.cam_surface.get_rect(center=self.cam_rect.center)
120             self.window.blit(self.cam_surface, rect)
121         else:
122             rect = self.cam_rect
123             pygame.draw.rect(self.window, (20, 20, 24), rect)
124             draw_text(self.window, "no camera (keyboard mode)", self.label_font,
125                       (200, 200, 200), rect.center, width=1)
126         pygame.draw.rect(self.window, (0, 0, 0), rect.inflate(6, 6), 3)
127         draw_text(self.window, "CAMERA", self.label_font, (230, 230, 230),
128                   (rect.centerx, rect.top - self.label_font.get_linesize()), width=1)
129
130     def _draw_map(self, minimap):
131         """The whole-map overview under the camera (only filled in during a game)."""
132         if minimap is not None:
133             rect = minimap(self.window, self.map_rect)
134         else:
135             rect = self.map_rect
136             pygame.draw.rect(self.window, (20, 20, 24), rect)
137             draw_text(self.window, "the whole map is shown here during the game", self.label_font,
138                       (150, 150, 150), rect.center, width=1)
139         pygame.draw.rect(self.window, (0, 0, 0), rect.inflate(6, 6), 3)
140         draw_text(self.window, "MAP", self.label_font, (230, 230, 230),
141                   (rect.centerx, rect.top - self.label_font.get_linesize()), width=1)

```

minimap.py (50 lines)

The whole map in the panel under the camera (chapter 10).

Lines	What happens there
1-3	Docstring and imports.
5-6	Colours of the character dot and the view frame.
9-13	Remembers its map and the cached small picture.
15-27	The map with its flag shrunk to the panel size; redone only when the size changes.
29-50	Fit the map into the panel keeping its shape, draw the turning rotor, the frame of the visible area and the character dot (kept inside the picture).

Full source

```

1  """Map overview for the panel under the camera: the whole map, the flag, the
2  part of the map the game view shows, and where the character is."""
3  import pygame
4
5  MARKER = (255, 220, 40)      # character dot (yellow, visible on sky and in the dark cave)
6  VIEW_FRAME = (255, 255, 255) # rectangle of the area shown in the game view
7
8
9  class MiniMap:
10     def __init__(self, game_map):
11         self.map = game_map
12         self._image = None      # map picture scaled for the panel (made once per panel size)
13         self._size = None
14
15     def _picture(self, size):
16         """The whole map with its flag, shrunk to `size`. Cached: scaling the big map
17         picture is slow, so it is redone only when the panel size changes (F11)."""
18         if size != self._size:
19             t = self.map.terrain
20             k = size[1] / t.height
21             img = pygame.transform.smoothscale(t.surface, size)
22             flag = self.map.flag
23             fw, fh = flag.image.get_size()
24             small = pygame.transform.smoothscale(flag.image, (max(1, round(fw * k)), max(1, round(fh * k))))
25             img.blit(small, flag.pos * k)
26             self._image, self._size = img, size
27         return self._image
28
29     def draw(self, surface, rect, char_pos, view_rect):
30         """Draw into `rect` (window pixels); returns the rectangle actually used."""
31         t = self.map.terrain
32         k = min(rect.w / t.width, rect.h / t.height)      # fit the whole map, keep its shape
33         img = self._picture((max(1, round(t.width * k)), max(1, round(t.height * k))))
34         area = img.get_rect(center=rect.center)
35         surface.blit(img, area)
36         self.map.draw_movers_small(surface, area.topleft, k) # windmill rotor, turning
37
38         # the part of the map that the game view shows right now
39         view = pygame.Rect(area.x + view_rect.x * k, area.y + view_rect.y * k,
40                             max(2, view_rect.w * k), max(2, view_rect.h * k)).clip(area)
41         pygame.draw.rect(surface, (0, 0, 0), view.inflate(2, 2), 1)
42         pygame.draw.rect(surface, VIEW_FRAME, view, 2)
43
44         # the character: a dot with a dark ring, kept inside the picture even when
45         # the character is above the map's top edge
46         x = min(max(area.x + char_pos[0] * k, area.left + 6), area.right - 6)
47         y = min(max(area.y + char_pos[1] * k, area.top + 6), area.bottom - 6)
48         pygame.draw.circle(surface, (0, 0, 0), (x, y), 7)
49         pygame.draw.circle(surface, MARKER, (x, y), 5)
50         return area

```

menu.py (166 lines)

The menu screen with its three pages. Returns what the player chose.

Lines	What happens there
1-7	Docstring: how the page loop works.
8-13	Imports.
14-19	Help text on the player-count buttons.
22-43	Constructor: backdrop picture, demo character, fonts.
45-72	Create every button: PLAY, 1P TEST, the four player-count buttons, BACK and one picture button per map, laid out side by side with equal gaps.
74-114	The menu loop: handle quit/F11, convert mouse positions to canvas pixels, react to keys and clicks depending on the page, then draw.
115-123	Render the sheep with an emoji font, or None if none is installed (Linux).
125-134	Draw the credit line, with the sheep if it could be rendered.
136-166	Draw the current page.

Full source

```

1  """Main menu: title + Play / Test, then the player-count picker, then the map picker.
2
3  The menu is one loop with a `page` variable ("title" -> "players" -> "maps").
4  Every frame it handles clicks/keys for the current page and redraws that page.
5  All positions are fractions of the canvas size (e.g. h * 0.80 = 80% down),
6  so the layout stays right if the canvas size ever changes.
7  """
8  import pygame
9
10 import config
11 from ui import Button, MapButton, draw_text
12
13 # two lines of help text shown on each player-count button
14 PLAYER_INFO = {
15     1: ["two long legs", "left arm + right arm"],
16     2: ["P1 arms: top legs", "P2 arms: bottom legs"],
17     3: ["P1 arms: top legs", "P2 left arm, P3 right arm"],
18     4: ["one arm = one leg", "P1 P2 top, P3 P4 bottom"],
19 }
20
21
22 class Menu:
23     """run() returns ("play", players, map index), ("test", 1), ("options",) or None to quit."""
24
25     def __init__(self, display, maps, input_source):
26         self.display = display
27         self.canvas = display.canvas
28         self.input = input_source
29         w, h = self.canvas.get_size()
30         # background picture: the whole first map shrunk to the canvas
31         self.backdrop = pygame.transform.smoothscale(maps[0].terrain.surface, (w, h))
32         demo = pygame.image.load(config.asset("character-demo.png")).convert_alpha()
33         size = int(h * 0.42)
34         self.demo = pygame.transform.smoothscale(demo, (size, size))
35
36         # SysFont("a,b,...") uses the first installed font of the list (see config FONT_*)
37         self.title_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.13), bold=True)
38         self.big_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.06), bold=True)
39         self.mid_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.04), bold=True)
40         self.small_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.024), bold=True)
41         self.credit_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.034), bold=True)
42         self.sheep = self._render_emoji(config.GAME_CREDIT_EMOJI, int(h * 0.04))
43         self.clock = pygame.time.Clock()
44
45         cx = w // 2
46         self.play = Button("PLAY", (cx, int(h * 0.80)), (int(w * 0.22), int(h * 0.12)),
47                             self.big_font, (220, 60, 60))
48         self.options = Button("OPTIONS", (int(w * 0.20), int(h * 0.80)), (int(w * 0.15), int(h * 0.09)),
49                               self.mid_font, (190, 120, 40))
50         self.test = Button("1P TEST", (int(w * 0.80), int(h * 0.80)), (int(w * 0.15), int(h * 0.09)),
51                             self.mid_font, (60, 110, 200))
52         bw, bh = int(w * 0.21), int(h * 0.22)
53         self.counts = []
54         for i, n in enumerate(range(1, config.MAX_PLAYERS + 1)):
55             x = int(w * (0.5 + (i - 1.5) * 0.235)) # 4 buttons spread around the centre
56             label = f"{n} PLAYER" + ("S" if n > 1 else "")
57             self.counts.append((n, Button(label, (x, int(h * 0.55)), (bw, bh), self.big_font,
58                                           config.PLAYER_COLORS_RGB[i], PLAYER_INFO[n],
59                                           self.small_font)))
60         self.back = Button("BACK", (cx, int(h * 0.85)), (int(w * 0.14), int(h * 0.09)),
61                             self.big_font, (90, 90, 90))
62         self.maps = []
63         for game_map in maps:
64             self.maps.append(MapButton(game_map.name.upper(), game_map.thumbnail,
65                                       (0, int(h * 0.58)), self.mid_font))
66         # side by side, centred, with equal gaps (the cards have different widths)
67         gap = 40
68         x = (w - sum(b.rect.w for b in self.maps) - gap * (len(self.maps) - 1)) // 2
69         for b in self.maps:
70             b.rect.left = x

```

```

71         x += b.rect.w + gap
72         self.players = 1
73
74     def run(self):
75         page = "title"
76         while True:
77             self.clock.tick(config.FPS)
78             for event in pygame.event.get():
79                 if event.type == pygame.QUIT:
80                     return None
81                 if self.display.handle_event(event):
82                     continue
83                 # mouse events have a window position: convert it to canvas pixels,
84                 # because the buttons were placed on the canvas
85                 pos = self.display.to_canvas(event.pos) if hasattr(event, "pos") else (-1, -1)
86                 key = event.key if event.type == pygame.KEYDOWN else None
87                 if key == pygame.K_ESCAPE:
88                     if page == "title":
89                         return None
90                     page = "players" if page == "maps" else "title"
91                 if page == "title":
92                     if self.play.clicked(event, pos) or key == pygame.K_RETURN:
93                         page = "players"
94                     elif self.test.clicked(event, pos) or key == pygame.K_t:
95                         return "test", 1
96                     elif self.options.clicked(event, pos) or key == pygame.K_o:
97                         return ("options",)
98                 elif page == "players":
99                     if self.back.clicked(event, pos):
100                         page = "title"
101                     for n, button in self.counts:
102                         if button.clicked(event, pos) or key == pygame.K_0 + n:
103                             self.players, page = n, "maps"
104                             break
105                 else:
106                     if self.back.clicked(event, pos):
107                         page = "players"
108                     for i, button in enumerate(self.maps):
109                         if button.clicked(event, pos) or key == pygame.K_1 + i:
110                             return "play", self.players, i
111             self.draw(page)
112             self.display.present()
113
114     @staticmethod
115     def _render_emoji(char, size):
116         """The emoji as a picture, or None if no emoji font is installed (common on
117         Linux) - then the credit line is shown without it instead of an empty box."""
118         if pygame.font.match_font(config.FONT_EMOJI) is None:
119             return None
120         try:
121             return pygame.font.SysFont(config.FONT_EMOJI, size).render(char, True, (255, 255, 255))
122         except (pygame.error, ValueError): # e.g. a bitmap emoji font that can't be resized
123             return None
124
125     def draw_credit(self, cx, cy):
126         """'From The Group MeEeEe' + a colour sheep emoji (if an emoji font exists)."""
127         text = self.credit_font.render(config.GAME_CREDIT, True, (255, 255, 255))
128         sheep = self.sheep
129         gap = 8 if sheep else 0
130         left = cx - (text.get_width() + gap + (sheep.get_width() if sheep else 0)) // 2
131         draw_text(self.canvas, config.GAME_CREDIT, self.credit_font, (255, 255, 255),
132                  (left + text.get_width() // 2, cy), width=2)
133         if sheep:
134             self.canvas.blit(sheep, sheep.get_rect(midleft=(left + text.get_width() + gap, cy)))
135
136     def draw(self, page):
137         c = self.canvas
138         w, h = c.get_size()
139         mouse = self.display.to_canvas(pygame.mouse.get_pos())
140         c.blit(self.backdrop, (0, 0))

```

```

141     shade = pygame.Surface((w, h), pygame.SRCALPHA)
142     shade.fill((0, 0, 0, 90))
143     c.blit(shade, (0, 0))
144     draw_text(c, config.GAME_TITLE, self.title_font, (255, 215, 60),
145              (w // 2, int(h * 0.14)), width=5)
146     self.draw_credit(w // 2, int(h * 0.245))
147     if page == "title":
148         c.blit(self.demo, self.demo.get_rect(center=(w // 2, int(h * 0.51))))
149         self.play.draw(c, mouse)
150         self.test.draw(c, mouse)
151         self.options.draw(c, mouse)
152         draw_text(c, "F11 fullscreen", self.small_font, (230, 230, 230),
153                  (int(w * 0.92), int(h * 0.96)), width=2)
154     elif page == "players":
155         draw_text(c, "How many players?", self.big_font, (255, 255, 255),
156                  (w // 2, int(h * 0.32)))
157         for _, button in self.counts:
158             button.draw(c, mouse)
159         self.back.draw(c, mouse)
160     else:
161         players = f"{self.players} player" + ("s" if self.players > 1 else "")
162         draw_text(c, f"Choose a map ({players})", self.big_font, (255, 255, 255),
163                  (w // 2, int(h * 0.32)))
164         for button in self.maps:
165             button.draw(c, mouse)
166         self.back.draw(c, mouse)

```

ui.py (136 lines)

Reusable drawing helpers for the screens.

Lines	What happens there
1-2	Docstring.
5-18	Text with a black outline: the text is pasted 8 times in black around its position, then once in colour on top.
21-31	A rounded clickable rectangle.
33-36	True for a left click inside the button.
38-54	Hover effect, shadow, body, border, label and optional subtitle lines.
57-77	A button that shows a map picture with its name; reuses Button's click test.
80-117	A slider: mouse x -> value snapped to the step; click or drag; draws track, fill and knob.
120-136	An on/off switch: click to flip; draws a pill with a knob left (off) or right (on).

Full source

```

1  """Small UI helpers: clickable buttons, sliders, switches and outlined text."""
2  import pygame
3
4
5  def draw_text(surface, text, font, color, center, outline=(0, 0, 0), width=3):
6      """Text with a dark outline so it reads on top of the busy map art."""
7      img = font.render(text, True, color)          # True = smooth (anti-aliased) letters
8      rect = img.get_rect(center=center)
9      if outline is not None:
10         # the outline = the same text in black, pasted 8 times around the real
11         # position (left, right, up, down and the diagonals), then the text on top
12         shadow = font.render(text, True, outline)
13         for dx in range(-width, width + 1, width):
14             for dy in range(-width, width + 1, width):
15                 if dx or dy:
16                     surface.blit(shadow, rect.move(dx, dy))
17             surface.blit(img, rect)
18         return rect
19
20
21  class Button:
22      """A rounded, clickable rectangle with a label (and optional lines of small text)."""
23
24      def __init__(self, text, center, size, font, color=(70, 160, 70), subtitle=None, small_font=None):
25          self.text = text
26          self.rect = pygame.Rect(0, 0, *size)
27          self.rect.center = center
28          self.font = font
29          self.color = pygame.Color(color)
30          self.subtitle = subtitle
31          self.small_font = small_font
32
33      def clicked(self, event, pos):
34          """pos = event.pos already mapped to canvas coordinates."""
35          return (event.type == pygame.MOUSEBUTTONDOWN and event.button == 1
36                  and self.rect.collidepoint(pos))
37
38      def draw(self, surface, mouse=None):
39          hover = mouse is not None and self.rect.collidepoint(mouse) # mouse over the button?
40          # hovering: 25% lighter colour and slightly bigger, so it feels "pressable"
41          fill = self.color.lerp((255, 255, 255), 0.25) if hover else self.color
42          rect = self.rect.inflate(8, 8) if hover else self.rect
43          pygame.draw.rect(surface, (0, 0, 0), rect.move(4, 5), border_radius=18) # drop shadow
44          pygame.draw.rect(surface, fill, rect, border_radius=18)                # body
45          pygame.draw.rect(surface, (0, 0, 0), rect, 4, border_radius=18)        # 4 px black border
46          if self.subtitle and self.small_font: # subtitle = list of lines
47              draw_text(surface, self.text, self.font, (255, 255, 255),
48                        (rect.centerx, rect.centery - rect.h * 0.18))
49              line_h = self.small_font.get_linesize()
50              for i, line in enumerate(self.subtitle):
51                  draw_text(surface, line, self.small_font, (255, 255, 255),
52                            (rect.centerx, rect.centery + rect.h * 0.15 + i * line_h), width=2)
53          else:
54              draw_text(surface, self.text, self.font, (255, 255, 255), rect.center)
55
56
57  class MapButton(Button):
58      """A button showing a map picture with its name underneath."""
59
60      def __init__(self, name, image, center, font, color=(60, 60, 70)):
61          pad, label_h = 14, font.get_linesize() + 6
62          size = (image.get_width() + 2 * pad, image.get_height() + label_h + 2 * pad)
63          super().__init__(name, center, size, font, color) # reuse Button's setup and clicked()
64          self.image, self.pad = image, pad
65
66      def draw(self, surface, mouse=None):
67          hover = mouse is not None and self.rect.collidepoint(mouse)
68          fill = self.color.lerp((255, 255, 255), 0.25) if hover else self.color
69          rect = self.rect.inflate(10, 10) if hover else self.rect
70          pygame.draw.rect(surface, (0, 0, 0), rect.move(4, 5), border_radius=18)

```

```

71     pygame.draw.rect(surface, fill, rect, border_radius=18)
72     pygame.draw.rect(surface, (0, 0, 0), rect, 4, border_radius=18)
73     img_rect = self.image.get_rect(midtop=(rect.centerx, rect.top + self.pad))
74     surface.blit(self.image, img_rect)
75     pygame.draw.rect(surface, (0, 0, 0), img_rect, 2)
76     draw_text(surface, self.text, self.font, (255, 255, 255),
77               (rect.centerx, (img_rect.bottom + rect.bottom) // 2), width=2)
78
79
80 class Slider:
81     """Horizontal slider for a number between lo and hi. Click or drag to change it."""
82
83     def __init__(self, rect, lo, hi, step, color=(70, 160, 220)):
84         self.rect = pygame.Rect(rect) # the track
85         self.lo, self.hi, self.step = lo, hi, step
86         self.color = pygame.Color(color)
87         self.dragging = False
88
89     def value_at(self, x):
90         """Mouse x position -> value on the track (snapped to the step)."""
91         t = min(max((x - self.rect.x) / self.rect.w, 0.0), 1.0)
92         v = self.lo + t * (self.hi - self.lo)
93         return round(round(v / self.step) * self.step, 6)
94
95     def handle(self, event, pos):
96         """Returns the new value while the slider is clicked/dragged, otherwise None.
97         pos = mouse position already converted to canvas coordinates."""
98         grab = self.rect.inflate(16, 24) # a bit bigger than the thin track: easier to hit
99         if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1 and grab.collidepoint(pos):
100             self.dragging = True
101             return self.value_at(pos[0])
102         if event.type == pygame.MOUSEMOTION and self.dragging:
103             return self.value_at(pos[0])
104         if event.type == pygame.MOUSEBUTTONUP and event.button == 1:
105             self.dragging = False
106         return None
107
108     def draw(self, surface, value):
109         t = (value - self.lo) / (self.hi - self.lo) if self.hi > self.lo else 0
110         knob_x = self.rect.x + t * self.rect.w
111         track = self.rect.inflate(0, -self.rect.h + 8) # 8 px tall bar, centred
112         pygame.draw.rect(surface, (60, 60, 66), track, border_radius=4)
113         filled = track.copy()
114         filled.w = max(0, int(knob_x - track.x))
115         pygame.draw.rect(surface, self.color, filled, border_radius=4) # part left of the knob
116         pygame.draw.circle(surface, (0, 0, 0), (knob_x, self.rect.centery), 11)
117         pygame.draw.circle(surface, (255, 255, 255), (knob_x, self.rect.centery), 8)
118
119
120 class Toggle:
121     """On/off switch. Click anywhere on it to flip."""
122
123     def __init__(self, rect, color=(70, 180, 90)):
124         self.rect = pygame.Rect(rect)
125         self.color = pygame.Color(color)
126
127     def clicked(self, event, pos):
128         return (event.type == pygame.MOUSEBUTTONDOWN and event.button == 1
129               and self.rect.inflate(10, 10).collidepoint(pos))
130
131     def draw(self, surface, on):
132         r = self.rect.h // 2
133         pygame.draw.rect(surface, self.color if on else (90, 90, 96), self.rect, border_radius=r)
134         pygame.draw.rect(surface, (0, 0, 0), self.rect, 2, border_radius=r)
135         knob = (self.rect.right - r if on else self.rect.x + r, self.rect.centery)
136         pygame.draw.circle(surface, (255, 255, 255), knob, r - 4)

```

game.py (305 lines)

The level itself: the game loop, players -> legs, physics substeps, camera, timer, flag, HUD and the finish screen.

Lines	What happens there
1-10	Docstring listing the six steps of every frame.
11-19	Imports.
21-24	Rows of the score board; readable leg names for the HUD.
27-41	For each (player, limb, leg) row of the scheme: copy the limb's angles to the leg as its target and set the leg's sticky flag from the mouth (and head tilt).
44-75	Constructor: fonts, clock, sprite scale (character = 1/8 of the map height), score board and name-entry state.
77-78	Start position of the current map.
80-86	Create a character with only the controlled legs; sprites are cached per leg length.
88-98	Back to the start: rotor, character (at a spot where no foot is in the rock), view, timer, name entry.
99-145	Set up the map, view, input and character, then the frame loop: events (typing the name first), players, physics substeps (the rotor turns too), camera, timer, flag check, drawing.
147-153	Flag touched: stop the timer, compare with the saved best, start the name entry.
155-174	Typing: characters, Backspace, Enter saves to the score board, Esc skips.
176-179	Method wrapper around the module function.
180-193	Draw in painter's order (map, flag, character, windmill in front, HUD) and present.
195-197	Called by the display to draw the map panel.
199-204	Coloured dot on each knee showing which player drives that leg.
206-221	Player lines and the info line.
222-225	Seconds -> mm:ss.cc.
227-246	Dark overlay with COMPLETED!, time and best time, then name entry or board.
248-258	Question, text box with a blinking cursor, help line.
260-278	Top 8 for this map and player count; this run in gold, on the last line if lower.
280-286	Shorten a long name with '...' so it never runs into the time column.
288-292	One table cell aligned left or right.
294-299	Timer in the top-right corner (gold when finished).
301-305	Small text with a shadow.

Full source

```

1  """Game scene: ties terrain, character, camera view and the players' input together.
2
3  Every frame of Game.run():
4      1. handle keys/window events
5      2. read the players (arm angles, mouths) from the input source
6      3. turn them into leg targets and sticky flags (apply_players)
7      4. run the physics a few small steps (substeps)
8      5. move the camera view, update the timer, check the flag
9      6. draw everything and show it (display.present)
10 """
11 import pygame
12
13 import config
14 from assets import CharacterSprites
15 from character import Character
16 from minimap import MiniMap
17 from scoreboard import NAME_MAX, ScoreBoard
18 from ui import draw_text
19 from viewport import Viewport, view_size
20
21 BOARD_ROWS = 8      # rows of the score board on the completion screen
22
23 LEG_LABELS = {"upper_left": "top-left", "upper_right": "top-right",
24              "lower_left": "bottom-left", "lower_right": "bottom-right"}
25
26
27 def apply_players(character, scheme, players):
28     """Players' limb angles -> leg targets, mouths (and head tilt) -> sticky feet."""
29     # scheme rows look like (0, "left_arm", "upper_left"): player 0's left arm
30     # drives the character's upper-left leg (see config.CONTROL_SCHEMES)
31     for player, limb, leg_name in scheme:
32         if player >= len(players):      # vision thread not switched over yet
33             continue
34         st = players[player]
35         leg = character.legs[leg_name]
36         pose = st.limbs.get(limb)
37         if pose is not None:             # arm not seen this frame: the leg keeps its last target
38             leg.set_target(pose.thigh, pose.bend)
39         # 1 player: head tilt picks a side, only that side's foot gets sticky
40         side_ok = st.sticky_side is None or limb.startswith(st.sticky_side)
41         leg.sticky = st.mouth_open and side_ok
42
43
44 class Game:
45     """run(num_players, game_map) plays until Esc ('menu') or window close ('quit').
46     Touching the flag with the head completes the level and stops the timer; the
47     team then types a name and the time goes on the score board (scores.json)."""
48
49     def __init__(self, display, input_source, debug=False):
50         self.display = display
51         self.canvas = display.canvas      # draw here; display.present() shows it
52         self.map = self.terrain = self.flag = self.view = None  # set when a game starts
53         self.input = input_source
54         self.debug = debug
55
56         # sprite scale: the 500 px character canvas becomes 1/8 of the map height
57         self.sprite_scale = config.MAP_HEIGHT * config.CHARACTER_FRACTION / config.SPRITE_CANVAS
58         self._sprites = {}               # leg stretch -> CharacterSprites
59         self._minimaps = {}              # map name -> MiniMap (the overview under the camera)
60         self.minimap = None
61         self.character = None
62         self.font = pygame.font.SysFont(config.FONT_TEXT, 20, bold=True)
63         self.timer_font = pygame.font.SysFont(config.FONT_MONO, 40, bold=True)
64         self.clock = pygame.time.Clock()  # measures frame time and limits the FPS
65         self.scheme = config.CONTROL_SCHEMES[2]
66         self.elapsed = 0.0                # timer, seconds
67         self.finished = False
68         self.scores = ScoreBoard()        # all finished runs, saved in scores.json
69         self.entering = False             # completion screen: typing the team name
70         self.name = ""                    # the name being typed (kept for the next run)

```

```

71     self.saved = None          # the score-board row of this run, once saved
72     self.new_best = False
73     self.big_font = pygame.font.SysFont(config.FONT_HEAVY, 72, bold=True)
74     self.mid_font = pygame.font.SysFont(config.FONT_HEAVY, 34, bold=True)
75     self.row_font = pygame.font.SysFont(config.FONT_HEAVY, 28, bold=True) # score board rows
76
77     def spawn_point(self):
78         return pygame.Vector2(self.map.spawn)
79
80     def build_character(self, num_players):
81         """Only the legs that someone controls, stretched for small teams."""
82         stretch = config.LEG_STRETCH.get(num_players, 1.0)
83         if stretch not in self._sprites: # build each sprite set once, then reuse it
84             self._sprites[stretch] = CharacterSprites(self.sprite_scale, stretch)
85         legs = [leg for _, leg in self.scheme]
86         return Character(self._sprites[stretch], self.spawn_point(), legs)
87
88     def restart(self):
89         self.map.reset()
90         self.character.reset(self.spawn_point()) # rest pose first...
91         self.character.reset(self.character.free_spot(self.spawn_point(), self.terrain)) # ...then a clear spot
92         self.view.snap(self.character.pos)
93         self.elapsed = 0.0
94         self.finished = False
95         self.entering = False
96         self.saved = None
97
98     # ----- loop -----
99     def run(self, num_players, game_map):
100         self.map, self.terrain, self.flag = game_map, game_map.terrain, game_map.flag
101         if game_map.name not in self._minimaps: # one overview per map, kept for next time
102             self._minimaps[game_map.name] = MiniMap(game_map)
103         self.minimap = self._minimaps[game_map.name]
104         self.view = Viewport(self.terrain.width, self.terrain.height, *view_size())
105         self.input.set_num_players(num_players)
106         self.scheme = config.CONTROL_SCHEMES[num_players]
107         self.character = self.build_character(num_players)
108         self.restart()
109         self.clock.tick() # reset the clock so the first dt is small
110         while True:
111             # tick() waits so the loop runs at most FPS times per second and returns
112             # the ms since the last frame. dt is capped at 1/20 s so a hiccup (e.g.
113             # dragging the window) can't make the physics take one giant step.
114             dt = min(self.clock.tick(config.FPS) / 1000.0, 1 / 20)
115             for event in pygame.event.get():
116                 if event.type == pygame.QUIT:
117                     return "quit"
118                 if self.display.handle_event(event): # F11 etc.
119                     continue
120                 if self.entering and self.name_key(event, num_players):
121                     continue # a key used for typing the name
122                 if event.type == pygame.KEYDOWN:
123                     if event.key == pygame.K_ESCAPE or (
124                         self.finished and event.key == pygame.K_RETURN):
125                         return "menu"
126                     if event.key == pygame.K_F5:
127                         self.restart()
128                     elif event.key == pygame.K_F1:
129                         self.debug = not self.debug
130
131             players = self.input.get_players()
132             self.apply_players(players)
133             # several small physics steps per frame: smaller steps = fewer feet
134             # passing through thin rock and a more stable solver
135             sub = dt / config.PHYSICS_SUBSTEPS
136             for _ in range(config.PHYSICS_SUBSTEPS):
137                 self.map.update(sub) # windmill rotor turns
138                 self.character.update(sub, self.terrain)
139             self.view.follow(self.character.pos, dt)
140             if not self.finished:

```

```

141         self.elapsed += dt
142         if self.flag.touches(self.character.pos, self.character.head_radius):
143             self.complete(num_players)
144
145         self.draw(players)
146
147     def complete(self, num_players):
148         """Flag reached: stop the timer and ask for the team name."""
149         self.finished = True
150         best = self.scores.best(self.map.name, num_players)
151         self.new_best = best is None or self.elapsed < best
152         self.entering, self.saved = True, None
153         pygame.key.start_text_input() # deliver typed letters as TEXTINPUT events
154
155     def name_key(self, event, num_players):
156         """Typing the team name. True if the event was used for it.
157         Enter saves, Esc skips saving; both then show the score board."""
158         if event.type == pygame.TEXTINPUT: # a typed character (works for any keyboard layout)
159             text = "".join(ch for ch in event.text if ch.isprintable())
160             self.name = (self.name + text)[:NAME_MAX]
161             return True
162         if event.type != pygame.KEYDOWN:
163             return False
164         if event.key in (pygame.K_RETURN, pygame.K_KP_ENTER):
165             if self.name.strip(): # no empty names
166                 self.saved = self.scores.add(self.name, self.map.name, num_players, self.elapsed)
167                 self.entering = False
168             elif event.key == pygame.K_BACKSPACE:
169                 self.name = self.name[:-1]
170             elif event.key == pygame.K_ESCAPE:
171                 self.entering = False # don't save, just show the board
172             else:
173                 return event.key not in (pygame.K_F1, pygame.K_F5) # letters etc. only type
174             return True
175
176     def apply_players(self, players):
177         apply_players(self.character, self.scheme, players)
178
179 # ----- drawing -----
180     def draw(self, players):
181         # Painter's order: things drawn later cover earlier ones.
182         rect = self.view.rect
183         offset = pygame.Vector2(rect.topleft)
184         self.canvas.blit(self.terrain.surface, (0, 0), rect) # copy just the visible part of the map
185         self.flag.draw(self.canvas, offset)
186         self.character.draw(self.canvas, offset, self.debug)
187         self.map.draw_movers(self.canvas, offset) # windmill tower + rotor, in front
188         self.draw_leg_owners(offset)
189         self.draw_hud(players)
190         self.draw_timer()
191         if self.finished:
192             self.draw_complete()
193         self.display.present(self.draw_minimap)
194
195     def draw_minimap(self, surface, rect):
196         """Called by the display: the whole map with the character and the visible area."""
197         return self.minimap.draw(surface, rect, self.character.pos, self.view.rect)
198
199     def draw_leg_owners(self, offset):
200         """Small dot on each knee in the colour of the player driving that leg."""
201         for player, _, leg_name in self.scheme:
202             _, knee, _ = self.character.legs[leg_name].joints(self.character.pos - offset)
203             pygame.draw.circle(self.canvas, (0, 0, 0), knee, 7) # black ring
204             pygame.draw.circle(self.canvas, config.PLAYER_COLORS_RGB[player], knee, 5)
205
206     def draw_hud(self, players):
207         """Status text in the top-left corner."""
208         y = 10
209         n = len([p for p, _, _ in self.scheme]) # number of players
210         for i in range(n):

```

```

211         st = players[i] if i < len(players) else None
212         legs = ", ".join(LEG_LABELS[leg] for p, _, leg in self.scheme if p == i)
213         state = ("STICKY" if st.mouth_open else "free") if st and st.visible else "not seen"
214         self._text(f"P{i + 1} [{legs}]: {state}", (10, y), config.PLAYER_COLORS_RGB[i])
215         y += 24
216         height = self.terrain.height - self.character.pos.y # y grows downward
217         self._text(f"{self.map.name} height {height / self.terrain.height * 100:.0f}% "
218                   f"F5 restart F1 debug F11 fullscreen Esc menu {self.clock.get_fps():.0f} FPS",
219                   (10, y), (255, 255, 255))
220
221     @staticmethod
222     def format_time(t):
223         """83.456 s -> "01:23.46"."""
224         minutes, seconds = divmod(t, 60)
225         return f"{int(minutes):02d}:{seconds:05.2f}"
226
227     def draw_complete(self):
228         """The "COMPLETED!" overlay: time, then the name entry or the score board."""
229         w, h = self.canvas.get_size()
230         shade = pygame.Surface((w, h), pygame.SRCALPHA) # see-through dark layer
231         shade.fill((0, 0, 0, 150)) # alpha 150 of 255
232         self.canvas.blit(shade, (0, 0))
233         draw_text(self.canvas, "COMPLETED!", self.big_font, (255, 215, 60), (w // 2, h * 0.12), width=5)
234         n = len({p for p, _, _ in self.scheme})
235         if self.new_best:
236             note = "NEW BEST!"
237         else:
238             note = f"best {self.format_time(self.scores.best(self.map.name, n))}"
239         draw_text(self.canvas, f"time {self.format_time(self.elapsed)} {note}", self.mid_font,
240                   (255, 255, 255), (w // 2, h * 0.25))
241         if self.entering:
242             self.draw_name_entry(w, h)
243         else:
244             self.draw_board(w, h, n)
245             draw_text(self.canvas, "Enter: menu F5: play again", self.font,
246                       (255, 255, 255), (w // 2, h * 0.94), width=2)
247
248     def draw_name_entry(self, w, h):
249         draw_text(self.canvas, "Team name for the score board:", self.mid_font,
250                   (255, 255, 255), (w // 2, h * 0.43))
251         box = pygame.Rect(0, 0, w * 0.42, h * 0.11)
252         box.center = (w // 2, h * 0.56)
253         pygame.draw.rect(self.canvas, (25, 25, 32), box, border_radius=12)
254         pygame.draw.rect(self.canvas, (255, 215, 60), box, 3, border_radius=12)
255         cursor = "|" if pygame.time.get_ticks() // 500 % 2 else " " # blinks once a second
256         draw_text(self.canvas, self.name + cursor, self.mid_font, (255, 255, 255), box.center)
257         draw_text(self.canvas, "Enter: save Esc: don't save", self.font,
258                   (255, 255, 255), (w // 2, h * 0.70), width=2)
259
260     def draw_board(self, w, h, n):
261         """Fastest runs on this map with this many players; this run highlighted."""
262         rows = self.scores.ranking(self.map.name, n)
263         title = f"BEST TIMES {self.map.name}, {n} player{'s' if n > 1 else ''}"
264         draw_text(self.canvas, title, self.mid_font, (120, 255, 120), (w // 2, h * 0.37))
265         if not rows:
266             draw_text(self.canvas, "no times saved yet", self.font, (220, 220, 220), (w // 2, h * 0.50))
267             return
268         shown = list(enumerate(rows[:BOARD_ROWS], 1)) # (place, row)
269         if self.saved is not None and self.saved not in rows[:BOARD_ROWS]:
270             # this run is further down: show it in the last line with its real place
271             shown[-1] = (rows.index(self.saved) + 1, self.saved)
272         for i, (place, row) in enumerate(shown):
273             y = h * 0.46 + i * h * 0.058
274             color = (255, 215, 60) if row is self.saved else (255, 255, 255)
275             self._cell(f"{place}.", w * 0.25, y, color, "right")
276             self._cell(self._fit(row["name"], w * 0.37), w * 0.27, y, color, "left")
277             self._cell(self._fit(row["time"], w * 0.73), w * 0.73, y, color, "right")
278             self._cell(row.get("date", "")[:10], w * 0.76, y, (180, 180, 180), "left")
279
280     def _fit(self, text, width):

```

```

281     """Shorten text with '...' until it is at most `width` px wide in the row font."""
282     if self.row_font.size(text)[0] <= width:
283         return text
284     while text and self.row_font.size(text + "...")[0] > width:
285         text = text[:-1]
286     return text + "..."
287
288     def _cell(self, text, x, y, color, align):
289         """One table cell: outlined text whose left or right edge is at x."""
290         width = self.row_font.size(text)[0]
291         cx = x + width / 2 if align == "left" else x - width / 2
292         draw_text(self.canvas, text, self.row_font, color, (cx, y), width=2)
293
294     def draw_timer(self):
295         text = self.format_time(self.elapsed)
296         color = (255, 215, 60) if self.finished else (255, 255, 255) # gold when finished
297         img = self.timer_font.render(text, True, (255, 255, 255)) # only to measure the width
298         center = (self.canvas.get_width() - img.get_width() // 2 - 20, 32) # top-right corner
299         draw_text(self.canvas, text, self.timer_font, color, center)
300
301     def _text(self, msg, pos, color):
302         """Small text with a 2 px black shadow so it reads on any background."""
303         shadow = self.font.render(msg, True, (0, 0, 0))
304         self.canvas.blit(shadow, (pos[0] + 2, pos[1] + 2))
305         self.canvas.blit(self.font.render(msg, True, color), pos)

```

scoreboard.py (70 lines)

Finish times with team names, kept in scores.json (chapter 11).

Lines	What happens there
1-9	Docstring with the file format.
10-17	Imports, file path and longest name.
20-23	Load the file at start.
25-41	Read the file; keep only well-formed rows; rename a broken file instead of crashing.
43-52	Write a temporary file, then swap it in.
54-60	Store a run with the date and save.
62-65	Runs on one map with one player count, fastest first.
67-70	The fastest time or None.

Full source

```

1  """Score board: finish times with the players' team name, kept in scores.json.
2
3  Every finished run that gets a name is stored (nothing is ever thrown away);
4  rankings are made per map and player count, fastest first. scores.json looks like
5      {"scores": [{"name": "MeEeE", "map": "Cave", "players": 2,
6                  "time": 83.46, "date": "2026-10-06 14:03"}, ...]}
7  A missing or broken file never crashes the game: it is treated as empty, and a
8  broken file is kept as scores.json.broken instead of being overwritten.
9  """
10 import datetime
11 import json
12 import os
13
14 import config
15
16 SCORES_FILE = os.path.join(config.ROOT, "scores.json")
17 NAME_MAX = 16          # longest team name (characters)
18
19
20 class ScoreBoard:
21     def __init__(self, path=SCORES_FILE):
22         self.path = path
23         self.scores = self._load()
24
25     def _load(self):
26         if not os.path.exists(self.path):
27             return []
28         try:
29             with open(self.path, encoding="utf-8") as f:
30                 rows = json.load(f)["scores"]
31                 # keep only rows that have everything with the right type
32                 return [r for r in rows if isinstance(r, dict)
33                         and isinstance(r.get("name"), str) and isinstance(r.get("map"), str)
34                         and isinstance(r.get("players"), int) and isinstance(r.get("time"), (int, float))]
35         except (OSError, ValueError, KeyError, TypeError) as e:
36             print(f"scores.json could not be read ({e}); starting an empty score board")
37             try:
38                 os.replace(self.path, self.path + ".broken")    # keep it for a human to look at
39             except OSError:
40                 pass
41             return []
42
43     def _save(self):
44         # write to a temporary file first, then swap it in: a crash half-way
45         # through writing can never leave a cut-off scores.json behind
46         tmp = self.path + ".tmp"
47         try:
48             with open(tmp, "w", encoding="utf-8") as f:
49                 json.dump({"scores": self.scores}, f, indent=2, ensure_ascii=False)
50             os.replace(tmp, self.path)
51         except OSError as e:
52             print(f"scores.json could not be saved ({e})")
53
54     def add(self, name, map_name, players, time):
55         """Store a finished run and save the file. Returns the new row."""
56         row = {"name": name.strip()[:NAME_MAX] or "?", "map": map_name, "players": players,
57               "time": round(time, 2), "date": datetime.datetime.now().strftime("%Y-%m-%d %H:%M")}
58         self.scores.append(row)
59         self._save()
60         return row
61
62     def ranking(self, map_name, players):
63         """All runs on this map with this many players, fastest first."""
64         rows = [r for r in self.scores if r["map"] == map_name and r["players"] == players]
65         return sorted(rows, key=lambda r: r["time"])
66
67     def best(self, map_name, players):
68         """Fastest time so far, or None."""
69         rows = self.ranking(map_name, players)
70         return rows[0]["time"] if rows else None

```

test_scene.py (77 lines)

1-player test screen: same input and leg logic as the game, but no physics, so the player can check the controls.

Lines	What happens there
1-2	Docstring.
3-9	Imports.
11-13	Settings: one player, bigger character, white background.
16-31	Constructor: centre position, scheme, its own (bigger) character and fonts.
33-53	Loop: events, players, <code>apply_players</code> , move only the leg angles, draw.
55-77	White background, character, the mouth state in huge letters, sticky side and head roll, help line.

Full source

```

1  """1-player test: the character hangs fixed in front of a white background so a
2  player can check arm -> leg control, mouth (sticky) and head tilt without physics."""
3  import pygame
4
5  import config
6  from assets import CharacterSprites
7  from character import Character
8  from game import apply_players
9  from ui import draw_text
10
11  PLAYERS = 1
12  TEST_CHARACTER_SCALE = 0.62  # sprite scale (canvas px per png px), bigger than in game
13  BG = (255, 255, 255)
14
15
16  class TestScene:
17      """run() until Esc ('menu') or window close ('quit')."""
18
19      def __init__(self, display, input_source):
20          self.display = display
21          self.canvas = display.canvas
22          self.input = input_source
23          w, h = self.canvas.get_size()
24          self.center = pygame.Vector2(w / 2, h * 0.56)
25          self.scheme = config.CONTROL_SCHEMES[PLAYERS]
26          sprites = CharacterSprites(TEST_CHARACTER_SCALE, config.LEG_STRETCH.get(PLAYERS, 1.0))
27          self.character = Character(sprites, self.center, [leg for _, leg in self.scheme])
28          self.huge_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.12), bold=True)
29          self.mid_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.05), bold=True)
30          self.small_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.03), bold=True)
31          self.clock = pygame.time.Clock()
32
33      def run(self):
34          self.input.set_num_players(PLAYERS)
35          self.character.reset(self.center)
36          self.clock.tick()
37          while True:
38              dt = min(self.clock.tick(config.FPS) / 1000.0, 1 / 20)
39              for event in pygame.event.get():
40                  if event.type == pygame.QUIT:
41                      return "quit"
42                  if self.display.handle_event(event):
43                      continue
44                  if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
45                      return "menu"
46              players = self.input.get_players()
47              apply_players(self.character, self.scheme, players)
48              # no Character.update() here = no physics: only the leg angles are
49              # moved towards the arm angles, the body never falls or moves
50              for leg in self.character.legs.values():  # legs move, body stays put
51                  leg.update_angles(dt)
52              self.draw(players[0] if players else None)
53              self.display.present()
54
55      def draw(self, st):
56          c = self.canvas
57          w, h = c.get_size()
58          c.fill(BG)
59          self.character.draw(c, pygame.Vector2(0, 0))  # offset 0: the position is already a canvas position
60
61          if st is None or not st.visible:
62              mouth, color = "NOT SEEN", (150, 150, 150)
63          elif st.mouth_open:
64              mouth, color = "MOUTH OPEN", (40, 200, 40)
65          else:
66              mouth, color = "MOUTH CLOSED", (220, 60, 60)
67          draw_text(c, mouth, self.huge_font, color, (w // 2, int(h * 0.11)), width=4)
68
69          if st is not None and st.sticky_side is not None:
70              side = st.sticky_side.upper()
71
72              draw_text(c, f"sticky foot: {side}", self.mid_font, (255, 255, 255),
73                      (w // 2, int(h * 0.22)), width=3)
74              draw_text(c, f"head roll {st.head_tilt:+.0f} deg (switch at "
75                      f"+{config.HEAD_TILT_ANGLE:.0f})", self.small_font, (60, 60, 60),
76                      (w // 2, int(h * 0.285)), outline=None)
77              draw_text(c, "1 PLAYER TEST  Esc: menu  F11: fullscreen", self.small_font, (90, 90, 90),
78                      (int(w * 0.2), int(h * 0.97)), outline=None)

```

options_menu.py (169 lines)

The Options screen: tabs on the left, the settings of the selected tab as sliders and switches on the right, a help line and a live readout (chapter 12).

Lines	What happens there
1-12	Docstring and imports.
14	Height of one setting row.
17-25	Value -> text: ON/OFF, or the number with the decimals of its step and its unit.
28-47	Constructor: dark backdrop and fonts.
49-53	One tab button per group.
54-65	One row per setting: label position, value position and its widget.
66-71	RESET TO DEFAULTS and BACK buttons.
73-74	The settings of the selected tab.
76-111	Loop: track 1 player on the Head tilt tab (tilt is only measured then), else up to 4; handle quit/F11/Esc/BACK (all save), Reset, tabs, switches and sliders; draw.
113-118	Make the window match the Fullscreen setting at once.
119-153	Backdrop, title, tabs, panel with rows (changed values in gold with *), live line, help line, buttons.
155-169	Live mouth scores (Mouth tab) or head roll (Head tilt tab) from the camera.

Full source

```

1  """Options screen: tune the settings in settings.SETTINGS with sliders and switches.
2
3  Changes apply immediately (they are written into `config`) and are saved to
4  settings.json when the player leaves the screen. While tuning, the Mouth and
5  Head tilt tabs show the live camera values so the thresholds can be set by
6  watching them.
7  """
8  import pygame
9
10 import config
11 from settings import GROUPS, SETTINGS
12 from ui import Button, Slider, Toggle, draw_text
13
14 ROW_H = 0.105          # height of one setting row (fraction of the canvas height)
15
16
17 def format_value(setting, value):
18     if setting.kind == "bool":
19         return "ON" if value else "OFF"
20     if setting.kind == "int" or setting.step >= 1:
21         text = f"{value:.0f}"
22     else:
23         decimals = len(f"{setting.step:g}".split(".")[1])    # 0.05 -> 2 decimals
24         text = f"{value:.{decimals}f}"
25     return f"{text} {setting.unit}".strip()
26
27
28 class OptionsScene:
29     """run() until BACK/Esc ('menu') or window close ('quit'); saves on the way out."""
30
31     def __init__(self, display, maps, input_source, settings):
32         self.display = display
33         self.canvas = display.canvas
34         self.input = input_source
35         self.settings = settings
36         w, h = self.canvas.get_size()
37         self.backdrop = pygame.transform.smoothscale(maps[0].terrain.surface, (w, h))
38         shade = pygame.Surface((w, h), pygame.SRCALPHA)
39         shade.fill((0, 0, 0, 165))
40         self.backdrop.blit(shade, (0, 0))
41
42         self.title_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.075), bold=True)
43         self.tab_font = pygame.font.SysFont(config.FONT_HEAVY, int(h * 0.032), bold=True)
44         self.label_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.034), bold=True)
45         self.value_font = pygame.font.SysFont(config.FONT_MONO, int(h * 0.034), bold=True)
46         self.help_font = pygame.font.SysFont(config.FONT_TEXT, int(h * 0.028))
47         self.clock = pygame.time.Clock()
48
49         # left column: one tab per group
50         self.tabs = []
51         for i, group in enumerate(GROUPS):
52             self.tabs.append((group, Button(group.upper(), (int(w * 0.13), int(h * (0.22 + i * 0.095))),
53                                             (int(w * 0.19), int(h * 0.075)), self.tab_font, (70, 70, 80))))
54         # right panel: one row per setting of the selected group
55         self.panel = pygame.Rect(int(w * 0.255), int(h * 0.17), int(w * 0.715), int(h * 0.62))
56         self.rows = {}          # key -> (label pos, value pos, widget)
57         for group in GROUPS:
58             items = [s for s in SETTINGS if s.group == group]
59             for i, s in enumerate(items):
60                 y = int(self.panel.y + h * (0.075 + i * ROW_H))
61                 if s.kind == "bool":
62                     widget = Toggle((int(w * 0.80), y - 16, 74, 32))
63                 else:
64                     widget = Slider((int(w * 0.66), y - 12, int(w * 0.28), 24), s.lo, s.hi, s.step)
65                 self.rows[s.key] = ((self.panel.x + 24, y), (int(w * 0.585), y), widget)
66         self.reset = Button("RESET TO DEFAULTS", (int(w * 0.36), int(h * 0.92)),
67                             (int(w * 0.25), int(h * 0.085)), self.tab_font, (190, 120, 40))
68         self.back = Button("BACK", (int(w * 0.84), int(h * 0.92)), (int(w * 0.15), int(h * 0.085)),
69                             self.tab_font, (90, 90, 90))
70         self.group = GROUPS[0]

```

```

71         self.hover_help = ""
72
73     def visible(self):
74         return [s for s in SETTINGS if s.group == self.group]
75
76     def run(self):
77         self.group = GROUPS[0]
78         while True:
79             self.clock.tick(config.FPS)
80             # head tilt is only measured in 1-player mode; elsewhere watch up to 4 people
81             self.input.set_num_players(1 if self.group == "Head tilt" else config.MAX_PLAYERS)
82             for event in pygame.event.get():
83                 if event.type == pygame.QUIT:
84                     self.settings.save()
85                     return "quit"
86                 if self.display.handle_event(event):
87                     continue
88                 pos = self.display.to_canvas(event.pos) if hasattr(event, "pos") else (-1, -1)
89                 escape = event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE
90                 if escape or self.back.clicked(event, pos):
91                     self.settings.save()
92                     return "menu"
93                 if self.reset.clicked(event, pos):
94                     self.settings.reset()
95                     self._apply_fullscreen()
96                 for group, button in self.tabs:
97                     if button.clicked(event, pos):
98                         self.group = group
99                 for s in self.visible():
100                     widget = self.rows[s.key][2]
101                     if s.kind == "bool":
102                         if widget.clicked(event, pos):
103                             self.settings.set(s.key, not self.settings.get(s.key))
104                             if s.key == "START_FULLSCREEN":
105                                 self._apply_fullscreen()
106                         else:
107                             value = widget.handle(event, pos)
108                             if value is not None:
109                                 self.settings.set(s.key, value)
110             self.draw()
111             self.display.present()
112
113     def _apply_fullscreen(self):
114         """Make the window match the Fullscreen setting right away."""
115         if self.display.fullscreen != config.START_FULLSCREEN:
116             self.display.toggle_fullscreen()
117
118     # ----- draw -----
119     def draw(self):
120         c = self.canvas
121         w, h = c.get_size()
122         mouse = self.display.to_canvas(pygame.mouse.get_pos())
123         c.blit(self.backdrop, (0, 0))
124         draw_text(c, "Options", self.title_font, (255, 215, 60), (w // 2, int(h * 0.08)), width=4)
125
126         for group, button in self.tabs:
127             button.color = pygame.Color((60, 110, 200) if group == self.group else (70, 70, 80))
128             button.draw(c, mouse)
129
130         pygame.draw.rect(c, (25, 27, 32), self.panel, border_radius=16)
131         pygame.draw.rect(c, (0, 0, 0), self.panel, 3, border_radius=16)
132         help_text = ""
133         for s in self.visible():
134             (lx, ly), (vx, vy), widget = self.rows[s.key]
135             value = self.settings.get(s.key)
136             changed = value != self.settings.defaults[s.key]
137             label = self.label_font.render(s.label + (" *" if changed else ""), True,
138                                           (255, 225, 120) if changed else (235, 235, 235))
139             c.blit(label, label.get_rect(midleft=(lx, ly)))
140             draw_text(c, format_value(s, value), self.value_font, (255, 255, 255), (vx, vy), width=1)

```

```

141         widget.draw(c, value)
142         row = pygame.Rect(self.panel.x, ly - h * ROW_H / 2, self.panel.w, h * ROW_H)
143         if row.collidepoint(mouse) or getattr(widget, "dragging", False):
144             help_text = s.help
145         live = self._live_text()
146         if live:
147             draw_text(c, live, self.label_font, (120, 230, 140),
148                     (self.panel.centerx, self.panel.bottom - int(h * 0.05)), width=1)
149         default_help = "*" = changed from the default. Changes apply at once and are saved when you leave."
150         draw_text(c, help_text or default_help, self.help_font, (210, 210, 210),
151                 (w // 2, int(h * 0.835)), width=1)
152         self.reset.draw(c, mouse)
153         self.back.draw(c, mouse)
154
155     def _live_text(self):
156         """Live camera values for the tabs where they help tuning."""
157         players = self.input.get_players()
158         if self.group == "Mouth":
159             parts = []
160             for i, st in enumerate(players):
161                 if st.visible and st.face_found:
162                     parts.append(f"P{i + 1} {st.mouth_score:.2f} {'OPEN' if st.mouth_open else 'closed'}")
163             return "Live: " + (" ".join(parts) if parts else "no face seen")
164         if self.group == "Head tilt":
165             st = players[0] if players else None
166             if st is None or st.sticky_side is None:
167                 return "Live head tilt is measured in 1-player mode (try it in 1P TEST)."
168             return f"Live: roll {st.head_tilt:+.0f} deg -> sticky foot {st.sticky_side.upper()}"
169         return ""

```

maps.py (84 lines)

Turns an entry of `config.MAPS` into a playable map.

Lines	What happens there
1-7	Docstring and imports.
9	Height of the menu picture.
12-26	Load both pictures and scale them to the common height (times the map's own scale).
27	Build collision.
28-34	Windmill maps: paint the tower into the map picture and add the rotor as a moving solid part; then place the flag and find the start point.
36-48	Make the menu thumbnail with the flag (and rotor) pasted on it.
49-63	Moving parts: reset, turn, draw big and small. Maps without them do nothing.
65-80	From the configured point: go up out of rock if needed, then find floor and ceiling and start above the floor (or mid-gap in a low tunnel).
83-84	Build every map in the list.

Full source

```

1  """Playable maps: terrain + flag + start point, built from the entries in config.MAPS."""
2  import pygame
3
4  import config
5  from flag import Flag
6  from terrain import Terrain
7  from windmill import Windmill
8
9  THUMB_HEIGHT = 230  # menu picture height (canvas px)
10
11
12  class GameMap:
13      def __init__(self, spec):
14          # spec = one dictionary from config.MAPS (name, image files, spawn, scale)
15          self.name = spec["name"]
16          bg = pygame.image.load(config.asset(spec["background"])).convert_alpha()
17          fg = pygame.image.load(config.asset(spec["foreground"])).convert_alpha()
18          # Resize so the map is MAP_HEIGHT tall (times its own "scale"). The
19          # character, flag and view are sized from MAP_HEIGHT, so they look the
20          # same on every map; "scale" makes a map bigger compared to them.
21          height = round(config.MAP_HEIGHT * spec.get("scale", 1.0))  # MAP_HEIGHT x map scale
22          scale = height / fg.get_height()
23          if abs(scale - 1) > 1e-3:
24              size = (round(fg.get_width() * scale), height)  # skip if already right
25              bg = pygame.transform.smoothscale(bg, size)  # keep the shape
26              fg = pygame.transform.smoothscale(fg, size)
27          self.terrain = Terrain(bg, fg)
28          self.windmill = None
29          if "windmill" in spec:
30              self.windmill = Windmill(spec["windmill"], scale)
31              self.terrain.surface.blit(self.windmill.base, self.windmill.base_pos)  # tower: just a picture
32              self.terrain.movers.append(self.windmill)  # rotor: solid
33          self.flag = Flag(self.terrain)
34          self.spawn = self._find_spawn(*spec["spawn"])
35
36          # Small picture for the map-selection menu: the whole map shrunk to
37          # THUMB_HEIGHT tall, with a shrunk flag pasted at the same relative spot.
38          t = self.terrain.surface
39          thumb = (round(t.get_width() * THUMB_HEIGHT / t.get_height()), THUMB_HEIGHT)
40          self.thumbnail = pygame.transform.smoothscale(t, thumb)
41          self.thumbnail.blit(pygame.transform.smoothscale(
42              self.flag.image, [max(1, round(v * THUMB_HEIGHT / t.get_height()))
43                              for v in self.flag.image.get_size()]),
44              self.flag.pos * THUMB_HEIGHT / t.get_height())
45          if self.windmill:
46              self.windmill.draw_small(self.thumbnail, (0, 0), THUMB_HEIGHT / t.get_height())
47
48          # --- moving parts (windmill rotor); maps without them do nothing here ---
49          def reset(self):
50              if self.windmill:
51                  self.windmill.reset()
52
53          def update(self, dt):
54              if self.windmill:
55                  self.windmill.update(dt)
56
57          def draw_movers(self, target, offset):
58              if self.windmill:
59                  self.windmill.draw(target, offset)
60
61          def draw_movers_small(self, target, corner, k):
62              if self.windmill:
63                  self.windmill.draw_small(target, corner, k)
64
65          def _find_spawn(self, fx, fy):
66              """Drop from (fx, fy) (map fractions) to the floor below; start a little above it,
67              but not higher than the middle of the gap (so it fits in tunnels)."""
68              t = self.terrain
69              x, y = fx * t.width, fy * t.height
70              while y > 0 and t.distance(x, y) <= 0:  # start point in rock: go up into air

```

```

71     y -= 2
72     floor = y
73     while floor < t.height - 1 and t.distance(x, floor) > 0:    # walk down to the floor
74         floor += 2
75     ceiling = y
76     while ceiling > 0 and t.distance(x, ceiling) > 0:          # walk up to the ceiling (or top)
77         ceiling -= 2
78     lift = config.MAP_HEIGHT * config.SPAWN_LIFT
79     # normally start `lift` above the floor; in a low tunnel start at its middle
80     return pygame.Vector2(x, max(floor - lift, (floor + ceiling) / 2))
81
82
83 def load_maps():
84     return [GameMap(spec) for spec in config.MAPS]

```

windmill.py (156 lines)

The windmill: a tower picture and a solid, spinning rotor (chapter 7).

Lines	What happens there
1-14	Docstring: how the rotor collision works and how feet ride along.
15-23	Imports.
24-25	Empty border of the rotor's table; radius of the hub piece.
28-44	Speed, slipperiness, hub position (scaled to the map), angle; crop and scale the tower picture.
46-52	Crop the rotor picture to its content and scale it.
54-66	Distance table of the still rotor with a border, and how far it reaches.
68-93	Cut the rotor into three blades and the hub for fast drawing.
94-96	Back to angle 0.
98-102	Turn by $360 \times \text{rps} \times \text{dt}$; remember this turn and the cosine/sine for lookups.
104-107	Speed of the rotor surface at a point (for limiting pushes).
109-113	Turn a point stuck on the rotor by the last turn.
114-133	Far away: a safe estimate. Near: turn the point back and look it up in the table.
134-144	Draw the tower, then rotate and draw the rotor pieces inside the view (after the character, so the windmill is in front of it).
146-156	The rotor for the map panel and the menu picture.

Full source

```

1  """Windmill: a decorative tower (no collision) and a spinning rotor that is solid.
2
3  The rotor's collision works like the terrain's (a signed distance field, see
4  terrain.py), but the table is made once for the rotor standing still. To ask
5  "how far is point p from the rotor right now?", p is turned back by the
6  rotor's current angle around the hub and looked up in that still table.
7
8  Feet glued to the rotor (or gripping it by friction) ride along: carry() turns
9  their glue point with the rotor every physics step, so the rotor can lift the
10 character and fling it when the foot lets go.
11
12 Angles: degrees, clockwise on screen (pygame.Vector2.rotate turns that way
13 because y points down).
14 """
15 import math
16
17 import numpy as np
18 import pygame
19
20 import config
21 from terrain import Terrain
22
23 V = pygame.Vector2
24 PAD = 48 # px of empty border around the rotor in its distance table
25 HUB_RADIUS = 70 # px around the hub drawn as one piece (the ball and the blade roots)
26
27
28 class Windmill:
29     def __init__(self, spec, scale):
30         """spec = the "windmill" dictionary of a map in config.MAPS (image names,
31         hub position in image px, turns per second); scale = image px -> map px."""
32         self.rps = spec["rps"]
33         self.walkable_slope = config.ROTOR_WALKABLE_SLOPE # non-sticky feet slip on steeper blade parts
34         self.hub = V(spec["hub"]) * scale # rotation centre, map px
35         self.angle = 0.0 # current rotation (deg)
36         self.turn = 0.0 # rotation of the last update (deg)
37         self._cos, self._sin = 1.0, 0.0 # of -angle, for distance()
38
39         # tower: crop the big canvas to the tower, scale it, remember where it goes on the map
40         base = pygame.image.load(config.asset(spec["base"])).convert_alpha()
41         crop = base.get_bounding_rect(min_alpha=1).inflate(4, 4).clip(base.get_rect())
42         self.base = pygame.transform.smoothscale(
43             base.subsurface(crop), (round(crop.w * scale), round(crop.h * scale)))
44         self.base_pos = V(crop.topleft) * scale
45
46         # rotor: crop the big canvas to the rotor, then scale to map size
47         img = pygame.image.load(config.asset(spec["blade"])).convert_alpha()
48         crop = img.get_bounding_rect(min_alpha=1).inflate(4, 4).clip(img.get_rect())
49         size = (round(crop.w * scale), round(crop.h * scale))
50         rotor = pygame.transform.smoothscale(img.subsurface(crop), size)
51         self.corner = V(crop.topleft) * scale # rotor image corner on the map (angle 0)
52         self.rotor = rotor
53
54         # collision table of the still rotor, with a PAD px empty border so points
55         # near the rotor's edge still get a correct (positive) distance
56         alpha = pygame.surfarray.array_alpha(rotor).T > 127 # (h, w), True = solid
57         solid = np.zeros((alpha.shape[0] + 2 * PAD, alpha.shape[1] + 2 * PAD), bool)
58         solid[PAD:-PAD, PAD:-PAD] = alpha
59         self.sdf = Terrain._signed_distance(solid)
60         self.origin = self.corner - V(PAD, PAD) # map position of sdf[0, 0]
61         ys, xs = np.nonzero(alpha) # how far the rotor reaches from the hub
62         self.reach = float(np.hypot(xs + self.corner.x - self.hub.x,
63                                   ys + self.corner.y - self.hub.y).max())
64
65         self.pieces = self._split(rotor)
66         self._small = None # (k, rotor image) for the overview map
67
68     def _split(self, rotor):
69         """Cut the rotor into one piece per blade (120 deg slices around the hub) and
70         the hub ball.

```

```

71     Turning three thin pieces each frame is much faster than turning the one
72     big square picture, and pieces outside the view are skipped."""
73     w, h = rotor.get_size()
74     xs, ys = np.meshgrid(np.arange(w), np.arange(h))           # pixel coordinates
75     hub = self.hub - self.corner                               # hub inside the rotor image
76     ang = np.degrees(np.arctan2(ys - hub.y, xs - hub.x))        # direction of each pixel from the hub
77     near = np.hypot(xs - hub.x, ys - hub.y) <= HUB_RADIUS      # the hub ball: one piece, no seams
78     pieces = []
79     for k in range(4):
80         if k == 3:
81             inside = near
82         else:
83             centre = -90 + 120 * k                               # up, lower right, lower left
84             inside = (np.abs((ang - centre + 180) % 360 - 180) <= 60) & ~near
85         part = rotor.copy()
86         pygame.surfarray.pixels_alpha(part)[...] *= inside.T    # clear the other two slices
87         rect = part.get_bounding_rect(min_alpha=1)
88         piece = part.subsurface(rect).copy()
89         offset = V(rect.center) - hub                            # piece centre relative to the hub
90         pieces.append((piece, offset, math.hypot(*rect.size) / 2))
91     return pieces
92
93 # ----- motion -----
94 def reset(self):
95     self.angle, self.turn = 0.0, 0.0
96     self._cos, self._sin = 1.0, 0.0
97
98 def update(self, dt):
99     self.turn = 360.0 * self.rps * dt
100    self.angle = (self.angle + self.turn) % 360.0
101    a = math.radians(-self.angle)
102    self._cos, self._sin = math.cos(a), math.sin(a)
103
104 def velocity_at(self, p):
105     """How fast the rotor surface at point p moves (px/s)."""
106     r = V(p) - self.hub
107     return V(-r.y, r.x) * math.radians(360.0 * self.rps)    # clockwise on screen
108
109 def carry(self, p):
110     """Where a point stuck on the rotor is after the last update."""
111     return self.hub + (V(p) - self.hub).rotate(self.turn)
112
113 # ----- collision -----
114 def distance(self, x, y):
115     """Signed distance from (x, y) to the rotor at its current angle."""
116     dx, dy = x - self.hub.x, y - self.hub.y
117     r = math.hypot(dx, dy)
118     if r > self.reach + PAD // 2:
119         return r - self.reach    # far away: a safe (never too big) estimate
120     # turn the point back by the rotor angle -> position in the still table
121     lx = self.hub.x + dx * self._cos - dy * self._sin - self.origin.x
122     ly = self.hub.y + dx * self._sin + dy * self._cos - self.origin.y
123     hgt, wid = self.sdf.shape
124     if not (0 <= lx < wid - 1.001 and 0 <= ly < hgt - 1.001):
125         return float(PAD)    # inside the reach circle but beside the rotor's box
126     x0, y0 = int(lx), int(ly)    # bilinear sample, as in Terrain.distance
127     fx, fy = lx - x0, ly - y0
128     s = self.sdf
129     top = s[y0, x0] * (1 - fx) + s[y0, x0 + 1] * fx
130     bot = s[y0 + 1, x0] * (1 - fx) + s[y0 + 1, x0 + 1] * fx
131     return float(top * (1 - fy) + bot * fy)
132
133 # ----- drawing -----
134 def draw(self, target, offset):
135     """Draw the tower, then the rotor at its current angle; offset = top-left of the
136     view. Called after the character, so the windmill is in front of it."""
137     view = target.get_rect()
138     target.blit(self.base, self.base_pos - offset)
139     for piece, centre, radius in self.pieces:
140         pos = self.hub + centre.rotate(self.angle) - offset    # piece centre on screen

```

```

141         if not view.colliderect((pos.x - radius, pos.y - radius, 2 * radius, 2 * radius)):
142             continue # off screen: skip the slow part
143         img = pygame.transform.rotate(piece, -self.angle) # pygame turns counter-clockwise
144         target.blit(img, img.get_rect(center=(round(pos.x), round(pos.y))))
145
146     def draw_small(self, target, corner, k):
147         """Rotor for the overview map: whole map shrunk by k, map (0, 0) at `corner`."""
148         if self._small is None or self._small[0] != k:
149             size = (max(1, round(self.rotor.get_width() * k)), max(1, round(self.rotor.get_height() * k)))
150             small = pygame.transform.smoothscale(self.rotor, size)
151             self._small = (k, small, (self.hub - self.corner) * k)
152             _, small, hub = self._small
153             img = pygame.transform.rotate(small, -self.angle)
154             centre = V(small.get_size()) / 2 - hub # image centre relative to the hub
155             pos = V(corner) + self.hub * k + centre.rotate(self.angle)
156             target.blit(img, img.get_rect(center=(round(pos.x), round(pos.y))))

```

terrain.py (116 lines)

Map picture and collision via a signed distance field (see chapter 7).

Lines	What happens there
1-10	Docstring explaining the signed distance field.
11-17	Imports.
20-33	Constructor: compose the map picture (sky, background, foreground).
35-43	Build the solid mask from the foreground alpha, add invisible side walls, compute the distance field.
44-46	Moving solid parts (the windmill rotor) are added here by the map.
47-58	Two distance transforms (air -> rock, rock -> air) subtracted.
59-64	Distance to the nearest solid thing: rock or a moving part.
66-74	Which moving part a point touches, if any (for feet that ride the rotor).
76-95	Smooth lookup of the rock distance with special rules outside the map.
97-105	Direction out of the rock from the distance gradient.
107-116	First rock below the open sky in a column (used for the flag).

Full source

```

1  """Map image + collision. The opaque pixels of foreground.png are solid ground.
2
3  Collision uses a "signed distance field" (SDF): a big table with one number per
4  map pixel, saying how far that pixel is from the nearest rock edge:
5      > 0 in the air (e.g. +20 = 20 px away from the nearest rock)
6      < 0 inside rock (e.g. -5 = 5 px deep inside)
7      = 0 exactly on the surface
8  With it, "is this foot inside the ground, how deep, and which way is out?"
9  is just a table lookup, no matter how wiggly the rock outline is.
10 """
11 import math
12
13 import cv2
14 import numpy as np
15 import pygame
16
17 import config
18
19
20 class Terrain:
21     """Holds the rendered map and a signed distance field for collisions.
22
23     distance(p) > 0 in the air, < 0 inside rock; normal(p) points out of the rock.
24     """
25
26     def __init__(self, background, foreground):
27         self.width, self.height = foreground.get_size()
28         # One flat picture of the whole map (sky colour, then background, then
29         # foreground on top). The game just copies a rectangle of it each frame.
30         self.surface = pygame.Surface((self.width, self.height)).convert()
31         self.surface.fill(config.SKY_COLOR)
32         self.surface.blit(background, (0, 0))
33         self.surface.blit(foreground, (0, 0))
34
35         # Alpha channel of the foreground = how opaque each pixel is (0..255).
36         # array_alpha gives it as [x][y]; .T flips it to [y][x] (row, column),
37         # which is how numpy/OpenCV index images. More than half opaque = rock.
38         solid = pygame.surfarray.array_alpha(foreground).T > 127 # (h, w)
39         solid[:, :2] = True # invisible walls on the map's left/right edges
40         solid[:, -2:] = True
41         self.sdf = self._signed_distance(solid)
42         # moving solid parts (e.g. a windmill rotor). Each has distance(x, y);
43         # distance() below reports the nearest of the rock and all of them.
44         self.movers = []
45
46     @staticmethod
47     def _signed_distance(solid):
48         # cv2.distanceTransform: for every non-zero pixel, distance to the nearest
49         # zero pixel. Run it twice: once on the air (distance to rock) and once on
50         # the rock (distance to air); air distance minus rock distance gives one
51         # signed number per pixel.
52         air = (~solid).astype(np.uint8)
53         rock = solid.astype(np.uint8)
54         out = cv2.distanceTransform(air, cv2.DIST_L2, 5) # air px -> distance to rock
55         inside = cv2.distanceTransform(rock, cv2.DIST_L2, 5)
56         return (out - inside).astype(np.float32)
57
58     # ----- queries -----
59     def distance(self, x, y):
60         """Signed distance to the nearest solid thing: rock or a moving part."""
61         d = self.rock_distance(x, y)
62         for m in self.movers:
63             d = min(d, m.distance(x, y))
64         return d
65
66     def mover_at(self, x, y):
67         """The moving part a point touches (closer to it than to the rock), or None.
68         Used to make a foot that grabs a rotor ride along with it."""
69         best, d = None, self.rock_distance(x, y)
70         for m in self.movers:

```

```

71         dm = m.distance(x, y)
72         if dm < d:
73             best, d = m, dm
74     return best
75
76     def rock_distance(self, x, y):
77         """Bilinear sample of the SDF. Above the map the top row continues upward:
78         where it is sky, open sky; where it is rock (side walls, a solid top
79         border like the cave's), rock that gets deeper going up, so anything
80         poking through is pushed back down into the map instead of escaping."""
81         outside = max(0.0, -x, x - (self.width - 1.001)) # past the side walls = deeper in rock
82         above = max(0.0, -y) # how far above the map's top edge
83         # clamp the point into the table so we never index outside it
84         x = min(max(x, 0.0), self.width - 1.001)
85         y = min(max(y, 0.0), self.height - 1.001)
86         # Bilinear interpolation: the point usually lies between 4 table cells;
87         # blend them by how close the point is to each (fx, fy = 0..1 fractions).
88         # This makes the distance change smoothly instead of in 1 px steps.
89         x0, y0 = int(x), int(y)
90         fx, fy = x - x0, y - y0
91         s = self.sdf
92         top = s[y0, x0] * (1 - fx) + s[y0, x0 + 1] * fx
93         bot = s[y0 + 1, x0] * (1 - fx) + s[y0 + 1, x0 + 1] * fx
94         d = float(top * (1 - fy) + bot * fy) - outside
95         return d - above if d < 0 else d # rock continues (deeper) above the map; sky stays sky
96
97     def normal(self, x, y, h=2.0):
98         """Unit vector pointing out of the rock = direction the distance grows fastest.
99         Measured by sampling the distance h px left/right and up/down (the gradient)."""
100         dx = self.distance(x + h, y) - self.distance(x - h, y)
101         dy = self.distance(x, y + h) - self.distance(x, y - h)
102         n = math.hypot(dx, dy) # length of (dx, dy)
103         if n < 1e-6: # flat spot: no direction, guess "up"
104             return pygame.Vector2(0, -1)
105         return pygame.Vector2(dx / n, dy / n) # divide by length -> length 1
106
107     def ground_y(self, x):
108         """Top of the ground under the open sky in column x: the first rock below the
109         first air from the top (skips a solid border along the map's top edge).
110         0 if the column is rock from the top down (no sky)."""
111         col = self.sdf[:, int(min(max(x, 0), self.width - 1))] # one column of the table
112         air = np.nonzero(col > 0)[0] # row numbers that are air
113         if not len(air) or air[0] > self.height * 0.02: # no sky near the top
114             return 0
115         rock = np.nonzero(col[air[0]:] < 0)[0] # rock rows below that air
116         return int(air[0] + rock[0]) if len(rock) else self.height

```

flag.py (47 lines)

The goal flag.

Lines	What happens there
1-4	Docstring and imports.
7-17	Load and size the flag; build its pixel mask.
19-32	Find the right-most ground with room for the flag and take the highest point near it; place the pole foot there.
34-43	Pixel-perfect test: draw the head as a circle mask and check overlap with the flag mask.
45-47	Draw at its screen position.

Full source

```

1  """Goal flag: stands on the top-right peak; touching it with the head wins."""
2  import pygame
3
4  import config
5
6
7  class Flag:
8      def __init__(self, terrain):
9          img = pygame.image.load(config.asset("flag.png")).convert_alpha()
10         # Size: FLAG_FRACTION of the reference map height, so the flag is the same
11         # size on every map no matter how big that map is.
12         scale = config.MAP_HEIGHT * config.FLAG_FRACTION / img.get_height() # same size on every map
13         self.image = pygame.transform.smoothscale(
14             img, (round(img.get_width() * scale), round(img.get_height() * scale)))
15         # A mask is a yes/no grid of which pixels are visible; used for exact
16         # pixel-perfect touch detection (not just the rectangle around the flag).
17         self.mask = pygame.mask.from_surface(self.image)
18
19         # stand on the highest ground near the right edge, fully inside the map:
20         # find the right-most column with room above it for the whole flag, then
21         # take the highest such ground within FLAG_SEARCH of the map width from there
22         base_x, base_y = config.FLAG_BASE[0] * scale, config.FLAG_BASE[1] * scale # pole foot in the image
23         x_max = int(terrain.width - (self.image.get_width() - base_x)) # cloth must not leave the map
24         # columns (right to left) whose ground is low enough that the flag fits above it
25         fits = [x for x in range(x_max, int(base_x), -1) if terrain.ground_y(x) >= base_y]
26         if not fits:
27             fits = [x_max]
28         window = [x for x in fits if x >= fits[0] - terrain.width * config.FLAG_SEARCH]
29         peak_x = min(window, key=terrain.ground_y) # smallest y = highest ground on screen
30         # place the image so its pole foot (base_x, base_y) lands on the ground at peak_x
31         self.pos = pygame.Vector2(peak_x - base_x, terrain.ground_y(peak_x) - base_y)
32         self.rect = self.image.get_rect(topleft=self.pos)
33
34     def touches(self, center, radius):
35         """True if a circle (the head) overlaps the flag's visible pixels."""
36         r = int(radius)
37         # draw the head as a filled circle on a small transparent image ...
38         circle = pygame.Surface((2 * r + 1, 2 * r + 1), pygame.SRCALPHA)
39         pygame.draw.circle(circle, (255, 255, 255), (r, r), r)
40         # ... and ask pygame if its mask overlaps the flag mask at that position
41         # (offset = where the circle image's corner sits relative to the flag's corner)
42         offset = (int(center[0] - r - self.rect.x), int(center[1] - r - self.rect.y))
43         return self.mask.overlap(pygame.mask.from_surface(circle), offset) is not None
44
45     def draw(self, target, offset):
46         # offset = top-left of the visible view; map position - offset = screen position
47         target.blit(self.image, self.pos - offset)

```

viewport.py (44 lines)

The game camera (the visible part of the map).

Lines	What happens there
1-9	Docstring.
12-15	View size: 1/3 of the reference height, mountain map's shape.
18-22	Holds the map size, view size and top-left position.
24-28	Keep the view inside the map.
30-32	Jump to a target.
34-41	Glide towards a target (fast when far, slow when close).
42-44	Visible area as a rectangle.

Full source

```

1  """Game camera: a window onto the map that smoothly follows the character.
2
3  The map is much bigger than the screen; the viewport is the rectangle of the
4  map that is currently shown. Its top-left corner `pos` is what everything is
5  drawn relative to (screen position = map position - pos).
6  """
7  import pygame
8
9  import config
10
11
12  def view_size():
13      """Size of the game view in map pixels (the same for every map, see config.MAP_HEIGHT)."""
14      h = round(config.MAP_HEIGHT * config.VIEW_FRACTION) # 2164 / 3 = 721
15      return round(h * config.VIEW_ASPECT), h # 721 * (4096 / 2164) = 1365
16
17
18  class Viewport:
19      def __init__(self, map_w, map_h, w, h):
20          self.map_w, self.map_h = map_w, map_h
21          self.w, self.h = w, h
22          self.pos = pygame.Vector2() # top-left corner in map pixels
23
24      def _clamped(self, p):
25          """Keep the view inside the map (centred if the map is smaller than the view)."""
26          def clamp(v, size, view):
27              return (size - view) / 2 if size <= view else min(max(v, 0), size - view)
28          return pygame.Vector2(clamp(p.x, self.map_w, self.w), clamp(p.y, self.map_h, self.h))
29
30      def snap(self, target):
31          """Jump straight to `target` (centred), e.g. at a restart."""
32          self.pos = self._clamped(target - pygame.Vector2(self.w, self.h) / 2)
33
34      def follow(self, target, dt):
35          """Glide towards `target`: each frame cover a fraction (CAMERA_LERP * dt) of
36          the remaining distance, so the camera moves fast when far and slows down
37          as it arrives (smooth, no jitter)."""
38          goal = self._clamped(target - pygame.Vector2(self.w, self.h) / 2)
39          self.pos += (goal - self.pos) * min(1.0, config.CAMERA_LERP * dt)
40
41      @property
42      def rect(self):
43          """The visible part of the map as a pygame.Rect (whole pixels)."""
44          return pygame.Rect(int(self.pos.x), int(self.pos.y), self.w, self.h)

```

character.py (329 lines)

The heart of the game: legs, kinematics, the physics step and drawing (see chapter 8).

Lines	What happens there
1-13	Docstring: how the physics works.
14-20	Imports and the short vector name V .
23-31	Angle helpers: wrap to -180..180 and turn towards a target the short way.
34-58	One leg: geometry (scaled and stretched), rest pose, current and target angles, sticky state.
59-60	Store the angles the player asks for.
62-67	Turn towards the target at most LEG_MAX_SPEED.
69-75	Outward angles -> screen rotations.
76-87	Hip, knee and foot positions (kinematics).
90-100	The character: head radius and the controlled legs.
102-112	Back to the start pose.
114-128	Find a start point where the head and all feet are clear of the rock (long 1-player legs would otherwise start inside the ground and be thrown up).
129-154	One physics step with the 'undo and hold the leg' check.
156-217	The position-based physics step: move glue/grip points that sit on the rotor, glue, predict, solve, cap, velocity.

Lines	What happens there
219-232	Save / restore everything a step changes.
234-236	How deep a foot is in the rock.
238-246	Feet deeper than allowed and deeper than before.
248-279	Glued: pull towards the anchor. Free: push out of rock, then static friction on walkable ground (much less on the rotor).
281-286	Limit a glued foot's depth.
288-297	Push the head circle out of the rock.
299-312	Contact flags, tear off over-stretched glue, forget grips of lifted feet.
313-329	Legs then head; shoe colour by state; optional debug circles.

Full source

```

1  """The four-legged character: leg kinematics, physics and drawing.
2
3  Physics is position based: every step the body falls, then contacts push the
4  body out of the rock. Because legs are rigid, a foot that is pressed into the
5  ground by its leg pushes the body away -> extending a grounded leg fast jumps.
6  Grounded feet on walkable slopes don't slide (static friction), so sweeping a
7  planted leg moves the body like walking. Sticky feet are anchored both ways,
8  so they can pull the body too (climb walls, hang from ledges).
9
10 Only the body (head) has a position and velocity. Legs have no mass: each is
11 just two angles (thigh, bend) that follow the player's arm. Where the feet are
12 follows from the body position plus those angles ("kinematics").
13 """
14 import math
15
16 import pygame
17
18 import config
19
20 V = pygame.Vector2  # short name: V(x, y) is a 2D vector with +, -, *, rotate, length...
21
22
23 def wrap_deg(a):
24     """Bring any angle into -180..180 (e.g. 270 -> -90), so 'the shortest way round' works."""
25     return (a + 180.0) % 360.0 - 180.0
26
27
28 def approach_angle(current, target, max_step):
29     """Move `current` towards `target` by at most `max_step` degrees, the short way round."""
30     diff = wrap_deg(target - current)
31     return current + max(-max_step, min(max_step, diff))
32
33
34 class Leg:
35     def __init__(self, name, hip, side, thigh, bend, scale, stretch=1.0):
36         self.name = name
37         self.side = side  # -1 left, +1 right
38         self.hip = V(hip) * scale  # offset from body centre
39         self.length = config.SEGMENT_LENGTH * stretch * scale  # hip -> knee distance
40         # the shin stick stretches, the shoe below it keeps its size (see assets.stretch_leg)
41         extra = (config.SHOE_TOP - config.LEG_PIVOT[1]) * (stretch - 1)
42         # which way the toe points when the shin hangs down: outward normally (= side),
43         # inward for legs in config.MIRRORED_SHOES (they point up, which flips it back out)
44         self.shoe_side = -side if name in config.MIRRORED_SHOES else side
45         # knee -> shoe centre while the shin hangs straight down; x follows the toe direction
46         self.foot_offset = V(self.shoe_side * config.FOOT_OFFSET[0], config.FOOT_OFFSET[1] + extra) * scale
47         self.foot_radius = config.FOOT_RADIUS * scale
48         self.rest = (thigh, bend)  # pose used at (re)start
49         self.thigh, self.bend = thigh, bend  # current angles (outward convention)
50         self.target_thigh, self.target_bend = thigh, bend  # angles the player asks for
51
52         self.sticky = False  # player wants this foot sticky (mouth open)
53         self.anchor = None  # world point a sticky foot is glued to
54         self.grip = None  # world point a grounded foot grips by friction
55         self.anchor_on = self.grip_on = None  # moving part (rotor) the anchor/grip rides on
56         self.contact = False  # foot touching the ground this step
57
58     # ----- angles -----
59     def set_target(self, thigh, bend):
60         self.target_thigh, self.target_bend = thigh, bend
61
62     def update_angles(self, dt):
63         """Turn towards the target, at most LEG_MAX_SPEED deg/s. The limit smooths
64         out jerky arm tracking and caps how hard a leg can kick."""
65         step = config.LEG_MAX_SPEED * dt
66         self.thigh = wrap_deg(approach_angle(self.thigh, self.target_thigh, step))
67         self.bend = wrap_deg(approach_angle(self.bend, self.target_bend, step))
68
69     def screen_angles(self):
70         """Rotation (deg, pygame convention) of thigh and shin sprites."""

```

```

71     # "outward" angles are mirrored for left legs: multiplying by side (-1 / +1)
72     # turns them into one screen rotation. shin direction = thigh + bend.
73     return self.side * self.thigh, self.side * (self.thigh + self.bend)
74
75     # ----- kinematics -----
76     def joints(self, body):
77         """World positions of hip, knee and the foot (shoe) centre."""
78         a_thigh, a_shin = self.screen_angles()
79         hip = body + self.hip
80         # V(0, length) is the thigh hanging straight down; rotating it gives the
81         # thigh's real direction. Same for the shin, starting at the knee.
82         knee = hip + V(0, self.length).rotate(-a_thigh)
83         foot = knee + self.foot_offset.rotate(-a_shin)
84         return hip, knee, foot
85
86     def foot(self, body):
87         return self.joints(body)[2]
88
89
90     class Character:
91     def __init__(self, sprites, spawn, leg_names=tuple(config.LEGS)):
92         self.sprites = sprites
93         s = sprites.scale
94         self.head_radius = config.HEAD_RADIUS * s
95         # only the legs someone controls (1 player = 2 legs, otherwise 4)
96         self.legs = {}
97         for name in leg_names:
98             hip, side, thigh, bend = config.LEGS[name]
99             self.legs[name] = Leg(name, hip, side, thigh, bend, s, sprites.leg_stretch)
100         self.reset(spawn)
101
102     def reset(self, spawn):
103         """Back to the start: standing still at `spawn`, legs in their rest pose."""
104         self.pos = V(spawn) # body (head) centre in map pixels
105         self.vel = V() # px per second
106         self.head_contact = False
107         for leg in self.legs.values():
108             leg.thigh, leg.bend = leg.rest
109             leg.set_target(*leg.rest)
110             leg.anchor = leg.grip = None
111             leg.anchor_on = leg.grip_on = None
112             leg.contact = False
113
114     def free_spot(self, start, terrain):
115         """A start point near `start` where the head and every foot (in the rest pose)
116         are out of the rock: lift it up, or if a ceiling is in the way, try a little
117         to the side. Without this, long legs can start inside the ground and the
118         solver throws the character into the air."""
119         for dx in (0, 40, -40, 80, -80, 120, -120, 160, -160):
120             for dy in range(0, -400, -4):
121                 p = V(start[0] + dx, start[1] + dy)
122                 if terrain.distance(*p) < self.head_radius + 2:
123                     break # head would hit a ceiling: try another column
124                 if all(terrain.distance(*leg.foot(p)) >= leg.foot_radius + 2 for leg in self.legs.values()):
125                     return p
126         return V(start)
127
128     # ----- physics -----
129     def update(self, dt, terrain):
130         """One physics step. While a foot is glued the body is pinned and can't
131         always get out of the way, so a leg could be forced into the rock. Then
132         the step is redone with that leg held still: it stops at the rock.
133         Feet may only sink in a little: free feet LEG_BLOCK_TOLERANCE, sticky
134         feet STICKY_SINK (looks like gripping). Glued feet pushed in by another
135         leg are kept within STICKY_SINK by the solver (_cap_sink)."""
136         before = self._snapshot() # to undo this step if needed
137         depth0 = {leg.name: self._foot_depth(leg, terrain) for leg in self.legs.values()}
138         old = {leg.name: (leg.thigh, leg.bend) for leg in self.legs.values()}
139         for leg in self.legs.values():
140             leg.update_angles(dt)

```

```

141     new = {leg.name: (leg.thigh, leg.bend) for leg in self.legs.values()}
142     self._step(dt, terrain)
143
144     # Did a moving leg drive its own foot into the rock while a foot is glued?
145     sunk = self._sunk_feet(depth0, terrain)
146     if not sunk or not any(leg.anchor is not None for leg in self.legs.values()):
147         return
148     hold = [leg for leg in sunk if new[leg.name] != old[leg.name]]
149     if hold:
150         # undo the step and redo it with those legs not moving: they stop at the rock
151         self._restore(before)
152         for leg in hold:
153             leg.thigh, leg.bend = old[leg.name]
154         self._step(dt, terrain)
155
156     def _step(self, dt, terrain):
157         self._rotor_hits = [] # (rotor, normal, point) of free feet / head pushed by a rotor
158         # 0) glue and grip points on a moving part (windmill rotor) move with it
159         for leg in self.legs.values():
160             if leg.anchor is not None and leg.anchor_on is not None:
161                 leg.anchor = leg.anchor_on.carry(leg.anchor)
162             if leg.grip is not None and leg.grip_on is not None:
163                 leg.grip = leg.grip_on.carry(leg.grip)
164
165         # 1) sticky feet: glue to the rock when close enough; let go when not sticky
166         for leg in self.legs.values():
167             if not leg.sticky:
168                 leg.anchor = None
169             elif leg.anchor is None:
170                 foot = leg.foot(self.pos)
171                 if terrain.distance(*foot) < leg.foot_radius + config.STICKY_GRAB_DIST:
172                     leg.anchor = foot
173                     leg.anchor_on = terrain.mover_at(*foot) # None = still rock
174
175         # 2) predict: gravity speeds the body up, drag slows it a little, then move
176         start = V(self.pos)
177         self.vel.y += config.GRAVITY * dt # +y is down on screen
178         self.vel *= max(0.0, 1.0 - config.AIR_DRAG * dt)
179         self.pos += self.vel * dt
180         predicted = V(self.pos)
181
182         # 3) solve: fix contacts by moving the body. Each fix can disturb another,
183         # so go round all of them a few times (Gauss-Seidel iterations).
184         for _ in range(config.SOLVER_ITERATIONS):
185             for leg in self.legs.values():
186                 self._solve_foot(leg, terrain)
187             self._solve_head(terrain)
188         for _ in range(2): # last word goes to the rock: no glued foot deeper than allowed
189             for leg in self.legs.values():
190                 if leg.anchor is not None:
191                     self._cap_sink(leg, terrain)
192
193         # 4) velocity = how far the body really moved this step / time.
194         # This is what turns a fast leg push into a jump: the push moved the
195         # body, so it now has speed and keeps flying after the foot leaves.
196         self._update_contacts(terrain)
197         self.vel = (self.pos - start) / dt
198         # A rotor that overlapped a leg deeply pushes it out in one step, which would
199         # look like a huge kick. Leaving a blade may only be ROTOR_PUSH_SPEED faster
200         # than the blade itself moves there (glued feet are not limited).
201         for rotor, n, p in self._rotor_hits:
202             away = self.vel.dot(n) - rotor.velocity_at(p).dot(n)
203             if away > config.ROTOR_PUSH_SPEED:
204                 self.vel -= n * (away - config.ROTOR_PUSH_SPEED)
205         if self.head_contact and not any(l.contact for l in self.legs.values()):
206             self.vel *= max(0.0, 1.0 - config.HEAD_FRICTION * dt) # head sliding on rock slows down
207         if self.pos != predicted: # something pushed or pulled the body
208             if self.vel.length() > config.MAX_SPEED: # pushes and kicks: limit the speed
209                 self.vel.scale_to_length(config.MAX_SPEED)
210         else:

```

```

211         # in free flight only the fall is limited. Limiting the whole speed here
212         # would stop the fall from speeding up after a fast sideways fling, and
213         # the character would drift down slowly as if gliding.
214         self.vel.y = min(self.vel.y, config.MAX_SPEED)
215     # stay inside the map horizontally and not below its bottom
216     self.pos.x = min(max(self.pos.x, 0), terrain.width)
217     self.pos.y = min(self.pos.y, terrain.height)
218
219     def _snapshot(self):
220         """Copy of everything _step changes, so a step can be undone."""
221         legs = {leg.name: (V(leg.anchor) if leg.anchor is not None else None,
222                             V(leg.grip) if leg.grip is not None else None,
223                             leg.contact, leg.anchor_on, leg.grip_on) for leg in self.legs.values()}
224         return V(self.pos), V(self.vel), self.head_contact, legs
225
226     def _restore(self, snap):
227         pos, vel, self.head_contact, legs = snap
228         self.pos, self.vel = V(pos), V(vel)
229         for leg in self.legs.values():
230             anchor, grip, leg.contact, leg.anchor_on, leg.grip_on = legs[leg.name]
231             leg.anchor = V(anchor) if anchor is not None else None
232             leg.grip = V(grip) if grip is not None else None
233
234     def _foot_depth(self, leg, terrain):
235         """How far the foot circle is inside the rock (<= 0: outside)."""
236         return leg.foot_radius - terrain.distance(*leg.foot(self.pos))
237
238     def _sunk_feet(self, depth0, terrain):
239         """Feet deeper in the rock than allowed and deeper than before this step."""
240         out = []
241         for leg in self.legs.values():
242             limit = config.STICKY_SINK if leg.sticky else config.LEG_BLOCK_TOLERANCE
243             d = self._foot_depth(leg, terrain)
244             if d > max(limit, depth0[leg.name]) + 0.05: # no creeping in, step by step
245                 out.append(leg)
246         return out
247
248     def _solve_foot(self, leg, terrain):
249         foot = leg.foot(self.pos)
250         if leg.anchor is not None: # sticky: pull or push
251             # move the body part of the way so the foot gets back to its glue point
252             self.pos += (leg.anchor - foot) * config.STICKY_STIFFNESS
253             self._cap_sink(leg, terrain) # ...but never deeper than STICKY_SINK
254             return
255         d = terrain.distance(*foot) # distance of the shoe centre to rock
256         if d >= leg.foot_radius: # shoe circle fully in the air: nothing to do
257             return
258         n = terrain.normal(*foot)
259         mover = terrain.mover_at(*foot) if terrain.movers else None # touching the rotor?
260         if mover is not None:
261             self._rotor_hits.append((mover, n, foot))
262             self.pos += n * (leg.foot_radius - d) # push out of the rock
263             # Friction only on walkable ground: -n.y is how much the surface faces up
264             # (1 = flat floor, 0 = vertical wall). Steeper than WALKABLE_SLOPE: slide.
265             # A moving part (windmill rotor) has its own, much smaller limit, so a
266             # non-sticky foot slips off it easily (glued feet never get here).
267             slope = mover.walkable_slope if mover is not None else config.WALKABLE_SLOPE
268             if -n.y < math.cos(math.radians(slope)):
269                 leg.grip = None # too steep, slide
270                 return
271         foot = leg.foot(self.pos)
272         if leg.grip is None: # first touch: remember where
273             leg.grip = V(foot)
274             leg.grip_on = mover # standing on the rotor: grip rides along
275         # tangent = along the surface. Move the body so the foot does not slide
276         # along the surface away from its grip point. If the leg sweeps backwards,
277         # the foot stays put and the BODY moves forward instead = walking.
278         tangent = V(-n.y, n.x)
279         self.pos += tangent * (leg.grip - foot).dot(tangent) # static friction
280

```

```

281 def _cap_sink(self, leg, terrain):
282     """Push the body so this glued foot is at most STICKY_SINK inside the rock."""
283     foot = leg.foot(self.pos)
284     excess = leg.foot_radius - config.STICKY_SINK - terrain.distance(*foot)
285     if excess > 0:
286         self.pos += terrain.normal(*foot) * excess
287
288 def _solve_head(self, terrain):
289     """The head is a circle too: push it out of the rock like a foot (no friction)."""
290     d = terrain.distance(*self.pos)
291     self.head_contact = d < self.head_radius + config.CONTACT_EPS
292     if d < self.head_radius:
293         n = terrain.normal(*self.pos)
294         mover = terrain.mover_at(*self.pos) if terrain.movers else None
295         if mover is not None:
296             self._rotor_hits.append((mover, n, V(self.pos)))
297         self.pos += n * (self.head_radius - d)
298
299 def _update_contacts(self, terrain):
300     """After solving: which feet touch the ground, and tear off over-stretched glue."""
301     for leg in self.legs.values():
302         foot = leg.foot(self.pos)
303         if leg.anchor is not None:
304             leg.contact = True
305             if foot.distance_to(leg.anchor) > config.STICKY_BREAK_DIST:
306                 leg.anchor = None # torn off
307             continue
308         leg.contact = terrain.distance(*foot) < leg.foot_radius + config.CONTACT_EPS
309         if not leg.contact:
310             leg.grip = None # lifted: forget the friction point
311
312 # ----- drawing -----
313 def draw(self, target, offset, debug=False):
314     body = self.pos - offset # map position -> screen position
315     sp = self.sprites
316     for leg in self.legs.values(): # legs first, so the head covers the hips
317         hip, knee, foot = leg.joints(body)
318         a_thigh, a_shin = leg.screen_angles()
319         sp.thigh[leg.side].draw(target, hip, a_thigh)
320         # shoe colour: purple = glued to rock, green = sticky in the air, red = normal
321         shin = (sp.shin_stuck if leg.anchor is not None
322                else sp.shin_sticky if leg.sticky else sp.shin)
323         shin[leg.shoe_side].draw(target, knee, a_shin)
324         if debug: # F1: collision circles (green glued, yellow touching, red in air)
325             color = (0, 255, 0) if leg.anchor else (255, 255, 0) if leg.contact else (255, 0, 0)
326             pygame.draw.circle(target, color, foot, leg.foot_radius, 1)
327     sp.head.draw(target, body, 0)
328     if debug:
329         pygame.draw.circle(target, (255, 0, 255), body, self.head_radius, 1)

```

assets.py (110 lines)

Prepares the character pictures (see chapter 10).

Lines	What happens there
1-9	Docstring and imports.
12-17	An image plus the joint it rotates around.
19-29	Rotate around the joint (see chapter 10).
31-35	Mirror image with mirrored joint.
38-47	Crop the empty canvas away, scale, move the joint along.
50-64	Make a leg picture longer without stretching the shoe.
67-82	Turn the red shoe green or purple while keeping its shading.
85-110	Build all pictures once: head, thigh, shin in red/green/purple, each for left and right.

Full source

```

1  """Character images: scaled, cropped, mirrored and tinted copies of the part pngs.
2
3  Everything here is done once when a character is built, so the game loop only
4  has to rotate and paste ready-made images.
5  """
6  import numpy as np
7  import pygame
8
9  import config
10
11
12  class PivotSprite:
13      """An image that is drawn rotated around a pivot point (e.g. a joint)."""
14
15      def __init__(self, image, pivot):
16          self.image = image
17          self.pivot = pygame.Vector2(pivot)  # pivot position inside the image (px)
18
19      def draw(self, target, pos, angle):
20          """Draw with the pivot at `pos`. angle (deg) turns the 'hanging down'
21          sprite counter-clockwise on screen, same as pygame.transform.rotate."""
22          # rotozoom rotates (smoothly) but makes a bigger image with the picture
23          # centred in it, so the pivot moves. Fix: take the vector from the pivot to
24          # the image centre, rotate that vector by the same angle, and put the
25          # rotated image's centre at pos + that vector -> the pivot ends at pos.
26          img = pygame.transform.rotozoom(self.image, angle, 1.0)
27          centre = pygame.Vector2(self.image.get_size()) / 2 - self.pivot
28          # Vector2.rotate turns the other way on screen (y points down), hence -angle
29          target.blit(img, img.get_rect(center=pos + centre.rotate(-angle)))
30
31      def flipped(self):
32          """Mirror image (left <-> right); the pivot is mirrored too."""
33          w = self.image.get_width()
34          return PivotSprite(pygame.transform.flip(self.image, True, False),
35                             (w - self.pivot.x, self.pivot.y))
36
37
38  def _crop_scale(surface, pivot, scale):
39      """Crop a canvas-sized png to its content and scale it; keep the pivot aligned."""
40      # bounding rect = smallest rectangle around the visible pixels (+2 px margin);
41      # cropping the empty canvas away makes rotating each frame much cheaper
42      rect = surface.get_bounding_rect(min_alpha=1).inflate(4, 4).clip(surface.get_rect())
43      part = surface.subsurface(rect).copy()
44      size = (max(1, round(rect.w * scale)), max(1, round(rect.h * scale)))
45      part = pygame.transform.smoothscale(part, size)
46      # the pivot was measured on the full canvas: shift it by the crop, then scale it
47      return PivotSprite(part, ((pivot[0] - rect.x) * scale, (pivot[1] - rect.y) * scale))
48
49
50  def stretch_leg(surface, k, split):
51      """Lengthen a leg png by k: rows between the pivot and `split` are stretched,
52      rows below `split` (the shoe) are only moved down. The pivot stays put."""
53      if k == 1:
54          return surface
55      w, h = surface.get_size()
56      top = int(config.LEG_PIVOT[1])
57      split = min(split, h)
58      mid_h = round((split - top) * k)  # new height of the stick part
59      out = pygame.Surface((w, top + mid_h + h - split), pygame.SRCALPHA)  # transparent canvas
60      out.blit(surface, (0, 0), (0, 0, w, top))  # 1) above the pivot: unchanged
61      mid = surface.subsurface((0, top, w, split - top))
62      out.blit(pygame.transform.smoothscale(mid, (w, mid_h)), (0, top))  # 2) the stick: stretched
63      out.blit(surface, (0, top + mid_h), (0, split, w, h - split))  # 3) the shoe: moved down
64      return out
65
66
67  def recolor_red(surface, weights):
68      """Filter that recolours the red shoe while keeping its shading.
69
70      On reddish pixels the 'redness' (R above the grey level) is redistributed

```

```

71     over the channels with `weights` (r, g, b), e.g. (0, 1, 0) = green.
72     """
73     out = surface.copy()
74     rgb = pygame.surfarray.pixels3d(out)      # numpy view of the pixels: writes change `out`
75     r, g, b = (rgb[..., i].astype(np.float32) for i in range(3))
76     mask = (r > g + 40) & (r > b + 40)        # "clearly red" pixels only (not white laces/black lines)
77     grey = (g + b) / 2                       # brightness without the red part
78     for i, w in enumerate(weights):
79         # new channel = grey + redness * weight -> bright red stays bright, dark stays dark
80         rgb[..., i][mask] = np.clip(grey + (r - grey) * w, 0, 255)[mask].astype(np.uint8)
81     del rgb                                  # release the pixel lock so the surface can be drawn
82     return out
83
84
85 class CharacterSprites:
86     """Head, thigh and shin(+shoe) sprites at game scale.
87
88     Leg sprites hang straight down from their pivot; shoes point right (+x),
89     so left legs use mirrored copies to keep the toes pointing outward.
90     """
91
92     def __init__(self, scale, leg_stretch=1.0):
93         def load(name):
94             return pygame.image.load(config.asset(name)).convert_alpha()
95
96         self.scale = scale                    # game px per png px
97         self.leg_stretch = leg_stretch        # 1.5 for the 1-player long legs
98         self.head = _crop_scale(load("head.png"), config.HEAD_CENTER, scale)
99         thigh = _crop_scale(stretch_leg(load("leg-part.png"), leg_stretch, config.SPRITE_CANVAS),
100                             config.LEG_PIVOT, scale)
101         shin = _crop_scale(stretch_leg(load("leg-part-with-shoe.png"), leg_stretch, config.SHOE_TOP),
102                             config.LEG_PIVOT, scale)
103         green = PivotSprite(recolor_red(shin.image, (0.0, 1.0, 0.0)), shin.pivot)
104         purple = PivotSprite(recolor_red(shin.image, (0.7, 0.0, 1.0)), shin.pivot)
105
106         # [side] -> sprite, side = -1 (left) or +1 (right)
107         self.thigh = {+1: thigh, -1: thigh.flipped()}
108         self.shin = {+1: shin, -1: shin.flipped()}          # normal: red shoe
109         self.shin_sticky = {+1: green, -1: green.flipped()} # sticky, in the air
110         self.shin_stuck = {+1: purple, -1: purple.flipped()} # sticky and glued to rock

```

inputs.py (67 lines)

The input interface and the keyboard input.

Lines	What happens there
1-7	Docstring and imports.
10-27	The interface: every input offers these methods.
30-45	Keyboard input: key layout and mouth keys.
47-49	Start every leg at its rest pose.
51-67	Turn held keys into angle changes, then hand each leg's pose to the player/limb that drives it.

Full source

```

1  """Input sources. The game only talks to InputSource, so camera and keyboard are swappable."""
2  import time
3
4  import pygame
5
6  import config
7  from player_state import LimbPose, PlayerState
8
9
10 class InputSource:
11     """The interface every input must offer. Subclasses override these methods;
12     the game calls them without knowing whether a camera or a keyboard is behind."""
13     num_players = 1
14
15     def set_num_players(self, n):
16         self.num_players = n
17
18     def get_players(self):
19         """List of PlayerState, one per player slot."""
20         raise NotImplementedError # a subclass must provide this
21
22     def get_preview(self):
23         """(RGB numpy frame or None, is_new) for the camera panel."""
24         return None, False
25
26     def close(self):
27         pass
28
29
30 class KeyboardInput(InputSource):
31     """Debug controls without a camera. Keys move a character leg directly; the
32     pose is handed to whichever player/limb drives that leg in the current scheme.
33
34     top-left    Q/A thigh, W/S knee    top-right    E/D thigh, R/F knee
35     bottom-left U/J thigh, I/K knee    bottom-right O/L thigh, P/; knee
36     hold 1-4 = that player's mouth open (sticky)
37     """
38     SPEED = 240.0 # deg/s
39     KEYS = { # leg -> (thigh +, thigh -, bend +, bend -)
40         "upper_left": (pygame.K_q, pygame.K_a, pygame.K_w, pygame.K_s),
41         "upper_right": (pygame.K_e, pygame.K_d, pygame.K_r, pygame.K_f),
42         "lower_left": (pygame.K_u, pygame.K_j, pygame.K_i, pygame.K_k),
43         "lower_right": (pygame.K_o, pygame.K_l, pygame.K_p, pygame.K_SEMICOLON),
44     }
45     MOUTH_KEYS = [pygame.K_1, pygame.K_2, pygame.K_3, pygame.K_4]
46
47     def __init__(self):
48         self.poses = {leg: LimbPose(*config.LEGS[leg][2:]) for leg in self.KEYS} # start at rest pose
49         self._t = time.time()
50
51     def get_players(self):
52         now = time.time()
53         step = self.SPEED * min(now - self._t, 0.1) # degrees to turn since last call
54         self._t = now
55         keys = pygame.key.get_pressed() # which keys are held right now
56         for leg, (tp, tm, bp, bm) in self.KEYS.items():
57             pose = self.poses[leg]
58             # True/False count as 1/0: (plus held) - (minus held) = +1, -1 or 0
59             pose.thigh += step * (keys[tp] - keys[tm])
60             pose.bend += step * (keys[bp] - keys[bm])
61
62         states = [PlayerState(visible=True, face_found=True, mouth_open=bool(keys[self.MOUTH_KEYS[i]]))
63                   for i in range(self.num_players)]
64         # hand each leg's pose to the player/limb that controls that leg
65         for player, limb, leg in config.CONTROL_SCHEMES[self.num_players]:
66             states[player].limbs[limb] = self.poses[leg]
67         return states

```

vision.py (384 lines)

Everything about the webcam: the background thread, both AI models and the per-player trackers (see chapter 9 and Part V).

Lines	What happens there
1-12	Docstring with the per-frame pipeline.
13-30	Imports.
31-40	One detected body: keypoints, confidences, box, face, player number.
41-45	Box size (closeness to camera).
46-51	Left/right position.
53-56	Average of visible head keypoints.
58-62	Body size for distance limits.
64-80	Square crop around the head for the face fallback.
83-101	Load YOLO on the fastest backend (TensorRT > GPU > CPU).
103-112	Run YOLO, convert results to Person objects.
115-135	Load MediaPipe (download the model if missing): VIDEO-mode for frames, IMAGE-mode for crops.
136-139	BGR numpy -> MediaPipe RGB image.
141-149	Faces on the whole frame, as pixel arrays.
151-165	Face in a head crop, mapped back to frame pixels.
167-170	Release the models.
173-192	Assign faces to bodies by pixel distance, closest first.
195-211	Number the biggest people left to right.
214-219	Lip gap / mouth width.
222-241	Smoothing, hysteresis and timeout for one player's mouth.
244-274	Ear-line roll angle -> sticky side, with a dead zone.
277-299	The camera input: models, camera, lock, thread start.
301-302	Ask the thread to switch player count.
304-314	Fresh trackers for a new player count.
315-333	The background loop: read, mirror, process, FPS, preview, store under the lock.
335-368	One frame -> PlayerStates.
369-371	Newest states (game thread).
373-378	Newest preview picture (game thread).
380-384	Stop the thread, release camera and models.

Full source

```

1  """Webcam -> players. YOLO pose for bodies, MediaPipe Face Landmarker for mouths.
2
3  Faces are found on the whole frame and given to the body whose head keypoints
4  are closest in pixels. Players are numbered left -> right on the mirrored image.
5  Everything runs in a background thread so the game keeps its frame rate.
6
7  Per camera frame (VisionInput._loop):
8      read frame -> mirror -> YOLO pose (bodies) -> pick/number players
9      -> MediaPipe faces -> match faces to bodies -> limb angles, mouth, head tilt
10     -> PlayerState per player + annotated preview picture for the game window
11 """
12 import importlib.util
13 import os
14 import threading
15 import time
16 import urllib.request
17 from dataclasses import dataclass
18
19 import cv2
20 import mediapipe as mp
21 import numpy as np
22 from mediapipe.tasks.python import BaseOptions, vision as mp_vision
23
24 import config
25 from inputs import InputSource
26 from player_state import L_SH, R_SH, LIMBS, AngleSmoother, PlayerState, limb_poses
27 from preview import PreviewRenderer
28
29
30 @dataclass
31 class Person:
32     """One body found by the pose model in the current frame."""
33     xy: np.ndarray          # (17, 2) keypoints, pixels
34     conf: np.ndarray        # (17,)
35     box: np.ndarray         # x1, y1, x2, y2
36     face: np.ndarray | None = None # (478, 2) face landmarks, pixels
37     player: int | None = None
38
39     # @property: used like a value (p.area), computed when read
40     @property
41     def area(self):
42         """Size of the body's box; bigger = closer to the camera."""
43         return float((self.box[2] - self.box[0]) * (self.box[3] - self.box[1]))
44
45     @property
46     def center_x(self):
47         """Where the person stands left/right: middle of the shoulders (or of the box)."""
48         ok = self.conf[[L_SH, R_SH]] > config.KEYPOINT_THR
49         if ok.all():
50             return float(self.xy[[L_SH, R_SH], 0].mean())
51         return float((self.box[0] + self.box[2]) / 2)
52
53     def head_center(self):
54         """Average of the visible nose/eye/ear keypoints (0-4), or None."""
55         ok = self.conf[:5] > config.KEYPOINT_THR
56         return self.xy[:5][ok].mean(axis=0) if ok.any() else None
57
58     def shoulder_width(self):
59         """Body size in pixels, used to scale distance limits (far people are small)."""
60         if (self.conf[[L_SH, R_SH]] > config.KEYPOINT_THR).all():
61             return float(np.linalg.norm(self.xy[L_SH] - self.xy[R_SH]))
62         return float(self.box[2] - self.box[0]) / 2
63
64     def head_box(self, shape):
65         """Square crop around nose/eyes/ears (fallback face search)."""
66         pts = self.xy[:5][self.conf[:5] > config.KEYPOINT_THR]
67         if len(pts) < 2:
68             return None
69         cx, cy = pts.mean(axis=0)
70         # span = how spread out the head keypoints are (np.ptp = max - min)

```

```

71     span = max(np.ptp(pts[:, 0]), np.ptp(pts[:, 1]), config.MIN_CROP / config.CROP_SCALE)
72     cy += config.CROP_DOWN * span # eyes/ears sit above the mouth: shift down
73     half = span * config.CROP_SCALE / 2
74     h, w = shape[:2]
75     # clip the square to the picture borders
76     x1, y1 = int(max(cx - half, 0)), int(max(cy - half, 0))
77     x2, y2 = int(min(cx + half, w)), int(min(cy + half, h))
78     if x2 - x1 < config.MIN_CROP or y2 - y1 < config.MIN_CROP:
79         return None
80     return x1, y1, x2, y2
81
82
83 class PoseDetector:
84     """YOLO pose on the fastest backend available:
85     TensorRT engine (GPU) > PyTorch on GPU > PyTorch on CPU."""
86
87     def __init__(self, model_path=config.POSE_MODEL):
88         import torch # slow imports, only needed in camera mode
89         from ultralytics import YOLO
90
91         gpu = torch.cuda.is_available() # NVIDIA GPU usable?
92         engine = os.path.splitext(model_path)[0] + ".engine" # yolo26n-pose.engine
93         # the engine file alone is not enough: the tensorrt package must be installed too
94         trt = importlib.util.find_spec("tensorrt") is not None
95         if gpu and trt and config.USE_TENSORRT and model_path.endswith(".pt") and os.path.exists(engine):
96             model_path, self.backend = engine, "TensorRT"
97         else:
98             self.backend = "GPU" if gpu else "CPU"
99             self.device = 0 if gpu else "cpu"
100            self.model = YOLO(model_path, task="pose")
101            print(f"Pose model: {model_path} on {self.backend}")
102
103     def detect(self, frame):
104         """All people in the frame as Person objects (keypoints + box)."""
105         r = self.model.predict(frame, verbose=False, device=self.device)[0] # [0] = first (only) image
106         if r.keypoints is None or len(r.keypoints) == 0 or r.keypoints.conf is None:
107             return []
108         # results live on the GPU as torch tensors: .cpu().numpy() makes plain arrays
109         xy = r.keypoints.xy.cpu().numpy()
110         cf = r.keypoints.conf.cpu().numpy()
111         boxes = r.bboxes.xyxy.cpu().numpy()
112         return [Person(p, c, b) for p, c, b in zip(xy, cf, boxes)]
113
114
115 class FaceDetector:
116     """MediaPipe Face Landmarker: 478 points per face (eyes, lips, outline...)."""
117
118     def __init__(self, max_faces):
119         if not os.path.exists(config.FACE_MODEL):
120             print("Downloading face_landmarker.task ...")
121             urllib.request.urlretrieve(config.FACE_URL, config.FACE_MODEL)
122
123         def make(mode, n):
124             return mp_vision.FaceLandmarker.create_from_options(mp_vision.FaceLandmarkerOptions(
125                 base_options=BaseOptions(model_asset_path=config.FACE_MODEL),
126                 running_mode=mode, num_faces=n))
127
128         # VIDEO mode: for one continuous stream (it reuses the last frame's result
129         # to track faces faster). IMAGE mode: every picture is independent, needed
130         # for the head crops, which come from different people each time.
131         self.full = make(mp_vision.RunningMode.VIDEO, max_faces)
132         self.crop = make(mp_vision.RunningMode.IMAGE, 1) if config.FACE_CROP_FALLBACK else None
133         self._last_ts = -1
134
135     @staticmethod
136     def _mp_image(bgr):
137         # OpenCV pictures are BGR, MediaPipe wants RGB in one continuous memory block
138         rgb = np.ascontiguousarray(cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB))
139         return mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
140

```

```

141 def detect_full(self, frame):
142     """Faces on the whole frame, as (478, 2) arrays of pixel positions."""
143     # VIDEO mode needs a timestamp (ms) that always increases
144     ts = max(int(time.monotonic() * 1000), self._last_ts + 1)
145     self._last_ts = ts
146     res = self.full.detect_for_video(self._mp_image(frame), ts)
147     h, w = frame.shape[:2]
148     # landmarks come as 0..1 fractions of the picture: multiply by its size
149     return [np.array([(p.x * w, p.y * h) for p in face]) for face in res.face_landmarks]
150
151 def detect_crop(self, frame, box):
152     """Face inside one head crop (fallback for faces too small on the full frame)."""
153     if self.crop is None:
154         return None
155     x1, y1, x2, y2 = box
156     crop = frame[y1:y2, x1:x2]
157     ch, cw = crop.shape[:2]
158     scale = max(1.0, config.UPSCALE_TO / max(ch, cw))
159     if scale > 1.0:
160         # enlarge tiny crops so the model sees detail
161         crop = cv2.resize(crop, None, fx=scale, fy=scale, interpolation=cv2.INTER_LINEAR)
162     res = self.crop.detect(self._mp_image(crop))
163     if not res.face_landmarks:
164         return None
165     # crop fractions -> crop pixels -> full-frame pixels (add the crop's corner)
166     return np.array([(p.x * cw + x1, p.y * ch + y1) for p in res.face_landmarks[0]])
167
168 def close(self):
169     self.full.close()
170     if self.crop:
171         self.crop.close()
172
173 def match_faces(persons, faces):
174     """Give each face to the person whose head keypoints are closest (greedy)."""
175     # 1) every (person, face) pair that is close enough, with its pixel distance
176     pairs = []
177     for i, p in enumerate(persons):
178         head = p.head_center()
179         if head is None:
180             continue
181         limit = max(config.FACE_MATCH_MIN_PX, config.FACE_MATCH_FACTOR * p.shoulder_width())
182         for j, f in enumerate(faces):
183             d = float(np.linalg.norm(f.mean(axis=0) - head)) # face centre <-> head centre
184             if d < limit:
185                 pairs.append((d, i, j))
186     # 2) closest pairs first; each person and each face is used at most once
187     used_p, used_f = set(), set()
188     for _, i, j in sorted(pairs):
189         if i not in used_p and j not in used_f:
190             persons[i].face = faces[j]
191             used_p.add(i)
192             used_f.add(j)
193
194 def assign_players(persons, frame_w, n):
195     """Pick the n biggest people and number them left -> right on screen."""
196     chosen = sorted(persons, key=lambda p: -p.area)[:n] # biggest = closest = the players
197     chosen.sort(key=lambda p: p.center_x) # then order them left to right
198     if len(chosen) == n:
199         slots = list(range(n))
200     else: # fewer people than players: use the screen region they stand in
201         slots, taken = [], set()
202         for p in chosen:
203             # split the picture into n equal strips; the strip you stand in = your number
204             s = min(n - 1, max(0, int(p.center_x / frame_w * n)))
205             while s in taken:
206                 s = (s + 1) % n
207             taken.add(s)
208             slots.append(s)
209     for p, s in zip(chosen, slots):

```

```

211     p.player = s
212
213
214 def mouth_ratio(pts):
215     """Lip gap / mouth width. Face-mesh points 13/14 = middle of the upper/lower
216     lip, 61/291 = the mouth corners. Dividing by the width makes the number the
217     same whether the face is near (big) or far (small) from the camera."""
218     width = np.linalg.norm(pts[61] - pts[291])
219     return float(np.linalg.norm(pts[13] - pts[14]) / width) if width else 0.0
220
221
222 class MouthTracker:
223     """Smoothed mouth opening with hysteresis, per player slot."""
224
225     def __init__(self):
226         self.score, self.open, self.last_seen = 0.0, False, 0.0
227
228     def update(self, pts, now):
229         if pts is None:
230             # face not found this frame
231             if now - self.last_seen > config.MOUTH_TIMEOUT:
232                 self.open, self.score = False, 0.0
233             return
234             # short drop-outs keep the last state
235         self.last_seen = now
236         # smoothed score (EMA): half new measurement, half previous value
237         self.score = config.MOUTH_EMA * mouth_ratio(pts) + (1 - config.MOUTH_EMA) * self.score
238         # Hysteresis: open above 0.35, closed below 0.25, in between keep the state.
239         # One single threshold would flicker open/closed when the score hovers near it.
240         if self.open and self.score < config.MOUTH_CLOSE_T:
241             self.open = False
242         elif not self.open and self.score > config.MOUTH_OPEN_T:
243             self.open = True
244
245
246 class HeadTilt:
247     """Which side sticks, from the sideways tilt (roll) of the head.
248
249     roll = angle of the line from the screen-left ear to the screen-right ear
250     (the eyes are used if an ear is not seen). On the mirrored image the
251     screen-right ear is the player's right ear, so roll > 0 means the right
252     ear is lower = head tilted to the right. Tilt right -> right side, tilt
253     left -> left side (HEAD_TILT_INVERT swaps this). Inside +-HEAD_TILT_ANGLE
254     the last side is kept.
255     """
256     PAIRS = ((3, 4), (1, 2)) # ears, then eyes
257
258     def __init__(self):
259         self.side = "right"
260         self.roll = 0.0
261
262     def update(self, person):
263         ok = person.conf > config.KEYPOINT_THR
264         for a, b in self.PAIRS:
265             if ok[a] and ok[b]:
266                 left, right = sorted((person.xy[a], person.xy[b]), key=lambda p: p[0])
267                 # slope angle of the line left ear -> right ear (0 = level head)
268                 self.roll = float(np.degrees(np.arctan2(right[1] - left[1], right[0] - left[0])))
269                 break
270             else:
271                 # for-else: runs when the loop found no usable pair
272                 return self.side
273         tilt_right = self.roll > config.HEAD_TILT_ANGLE
274         tilt_left = self.roll < -config.HEAD_TILT_ANGLE
275         if tilt_right or tilt_left:
276             self.side = "right" if tilt_right != config.HEAD_TILT_INVERT else "left"
277         return self.side
278
279
280 class VisionInput(InputSource):
281     """Camera input. A background thread does all detection; the game thread only
282     picks up the newest results. A lock makes sure the game never reads a result
283     while the thread is half-way through replacing it."""

```

```

281
282 def __init__(self, source=config.CAMERA_SOURCE, pose_model=config.POSE_MODEL):
283     self.pose = PoseDetector(pose_model)
284     self.faces = FaceDetector(config.MAX_PLAYERS + 2)
285     self.renderer = PreviewRenderer()
286     PreviewRenderer.backend = self.pose.backend
287     self.cap = cv2.VideoCapture(int(source) if str(source).isdigit() else source)
288     if not self.cap.isOpened():
289         raise SystemExit(f"Could not open camera/video source {source!r}")
290
291     self._lock = threading.Lock()
292     self._preview = None
293     self._new_preview = False
294     self._reset_players(self.num_players)
295     self._wanted_n = self.num_players
296     self._running = True
297     # daemon thread: does not keep Python alive when the game window closes
298     self._thread = threading.Thread(target=self._loop, daemon=True)
299     self._thread.start()
300
301 def set_num_players(self, n):
302     self._wanted_n = n          # picked up by the worker thread
303
304 def _reset_players(self, n):
305     """Fresh per-player trackers for a new player count (and its control scheme)."""
306     self.n = n
307     self.scheme = config.CONTROL_SCHEMES[n]
308     self.smoothers = [{limb: AngleSmoother() for limb in LIMBS} for _ in range(n)]
309     self.mouths = [MouthTracker() for _ in range(n)]
310     self.tilts = [HeadTilt() for _ in range(n)] if n in config.HEAD_TILT_PLAYERS else None
311     with self._lock:
312         self._states = [PlayerState() for _ in range(n)]
313
314 # ----- worker thread -----
315 def _loop(self):
316     fps, t_prev = 0.0, time.time()
317     while self._running:
318         if self._wanted_n != self.n:
319             self._reset_players(self._wanted_n)
320         ok, frame = self.cap.read()
321         if not ok:
322             time.sleep(0.01)
323             continue
324         if config.MIRROR_CAMERA:
325             frame = cv2.flip(frame, 1)          # 1 = flip left/right, like a mirror
326         states, persons = self._process(frame)
327         now = time.time()
328         fps = 0.9 * fps + 0.1 / max(now - t_prev, 1e-6)    # smoothed frames per second
329         t_prev = now
330         preview = cv2.cvtColor(self.renderer.draw(frame, persons, states, self.scheme, fps),
331                                cv2.COLOR_BGR2RGB)
332         with self._lock:
333             self._states, self._preview, self._new_preview = states, preview, True
334
335 def _process(self, frame):
336     """One camera frame -> one PlayerState per player slot."""
337     now = time.time()
338     persons = self.pose.detect(frame)          # bodies
339     assign_players(persons, frame.shape[1], self.n)    # who is P1, P2, ...
340     match_faces(persons, self.faces.detect_full(frame))    # faces -> bodies
341
342     states = [PlayerState() for _ in range(self.n)]
343     by_slot = {p.player: p for p in persons if p.player is not None}
344     for slot in range(self.n):
345         p = by_slot.get(slot)
346         st = states[slot]
347         if p is not None:
348             if p.face is None:                # retry on a head crop
349                 box = p.head_box(frame.shape)
350                 if box is not None:

```

```

351         p.face = self.faces.detect_crop(frame, box)
352         poses = limb_poses(p.xy, p.conf)
353         st.visible = True
354         for limb, smoother in self.smoothers[slot].items():
355             value = smoother.update(poses.get(limb))
356             if value is not None:
357                 st.limbs[limb] = value
358             st.face_found = p.face is not None
359         if self.tilts is not None:
360             tilt = self.tilts[slot]
361             st.sticky_side = tilt.update(p) if p is not None else tilt.side
362             st.head_tilt = tilt.roll
363         mouth = self.mouths[slot]
364         mouth.update(p.face if p is not None else None, now)
365         st.mouth_open, st.mouth_score = mouth.open, mouth.score
366         return states, persons
367
368     # ----- main thread -----
369     def get_players(self):
370         with self._lock:
371             return list(self._states)
372
373     def get_preview(self):
374         """Newest annotated camera frame (RGB numpy array) and whether it is new."""
375         with self._lock:
376             frame, fresh = self._preview, self._new_preview
377             self._new_preview = False
378             return frame, fresh
379
380     def close(self):
381         self._running = False
382         self._thread.join(timeout=2)
383         self.cap.release()
384         self.faces.close()

```

player_state.py (111 lines)

Data passed from input to game, and the arm-angle maths (see chapter 9).

Lines	What happens there
1-9	Docstring with the 17 COCO keypoints.
10-25	Imports and keypoint index constants.
26-32	Two angles from one limb.
33-41	Everything known about one player this frame.
44-53	Angle helpers.
56-62	Which keypoints form which limb, by screen side.
65-92	Torso direction and the two angles for every visible limb.
93-111	Wrap-safe exponential moving average.

Full source

```

1  """What the game needs to know about each human player, plus the limb-angle math.
2
3  The pose model gives 17 "keypoints" per person in COCO order:
4      0 nose, 1 left eye, 2 right eye, 3 left ear, 4 right ear,
5      5 left shoulder, 6 right shoulder, 7 left elbow, 8 right elbow,
6      9 left wrist, 10 right wrist, 11 left hip, 12 right hip,
7      13 left knee, 14 right knee, 15 left ankle, 16 right ankle
8  Here they are turned into two angles per limb, which drive one character leg.
9  """
10 import math
11 from dataclasses import dataclass, field
12
13 import numpy as np
14
15 import config
16
17 # COCO keypoints used here
18 L_SH, R_SH, L_HIP, R_HIP = 5, 6, 11, 12
19 ARM_CHAINS = [(5, 7, 9), (6, 8, 10)] # shoulder, elbow, wrist
20 LEG_CHAINS = [(11, 13, 15), (12, 14, 16)] # hip, knee, ankle
21 LIMBS = ("left_arm", "right_arm", "left_leg", "right_leg")
22
23
24 # @dataclass writes __init__ and printing for us: LimbPose(30.0, -10.0)
25 @dataclass
26 class LimbPose:
27     """Leg command from one body limb. Degrees, 'outward' convention (see config.LEGS)."""
28     thigh: float # shoulder/hip angle vs torso: 0 = hanging down, + = away from body
29     bend: float # elbow/knee angle: lower segment direction - upper segment direction
30
31
32 @dataclass
33 class PlayerState:
34     """Everything the game reads about one player, refreshed every camera frame."""
35     visible: bool = False
36     limbs: dict = field(default_factory=dict) # limb name -> LimbPose
37     mouth_open: bool = False
38     mouth_score: float = 0.0
39     face_found: bool = False
40     sticky_side: str | None = None # "left"/"right": only that side's limb sticks, None = all
41     head_tilt: float = 0.0 # deg, sideways head roll; > 0 = tilted to the right (1 player)
42
43
44 def _heading(v):
45     """Angle of a screen vector measured from 'down', positive towards +x."""
46     # atan2(x, y) instead of the usual atan2(y, x): measures from straight down
47     # (screen y grows downward), so an arm hanging down gives 0 degrees.
48     return math.degrees(math.atan2(v[0], v[1]))
49
50
51 def _wrap(a):
52     """Bring an angle into -180..180."""
53     return (a + 180.0) % 360.0 - 180.0
54
55
56 def limb_chains(xy):
57     """limb name -> keypoint chain, by screen side (left = smaller x)."""
58     # Sides come from where the joints are on screen, not from the model's
59     # "left/right" labels: on a mirrored picture those labels can be swapped.
60     arms = sorted(ARM_CHAINS, key=lambda c: xy[c[0]][0])
61     legs = sorted(LEG_CHAINS, key=lambda c: xy[c[0]][0])
62     return dict(zip(LIMBS, arms + legs))
63
64
65 def limb_poses(xy, conf, thr=None):
66     """limb name -> LimbPose for every limb whose three keypoints are visible."""
67     if thr is None:
68         thr = config.KEYPOINT_THR # read now (not at import) so the Options screen can change it
69     ok = conf > thr # True for every keypoint the model is sure enough about
70     if not (ok[L_SH] and ok[R_SH]):

```

```

71     return {}
72     # torso axis: shoulders -> hips, or straight down if hips are out of frame
73     if ok[L_HIP] and ok[R_HIP]:
74         down = (xy[L_HIP] + xy[R_HIP]) / 2 - (xy[L_SH] + xy[R_SH]) / 2
75     else:
76         down = np.array([0.0, 1.0])
77     torso = _heading(down) # leaning the whole body should not swing the legs
78
79     out = {}
80     for name, (a, b, c) in limb_chains(xy).items(): # a = shoulder, b = elbow, c = wrist
81         if not (ok[a] and ok[b] and ok[c]):
82             continue
83         # side = -1 for left limbs: mirror the angles so "away from the body" is
84         # positive on both sides (the same convention as the character's legs)
85         side = -1 if name.startswith("left") else 1
86         upper = side * _wrap(_heading(xy[b] - xy[a]) - torso) # upper arm direction vs torso
87         lower = side * _wrap(_heading(xy[c] - xy[b]) - torso) # forearm direction vs torso
88         out[name] = LimbPose(upper, _wrap(lower - upper)) # bend = forearm relative to upper arm
89     return out
90
91
92 @dataclass
93 class AngleSmoother:
94     """EMA on angles (wrap-safe) so jittery keypoints don't shake the legs."""
95     alpha: float | None = None # None = use config.ANGLE_EMA (read live, the Options screen changes it)
96     value: LimbPose | None = None
97
98     def update(self, new):
99         """Exponential moving average: move a fraction `alpha` of the way to the new
100         measurement. Wrapping the difference keeps 179 -> -179 a 2 degree step,
101         not a 358 degree swing. A missing measurement keeps the last value."""
102         if new is None:
103             return self.value
104         a = config.ANGLE_EMA if self.alpha is None else self.alpha
105         if self.value is None:
106             self.value = LimbPose(new.thigh, new.bend)
107         else:
108             self.value = LimbPose(
109                 _wrap(self.value.thigh + a * _wrap(new.thigh - self.value.thigh)),
110                 _wrap(self.value.bend + a * _wrap(new.bend - self.value.bend))
111             )
112         return self.value

```

preview.py (85 lines)

Draws the annotated camera picture.

Lines	What happens there
1-7	Docstring and imports.
9-15	Which points to connect: body skeleton, lips, leg labels.
18-60	Grey skeleton for everybody, controlling limbs thick and coloured, player label, lips.
61-68	Lip outline, yellow and thicker when open.
69-85	FPS line and one status line per player.

Full source

```

1  """Draws the camera preview: grey bodies, controlling limbs in the player's colour, mouths only."""
2  import cv2
3  import numpy as np
4  from mediapipe.tasks.python.vision import FaceLandmarksConnections
5
6  import config
7  from player_state import limb_chains
8
9  # pairs of COCO keypoints joined by a line (the stick figure)
10 BODY_EDGES = [(0, 1), (0, 2), (1, 3), (2, 4), (5, 6), (5, 11), (6, 12), (11, 12),
11               (5, 7), (7, 9), (6, 8), (8, 10), (11, 13), (13, 15), (12, 14), (14, 16)]
12 # pairs of face-mesh points that outline the lips (MediaPipe provides this list)
13 LIPS = [(c.start, c.end) for c in FaceLandmarksConnections.FACE_LANDMARKS_LIPS]
14 LEG_LABELS = {"upper_left": "top-left", "upper_right": "top-right",
15              "lower_left": "bottom-left", "lower_right": "bottom-right"}
16
17
18 class PreviewRenderer:
19     backend = "" # shown next to the FPS, e.g. "TensorRT"
20
21     def draw(self, frame, persons, states, scheme, fps):
22         img = frame.copy() # never draw on the frame the detectors use
23         ok_thr = config.KEYPOINT_THR
24         for p in persons:
25             ok = p.conf > ok_thr
26             pts = p.xy.astype(int) # OpenCV draws at whole pixels
27             grey = config.BODY_COLOR_BGR
28             # 1) the whole body in grey
29             for i, j in BODY_EDGES:
30                 if ok[i] and ok[j]:
31                     cv2.line(img, tuple(pts[i]), tuple(pts[j]), grey, 2, cv2.LINE_AA)
32             for k in np.nonzero(ok)[0]:
33                 cv2.circle(img, tuple(pts[k]), 3, grey, -1)
34
35             # 2) on top: only the limbs this player controls, thick and coloured
36             if p.player is not None:
37                 col = config.PLAYER_COLORS_BGR[p.player]
38                 chains = limb_chains(p.xy)
39                 for player, limb, _ in scheme:
40                     if player != p.player:
41                         continue
42                     a, b, c = chains[limb]
43                     for i, j in ((a, b), (b, c)):
44                         if ok[i] and ok[j]:
45                             cv2.line(img, tuple(pts[i]), tuple(pts[j]), col, 5, cv2.LINE_AA)
46                     for k in (a, b, c):
47                         if ok[k]:
48                             cv2.circle(img, tuple(pts[k]), 5, col, -1)
49                 head = p.head_center()
50                 if head is not None:
51                     cv2.putText(img, f"P{p.player + 1}", (int(head[0]) - 15, int(head[1]) - 50),
52                               cv2.FONT_HERSHEY_SIMPLEX, 0.8, col, 2)
53             # 3) the mouth outline (the only part of the face mesh that is shown)
54             if p.face is not None:
55                 self._draw_mouth(img, p, states)
56
57         self._draw_hud(img, states, scheme, fps)
58         return img
59
60     @staticmethod
61     def _draw_mouth(img, p, states):
62         face = p.face.astype(int)
63         is_open = p.player is not None and states[p.player].mouth_open
64         col = (0, 255, 255) if is_open else (255, 255, 255) # yellow when open (BGR)
65         for i, j in LIPS:
66             cv2.line(img, tuple(face[i]), tuple(face[j]), col, 2 if is_open else 1, cv2.LINE_AA)
67
68     @staticmethod
69     def _draw_hud(img, states, scheme, fps):
70         """Text in the top-left corner: FPS and one status line per player."""

```

```

71     y = 25
72     cv2.putText(img, f"{fps:.0f} FPS {PreviewRenderer.backend}", (10, y),
73                 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)
74     for i, st in enumerate(states):
75         y += 25
76         legs = ", ".join(LEG_LABELS[leg] for pl, _, leg in scheme if pl == i)
77         if not st.visible:
78             status = "not found"
79         else:
80             mouth = "OPEN (sticky)" if st.mouth_open else "closed"
81             status = f"mouth {mouth}" if st.face_found else "no face"
82             if st.sticky_side is not None:
83                 status += f" | sticks {st.sticky_side.upper()} (roll {st.head_tilt:+.0f} deg)"
84     cv2.putText(img, f"P{i + 1} [{legs}]: {status}", (10, y),
85                 cv2.FONT_HERSHEY_SIMPLEX, 0.55, config.PLAYER_COLORS_BGR[i], 2)

```

export_engine.py (17 lines)

Builds the TensorRT engine for the pose model (see Part V).

Lines	What happens there
1-8	Docstring: when and why to run it.
9-11	Imports.
13-17	Export with half precision at 640 px on GPU 0.

Full source

```

1  """Build the TensorRT engine for the pose model (takes a few minutes, needs an NVIDIA GPU).
2
3      python export_engine.py
4
5  The .engine only works on the GPU / driver / TensorRT version it was built with,
6  so run this again on another computer or after updating them.
7  Needs: pip install tensorrt-cu13 (and the CUDA build of torch)
8  """
9  from ultralytics import YOLO
10
11  import config
12
13  if __name__ == "__main__":
14      # format="engine": TensorRT. quantize=16: half-precision numbers (faster, tiny
15      # accuracy loss). imgsz=640: the picture size the network works on. device=0: first GPU.
16      path = YOLO(config.POSE_MODEL).export(format="engine", quantize=16, imgsz=640, device=0)
17      print("saved", path)

```

download_models.py (45 lines)

Run by the setup scripts: downloads both models and tells you whether the GPU will be used.

Lines	What happens there
1-11	Docstring and imports.
14-15	Work in the project folder (where the game looks for the models).
17-21	Download the face model if missing.
23-25	Loading YOLO once downloads the pose model.
27-41	Report: GPU (and TensorRT engine), NVIDIA GPU that PyTorch cannot use (driver too old), or CPU.
44-45	Run main() when started directly.

Full source

```

1  """Download the two AI models and report what hardware will be used.
2
3  Run by the setup scripts (setup.ps1 / setup.sh) at the end; safe to run again.
4  python download_models.py
5  """
6  import importlib.util
7  import os
8  import shutil
9  import urllib.request
10
11 import config
12
13
14 def main():
15     os.chdir(config.ROOT)          # the game loads the models from the project folder
16
17     if os.path.exists(config.FACE_MODEL):
18         print("face model: already there")
19     else:
20         print("face model: downloading face_landmarker.task ...")
21         urllib.request.urlretrieve(config.FACE_URL, config.FACE_MODEL)
22
23     from ultralytics import YOLO    # downloads yolo26n-pose.pt on first use
24     YOLO(config.POSE_MODEL)
25     print(f"pose model: {config.POSE_MODEL} ready")
26
27     import torch
28     if torch.cuda.is_available():
29         print(f"GPU: {torch.cuda.get_device_name(0)} - pose detection runs on the GPU")
30         engine = os.path.exists(os.path.splitext(config.POSE_MODEL)[0] + ".engine")
31         trt = importlib.util.find_spec("tensorrt") is not None
32         if engine and trt:
33             print("TensorRT engine found - fastest mode")
34         elif engine:
35             print("(a TensorRT engine file exists, but TensorRT is not installed here - it is not used)")
36     elif shutil.which("nvidia-smi"):
37         print("NVIDIA GPU found but PyTorch cannot use it - the game will run on the CPU.\n"
38             "  Usually the NVIDIA driver is too old for CUDA 13: update it to version 580 or newer,\n"
39             "  then run the setup again.")
40     else:
41         print("no NVIDIA GPU - pose detection runs on the CPU (works, about 9 detections/s)")
42
43
44 if __name__ == "__main__":
45     main()

```

setup.ps1 (99 lines)

Windows setup (double-click **setup.bat**, which runs this with permission to execute scripts).

Lines	What happens there
1-6	What it does and how to start it.
7-9	Options -TensorRT / -NoTensorRT (skip the question); stop on errors; work in the script's folder.
11-14	Helpers: print a step title; stop with a message if the last command failed.
16	The pinned PyTorch versions.
18-50	Find Python 3.12 (py -3.12, python, or the default install folder); if missing, offer winget install Python.Python.3.12 .
52-64	Rebuild .venv if it was made with another Python version; create it (or reuse it); update pip.
66-76	PyTorch: CUDA build if nvidia-smi exists, else the CPU build.
78-79	The game's packages from requirements.txt.
81-93	Optional TensorRT: ask (or use the option), install, build the engine.
95-96	Download the models and report GPU/CPU.
98-99	Tell the user how to start the game.

Full source

```

1  # All For One - one-time setup for Windows.
2  # Easiest: double-click setup.bat. Or in PowerShell, from the project folder:
3  # powershell -ExecutionPolicy Bypass -File setup.ps1 [-TensorRT | -NoTensorRT]
4  # Creates .venv (a private Python 3.12 environment for the game), installs the pinned
5  # packages, picks the CUDA (NVIDIA) or CPU build of PyTorch and downloads the models.
6  # If Python 3.12 is missing it offers to install it with winget.
7  param([switch]$TensorRT, [switch]$NoTensorRT)
8  $ErrorActionPreference = "Stop"
9  Set-Location $PSScriptRoot
10
11 function Step($text) { Write-Host "`n== $text" -ForegroundColor Cyan }
12 function Check($what) {
13     if ($LASTEXITCODE -ne 0) { Write-Host "FAILED: $what" -ForegroundColor Red; exit 1 }
14 }
15
16 $TorchVersion = "torch==2.14.1", "torchvision==0.29.1"
17
18 Step "1/6 Looking for Python 3.12"
19 # The game needs exactly Python 3.12: newer versions do not have packages for
20 # mediapipe / PyTorch yet. Other Python versions on the computer are left alone.
21 $Is312 = "import sys; sys.exit(0 if sys.version_info[:2] == (3, 12) else 1)"
22 function Find-Python312 {
23     $default = Join-Path $env:LOCALAPPDATA "Programs\Python\Python312\python.exe" # python.org / winget
24     foreach ($cand in @("py", "-3.12"), @("python"), @("python3"), @($default))) {
25         try {
26             $exe, $rest = $cand
27             & $exe @rest -c $Is312 2>$null
28             if ($LASTEXITCODE -eq 0) { return , $cand }
29         } catch { }
30     }
31     return $null
32 }
33 $py = Find-Python312
34 if (-not $py -and (Get-Command winget -ErrorAction SilentlyContinue)) {
35     Write-Host "Python 3.12 is not installed. It can be added next to any other Python versions."
36     $answer = Read-Host "Install Python 3.12 now with winget? [y/N]"
37     if ($answer -match '^[yY]') {
38         winget install -e --id Python.Python.3.12 --scope user --accept-package-agreements
39         $py = Find-Python312
40     }
41 }
42 if (-not $py) {
43     Write-Host "Python 3.12 not found." -ForegroundColor Red
44     Write-Host "Install it from https://www.python.org/downloads/ (3.12.x), or run:" -ForegroundColor Red
45     Write-Host "    winget install -e --id Python.Python.3.12" -ForegroundColor Red
46     Write-Host "then run the setup again." -ForegroundColor Red
47     exit 1
48 }
49 $exe, $rest = $py
50 & $exe @rest --version
51
52 Step "2/6 Creating the environment in .venv"
53 if (Test-Path ".venv\Scripts\python.exe") {
54     & ".venv\Scripts\python.exe" -c $Is312 2>$null
55     if ($LASTEXITCODE -ne 0) {
56         Write-Host ".venv was made with another Python version - rebuilding it with Python 3.12"
57         Remove-Item -Recurse -Force ".venv"
58     }
59 }
60 if (-not (Test-Path ".venv\Scripts\python.exe")) {
61     & $exe @rest -m venv .venv; Check "creating .venv"
62 } else { Write-Host ".venv already exists - updating it" }
63 $vpy = Join-Path $PSScriptRoot ".venv\Scripts\python.exe"
64 & $vpy -m pip install --upgrade pip --quiet; Check "upgrading pip"
65
66 Step "3/6 Installing PyTorch"
67 $gpu = $null -ne (Get-Command nvidia-smi -ErrorAction SilentlyContinue)
68 if ($gpu) {
69     Write-Host "NVIDIA GPU found - installing the CUDA build (large download, a few minutes)"
70     & $vpy -m pip install @TorchVersion --index-url https://download.pytorch.org/whl/cu130

```

```

71     Check "installing PyTorch (CUDA)"
72 } else {
73     Write-Host "No NVIDIA GPU - installing the small CPU build"
74     & $vpy -m pip install @TorchVersion --index-url https://download.pytorch.org/whl/cpu
75     Check "installing PyTorch (CPU)"
76 }
77
78 Step "4/6 Installing the game's packages"
79 & $vpy -m pip install -r requirements.txt; Check "installing requirements.txt"
80
81 Step "5/6 Optional: TensorRT (about 2x faster pose detection)"
82 $wantTrt = $false
83 if ($gpu -and -not $NoTensorRT) {
84     if ($TensorRT) { $wantTrt = $true }
85     else {
86         $answer = Read-Host "Install TensorRT and build the engine? Takes ~10 minutes [y/N]"
87         $wantTrt = $answer -match '^[yY]'
88     }
89 }
90 if ($wantTrt) {
91     & $vpy -m pip install -r requirements-gpu.txt; Check "installing TensorRT"
92     & $vpy export_engine.py; Check "building the TensorRT engine"
93 } else { Write-Host "skipped (the game works without it)" }
94
95 Step "6/6 Downloading the AI models"
96 & $vpy download_models.py; Check "downloading the models"
97
98 Write-Host "`nDone! Start the game with: run.bat" -ForegroundColor Green
99 Write-Host "(or: .venv\Scripts\python.exe main.py)" -ForegroundColor Green

```

setup.sh (119 lines)

Linux setup. Same steps as setup.ps1 plus Linux checks (venv package, OpenCV system library, webcam permission).

Lines	What happens there
1-6	Run with bash; what it does and how to start it.
7-8	Stop on any error, unset variable or failed pipe; work in the script's folder.
10-17	Read the --tensorrt / --no-tensorrt options.
18-20	Pinned PyTorch versions and print helpers.
22-58	Find Python 3.12: python3.12/python3/python , else with uv if installed, else offer to install uv (or show the deadsnakes alternative) and stop.
60-72	Rebuild .venv if it was made with another Python version; create it; if that fails, explain the missing python3.12-venv package.
74-83	PyTorch: CUDA build if an NVIDIA GPU answers nvidia-smi -L , else the CPU build.
85-90	Game packages; check that OpenCV can load (libGL) and explain the fix if not.
92-106	Optional TensorRT: ask only in an interactive terminal.
108-109	Download the models and report GPU/CPU.
111-117	Warn if there is no /dev/video* device or no permission to read it.
119	Tell the user how to start the game.

Full source

```

1  #!/usr/bin/env bash
2  # All For One - one-time setup for Linux (x86-64 PC). From the project folder:
3  #     ./setup.sh [--tensorrt | --no-tensorrt]
4  # Creates .venv (a private Python 3.12 environment for the game), installs the pinned
5  # packages, picks the CUDA (NVIDIA) or CPU build of PyTorch and downloads the models.
6  # If Python 3.12 is missing (e.g. Ubuntu with 3.14) it can get it with uv, without sudo.
7  set -euo pipefail
8  cd "$(dirname "$0")"
9
10 TRT=ask
11 for arg in "$@"; do
12     case "$arg" in
13         --tensorrt) TRT=yes ;;
14         --no-tensorrt) TRT=no ;;
15         *) echo "unknown option: $arg"; exit 1 ;;
16     esac
17 done
18 TORCH=(torch==2.14.1 torchvision==0.29.1)
19 step() { printf '\n\033[36m== %s\033[0m\n' "$1"; }
20 fail() { printf '\033[31m%s\033[0m\n' "$1"; exit 1; }
21
22 step "1/6 Looking for Python 3.12"
23 # The game needs exactly Python 3.12: newer versions (e.g. Ubuntu's 3.14) do not have
24 # packages for mediapipe / PyTorch yet. The system Python is never changed.
25 is312() { "$1" -c 'import sys; sys.exit(0 if sys.version_info[:2] == (3, 12) else 1)' 2>/dev/null; }
26 uv_python() { # uv downloads its own Python 3.12 into the home folder (no sudo)
27     uv python install 3.12 && PY="$(uv python find 3.12)"
28 }
29 PY=""
30 for cand in python3.12 python3 python; do
31     if command -v "$cand" >/dev/null 2>&1 && is312 "$cand"; then PY="$(command -v "$cand")"; break; fi
32 done
33 if [ -z "$PY" ] && command -v uv >/dev/null 2>&1; then
34     echo "Python 3.12 is not installed - getting it with uv"
35     uv_python
36 fi
37 if [ -z "$PY" ]; then
38     echo "Python 3.12 is not installed (this computer has: $(python3 --version 2>&1))."
39     echo "It can be installed for this user only with 'uv' (https://docs.astral.sh/uv/),"
40     echo "without sudo and without changing the system Python."
41     answer=n
42     if [ -t 0 ]; then read -r -p "Install uv and Python 3.12 now? [y/N] " answer; fi
43     case "$answer" in
44         [yY]*)
45         if command -v curl >/dev/null 2>&1; then curl -LsSf https://astral.sh/uv/install.sh | sh
46         else wget -qO- https://astral.sh/uv/install.sh | sh; fi
47         export PATH="$HOME/.local/bin:$PATH"
48         uv_python ;;
49     *)
50         fail "Install Python 3.12 one of these ways, then run ./setup.sh again:
51 a) uv (any Linux, no sudo): curl -LsSf https://astral.sh/uv/install.sh | sh
52 then: ~/.local/bin/uv python install 3.12
53 b) Ubuntu (deadsnakes): sudo add-apt-repository ppa:deadsnakes/ppa
54 sudo apt install python3.12 python3.12-venv" ;;
55     esac
56 fi
57 [ -n "$PY" ] && is312 "$PY" || fail "Could not get Python 3.12."
58 "$PY" --version
59
60 step "2/6 Creating the environment in .venv"
61 if [ -x .venv/bin/python ] && ! is312 .venv/bin/python; then
62     echo ".venv was made with $(.venv/bin/python --version 2>&1) - rebuilding it with Python 3.12"
63     rm -rf .venv
64 fi
65 if [ ! -x .venv/bin/python ]; then
66     "$PY" -m venv .venv || fail "Could not create .venv. On Ubuntu/Debian: sudo apt install python3.12-venv
67 then run ./setup.sh again."
68 else
69     echo ".venv already exists - updating it"
70 fi

```

```

71 VPY=.venv/bin/python
72 "$VPY" -m pip install --upgrade pip --quiet
73
74 step "3/6 Installing PyTorch"
75 GPU=no
76 if command -v nvidia-smi >/dev/null 2>&1 && nvidia-smi -L >/dev/null 2>&1; then GPU=yes; fi
77 if [ "$GPU" = yes ]; then
78     echo "NVIDIA GPU found - installing the CUDA build (large download, a few minutes)"
79     "$VPY" -m pip install "${TORCH[@]}" --index-url https://download.pytorch.org/whl/cu130
80 else
81     echo "No NVIDIA GPU - installing the small CPU build"
82     "$VPY" -m pip install "${TORCH[@]}" --index-url https://download.pytorch.org/whl/cpu
83 fi
84
85 step "4/6 Installing the game's packages"
86 "$VPY" -m pip install -r requirements.txt
87 if ! "$VPY" -c "import cv2" 2>/dev/null; then
88     fail "OpenCV cannot load a system library. On Ubuntu/Debian: sudo apt install libgl1 libglv2.0-0
89 then run ./setup.sh again."
90 fi
91
92 step "5/6 Optional: TensorRT (about 2x faster pose detection)"
93 if [ "$GPU" = yes ] && [ "$TRT" = ask ]; then
94     if [ -t 0 ]; then
95         read -r -p "Install TensorRT and build the engine? Takes ~10 minutes [y/N] " answer
96         case "$answer" in [yY]*) TRT=yes ;; *) TRT=no ;; esac
97     else
98         TRT=no
99     fi
100 fi
101 if [ "$GPU" = yes ] && [ "$TRT" = yes ]; then
102     "$VPY" -m pip install -r requirements-gpu.txt
103     "$VPY" export_engine.py
104 else
105     echo "skipped (the game works without it)"
106 fi
107
108 step "6/6 Downloading the AI models"
109 "$VPY" download_models.py
110
111 # webcam check: the camera is a /dev/video* device the user must be allowed to read
112 if ! ls /dev/video* >/dev/null 2>&1; then
113     printf '\033[33mNo webcam found (/dev/video*). Plug one in, or play with: ./run.sh --keyboard\033[0m\n'
114 elif [ ! -r /dev/video0 ]; then
115     printf '\033[33mYou may not have permission to use the webcam. Run:\n'
116     printf '  sudo usermod -aG video %s\nthen log out and back in.\033[0m\n' "$USER"
117 fi
118
119 printf '\n\033[32mDone! Start the game with: ./run.sh\033[0m\n'

```

requirements.txt (13 lines)

The pinned versions of the game's packages (PyTorch is installed separately by the setup scripts).

Lines	What happens there
1-5	Why the versions are pinned and why PyTorch is not in this list.
6-7	pygame and numpy with their exact tested versions.
8-11	Both OpenCV packages at one version (they share the cv2 module).
12-13	The two AI libraries.

Full source

```

1 # Packages the game needs, pinned to versions the game was tested with.
2 # Use the setup script instead of installing this by hand:
3 #   Windows: .\setup.ps1   Linux: ./setup.sh
4 # The setup script installs PyTorch FIRST (CUDA build on NVIDIA computers, small
5 # CPU build otherwise), because ultralytics would otherwise pull a default build.
6 pygame==2.6.1
7 numpy==2.5.3
8 # Both OpenCV packages install the same cv2 module (mediapipe wants "contrib", ultralytics
9 # the plain one); they MUST have the same version or they overwrite each other.
10 opencv-python==4.11.0.86
11 opencv-contrib-python==4.11.0.86
12 mediapipe==1.0.1
13 ultralytics==8.4.170

```

Part IV - Reference

14. Every setting in config.py

Read straight from the file when this guide was built. Grey rows are the section headers of the file.

Setting	Value	Meaning
ROOT	<code>os.path.dirname(os.path.abspath(__file__))</code>	
ASSET_DIR	<code>os.path.join(ROOT, "assets")</code>	
[maps]		
MAP_HEIGHT	2164	
MAPS	(several lines - see the listing)	
SPAWN_LIFT	1 / 8	start this far (MAP_HEIGHT fraction) above the floor, or mid-gap if lower
[map / view]		
VIEW_FRACTION	1 / 3	game view height = 1/3 of the map height
VIEW_ASPECT	4096 / 2164	view width / height (the mountain map's shape)
WINDOW_SCALE	1.0	max scale of the window (it also shrinks to fit the screen)
LAYOUT_MARGIN	0.025	gap between window edge / game / camera
LAYOUT_CAMERA_WIDTH	0.40	camera panel width
LAYOUT_SCREEN_FILL	0.92	window may use at most this fraction of the desktop
LAYOUT_BG	(40, 42, 48)	dark grey around the two panels
START_FULLSCREEN	False	F11 toggles fullscreen at any time
FPS	60	frames drawn per second (the loop waits to keep this rate)
PHYSICS_SUBSTEPS	3	physics steps per rendered frame (stability)
CAMERA_LERP	6.0	how fast the game view follows the character (1/s)
SKY_COLOR	(135, 206, 235)	fills transparent gaps behind the map images
[character size]		
CHARACTER_FRACTION	1 / 8	
SPRITE_CANVAS	500	size of the character png canvases
HEAD_CENTER	(250, 250)	
HEAD_RADIUS	95	collision circle of the head
LEG_PIVOT	(249.5, 190.5)	joint at the top of leg-part.png / leg-part-with-shoe.png
SEGMENT_LENGTH	117	pivot -> pivot of leg-part.png
FOOT_OFFSET	(20, 160)	shoe centre from LEG_PIVOT while the sprite hangs down (toe = +x)
FOOT_RADIUS	30	collision circle of the shoe
LEGS	(several lines - see the listing)	
MIRRORED_SHOES	<code>{"upper_left", "upper_right"}</code>	

Setting	Value	Meaning
CONTROL_SCHEMES	(several lines - see the listing)	
MAX_PLAYERS	4	highest player count offered in the menu
LEG_STRETCH	{1: 1.5}	
SHOE_TOP	300	canvas y where the shoe starts on leg-part-with-shoe.png
[flag]		
FLAG_SEARCH	0.1	fraction of the map width searched for the peak
FLAG_FRACTION	1 / 9	flag.png canvas height relative to MAP_HEIGHT
FLAG_BASE	(160, 492)	canvas px of flag.png that sits on the ground (pole foot)
[physics]		
GRAVITY	2000.0	px/s ² (map pixels)
MAX_SPEED	1500.0	px/s, speed limit while touching; falling speed limit in the air
AIR_DRAG	0.3	1/s
LEG_MAX_SPEED	420.0	deg/s, how fast a leg joint can follow the arm
SOLVER_ITERATIONS	4	passes over all contacts per physics step (more = stiffer, slower)
WALKABLE_SLOPE	55.0	deg; feet only get friction on surfaces flatter than this
ROTOR_WALKABLE_SLOPE	10.0	deg; same for the windmill rotor: slippery (sticky feet still glue)
ROTOR_PUSH_SPEED	400.0	px/s; a blade can push a free foot/head away at most this much faster than it moves
CONTACT_EPS	2.0	px; a foot this close to the ground still counts as touching
STICKY_GRAB_DIST	8.0	px; a sticky foot grabs ground within this distance
STICKY_BREAK_DIST	40.0	px; a sticky foot tears off if pulled this far from its anchor
STICKY_STIFFNESS	0.6	share of a glued foot's error fixed per solver pass (1 = all)
LEG_BLOCK_TOLERANCE	3.0	px a free foot may sink into rock while another foot is glued
STICKY_SINK	6.0	px a sticky/glued foot may sink into rock while another foot is glued
HEAD_FRICTION	6.0	1/s, damping while the head scrapes the ground
[vision]		
CAMERA_SOURCE	"0"	webcam index or video path
POSE_MODEL	"yolo26n-pose.pt"	YOLO "nano" pose model, downloaded automatically
USE_TENSORRT	True	
FACE_MODEL	os.path.join(ROOT, "face_landmarker.task")	MediaPipe face model file
FACE_URL	("https://storage.googleapis.com/mediapipe-mod...	
MIRROR_CAMERA	True	flip the webcam so it behaves like a mirror

Setting	Value	Meaning
KEYPOINT_THR	0.5	keypoints with lower confidence (0..1) count as "not seen"
FACE_MATCH_FACTOR	1.0	max face-to-head distance = factor * shoulder width
FACE_MATCH_MIN_PX	40	
FACE_CROP_FALLBACK	True	if the full-frame face search misses a player, retry on a head crop
CROP_SCALE	2.4	head crop size = spread of nose/eyes/ears * this
CROP_DOWN	0.25	move the crop down (in spreads) so chin and mouth are inside
MIN_CROP	24	px; smaller heads are skipped (too few pixels for a face mesh)
UPSCALE_TO	256	small crops are enlarged to this before the face model
MOUTH_OPEN_T, MOUTH_CLOSE_T	0.35, 0.25	hysteresis on lip gap / lip width
MOUTH_EMA	0.5	smoothing of the mouth score (1 = no smoothing)
MOUTH_TIMEOUT	0.5	s; forget the mouth state if the face is lost for this long
HEAD_TILT_PLAYERS	{1}	player counts that use this
HEAD_TILT_ANGLE	10.0	deg the ear line must tilt to switch side (in between: keep last)
HEAD_TILT_INVERT	False	True: swap (tilt right -> left-arm leg)
ANGLE_EMA	0.6	smoothing of arm angles (1 = no smoothing)
PLAYER_COLORS_BGR	[(0, 0, 255), (0, 220, 0), (255, 120, 0), (0, ...	
PLAYER_COLORS_RGB	[tuple(reversed(c)) for c in PLAYER_COLORS_BGR...	
BODY_COLOR_BGR	(150, 150, 150)	grey for non-controlling body parts in the preview
[fonts]		
FONT_HEAVY	"arialblack,arial,dejavusans,liberationssans,fr...	titles, buttons
FONT_TEXT	"arial,dejavusans,liberationssans,freesans"	normal text
FONT_MONO	"consolas,couriernew,dejavusansmono,liberation...	numbers, timer
FONT_EMOJI	"segoeuiemoji,notocoloremoji,notoemoji,symbola...	the sheep in the menu
[menu]		
GAME_TITLE	"All For One"	
GAME_CREDIT	"From The Group MeEeEe"	
GAME_CREDIT_EMOJI	"[sheep emoji]"	sheep

15. How do I...?

Most player-facing values can be changed in the **OPTIONS** screen without touching code; the table lists where everything lives in the code.

I want to...	Change this
set the game up on a new computer	Windows: <code>setup.bat</code> . Linux: <code>./setup.sh</code> . Then <code>run.bat</code> / <code>./run.sh</code> .
tune mouth, legs, physics while playing	The OPTIONS screen (saved in <code>settings.json</code>).
offer a new value in the Options screen	Add a Setting to SETTINGS in <code>settings.py</code> with a safe range. The code must read that config value when it uses it (not copy it at start-up).
rebuild this PDF after code changes	<code>pip install -r requirements-dev.txt</code> once, then <code>python tools/docs/make_docs.py</code> . Text is in <code>tools/docs/content.py</code> .
update a package version	Change the pin in <code>requirements.txt</code> , run the setup again, test.
add a map	Put <code>background-x.png</code> and <code>foreground-x.png</code> (transparent = air) in <code>assets/</code> and add an entry to MAPS in <code>config.py</code> with a spawn point (fractions of the map) and optionally a <code>scale</code> . The flag and the menu picture are automatic.
add a windmill to a map	Add a <code>"windmill"</code> entry (tower and rotor pictures, hub in image pixels, <code>rps</code>) to the map in MAPS , like the Windmill map.
change the rotor speed / slipperiness	<code>"rps"</code> of the map in MAPS / <code>ROTOR_WALKABLE_SLOPE</code> .
clear the score board	Delete <code>scores.json</code> (or remove lines from it).
make jumps stronger / weaker	<code>LEG_MAX_SPEED</code> (how fast legs straighten) or <code>GRAVITY</code> .
change who controls which leg	<code>CONTROL_SCHEMES</code> - each row is (player, limb, leg).
make the mouth easier to trigger	Lower <code>MOUTH_OPEN_T</code> (and keep <code>MOUTH_CLOSE_T</code> about 0.1 lower).
make legs smoother / more responsive	<code>ANGLE_EMA</code> (lower = smoother, slower).
change the character size	<code>CHARACTER_FRACTION</code> (relative to the map height).
make a map bigger relative to the character	That map's <code>"scale"</code> in MAPS .
change the player colours	<code>PLAYER_COLORS_BGR</code> (blue, green, red order!).
switch head-tilt direction	<code>HEAD_TILT_INVERT</code> .
start in fullscreen	<code>START_FULLSCREEN = True</code> .
use the camera without mirroring	<code>MIRROR_CAMERA = False</code> .

16. Glossary

Word	Meaning here
anchor	The point where a sticky foot is glued to the rock.
bilinear interpolation	Blending the 4 nearest table values by distance, for smooth results between pixels.
blit	Copy (paste) one picture onto another (pygame).
canvas	The fixed-size picture every scene draws on.
delta time (dt)	Seconds since the last frame/step; movement = speed x dt.
EMA	Exponential moving average: smooth = smooth + a x (new - smooth).
Gauss-Seidel	Fixing constraints one after another, repeated a few times.
hysteresis	Two thresholds (on/off) with a gap so a value near the edge does not flicker.

Word	Meaning here
inference	Running a trained AI model on new input.
keypoint / landmark	A named point found by a model (body joint / face point).
kinematics	Computing joint positions from angles and lengths.
lock	Makes one thread wait while another changes shared data.
normal	Unit vector pointing straight out of a surface.
pinned version	An exact package version in requirements.txt (==), so every install is the same.
PBD	Position-based dynamics: move, correct positions, derive velocity.
SDF	Signed distance field: distance to the nearest surface, negative inside.
substep	One of several smaller physics steps inside one frame.
tangent	Direction along a surface (normal turned 90 degrees).
thread	A part of the program that runs at the same time as others.
TensorRT / FP16	NVIDIA's optimiser for running networks on its GPUs / 16-bit numbers.
virtual environment (.venv)	A private Python installation for one project, with its own packages.

Part V - How the AI models work

This part explains the two models at the level needed to use them well, without the mathematics of training.

17. Pose detection: YOLO

What it does. Given a picture, `yolo26n-pose` returns for every person a box, a person-confidence and 17 keypoints (x, y and a confidence 0..1 for each). It is a *neural network*: millions of numbers (weights) learned by training on the COCO dataset, where people were labelled by hand with these 17 points.

How it works, roughly. YOLO stands for *You Only Look Once*: the whole picture goes through the network a single time and all people come out together, which is why it is fast enough for live video.

1. The frame is resized to fit 640 x 640 (padding the rest) and turned into numbers.
2. The *backbone* (stacked convolution layers) turns the picture into feature maps: coarse grids where each cell describes what is in its region - edges first, then shapes, then body parts.
3. The *neck* combines feature maps of different sizes, so both near (big) and far (small) people are found.
4. The *head* predicts, for each grid cell, whether a person is centred there, the box, and the 17 keypoint positions with their confidences.
5. Overlapping duplicates are removed, leaving one result per person. (Ultralytics designs the YOLO26 family to do this inside the network, without a separate clean-up step.)

The **n** means *nano*, the smallest and fastest size. Bigger sizes (s, m, l, x) are more accurate on small or far people but slower. **Keypoint confidence** drops for hidden or blurry joints; the game ignores keypoints below 0.5, and a limb with a missing joint simply keeps its last angle.

Speed: CPU, GPU and TensorRT

A network is mostly multiplication of large tables of numbers, which a graphics card does in parallel. With the CUDA build of PyTorch the model runs on the RTX GPU. **TensorRT** goes further: it compiles the network for *this specific* GPU - merging layers, choosing the fastest routines and using 16-bit numbers (FP16) instead of 32-bit. That is why the `.engine` file only works on the computer it was built on.

Backend	Pose model per frame	Whole vision step per frame
CPU	about 51 ms	about 110 ms (9 FPS)
GPU (CUDA PyTorch)	about 12 ms	about 33 ms
TensorRT FP16	about 5 ms	about 23 ms (40+ FPS)

Measured on the development laptop (RTX 4070 Laptop) on a sample photo; the rest of the vision step is mostly the face model, which runs on the CPU.

18. Face landmarks: MediaPipe Face Landmarker

What it does. For every face it returns **478 landmarks**: 468 points of a face mesh (outline, eyebrows, eyes, nose, lips) plus 10 iris points, each as a position relative to the picture. It can also return 52 *blendshape* scores such as `jawOpen` (your original test script printed this); the game uses the simpler lip-gap ratio instead, because it also works on the head-crop fallback.

How it works. It is two networks in a row:

1. A small, very fast **face detector** (BlazeFace) finds where faces are and how they are rotated.
2. Each face is cut out, straightened and resized, and a **landmark network** predicts the positions of all 478 points at once (it *regresses* the coordinates: the output layer simply is the list of x, y, z values).

In **VIDEO mode** (used on the full frame) it *tracks*: the landmarks of the previous frame tell it where the face is now, so the detector can often be skipped - faster and steadier. That is why it needs increasing timestamps. **IMAGE mode** (used for head crops) treats every picture independently, which is right there because consecutive crops can come from different people.

19. Limits to keep in mind

- **Distance**: the face model is meant for faces within about two metres; far away, faces get too small. The head-crop fallback enlarges small faces, which helps.
- **Light and blur**: dark rooms and fast arm movement lower keypoint confidence; limbs then hold their last angle.
- **Overlap**: players standing behind each other can swap numbers or hide limbs; standing side by side works best.
- **Side view**: a head turned far sideways hides one ear and much of the mouth; head tilt falls back to the eyes, the mouth may be lost briefly (kept for 0.5 s).
- **Mirroring**: the model's own left/right labels can be wrong on a mirrored picture - the code uses screen positions instead (chapter 9).

Everything in this guide refers to the code as it is in the repository at the time of writing. The tables in Part III find their line numbers automatically from the real files, so they match the listings.